



AMERICAN INTERNATIONAL UNIVERSITY – BANGLADESH (AIUB)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

COURSE TITLE

INTRODUCTION TO DATA SCIENCE

PROJECT TITLE

Predicting CO₂ Emissions in Asia Using Machine Learning

COURSE INSTRUCTOR: KAMRUN NAHAR KOLI

SECTION – K (GROUP 2)

Table of Contents

NAME	ID	CONTRIBUTION
Md. Towhidul Islam	23-55036-3	Z-score Normalization, SMOTE Balancing, Train/Validation/Test Split, Introduction, Research Objective, Modeling, Web Scraping
Abrar Kabir	23-55095-3	Outlier Detection (IQR & Z-score), Winsorization, Missing Value Handling, Merging Datasets, Modeling, Web Scraping
Tasnim Hassan Ahona	23-55085-3	Correlation Heatmaps (Pearson & Spearman), ANOVA, Chi-Square Test, Data Collection & Web Scraping, Data Import & Loading, Feature Selection
Anika Tabassum	23-55070-3	EDA Visualizations, Histogram & Density Plots, Scatter Plots, Violin Plots, Descriptive Statistics, Class Distribution Analysis, Conclusion & Future Work

Table of Contents:

Introduction	3
1. Research Objective Definition	3
2. Data Collection (Web Scraping)	4
2.1 Data Sources	4
2.2 Web Scraping Methodology	5
2.3 Final Collected Dataset	11
3. Data Understanding and Exploration	11
3.1 Loading and Initial Inspection of Collected Datasets	11
3.2 Creating the Main Dataset by Merging	14
3.3 Exploratory Data Analysis Visualizations	16
4. Data Preprocessing	23
4.1 Data Cleaning, Missing Value Handling, Outlier Treatment, and Dataset Saving	23
4.2 Feature Engineering, Target Creation, and Feature Selection	25
4.3 Model Matrix Creation and Data Splitting	27
4.4 Scaling, SMOTE, and Cross-Validation Setup	29
5. Modeling and Analysis	32
5.1 Classification Models	32
5.2 Regression Models	37
5.3 Summary of Modeling and Analysis	42
6. Results and Interpretation	42
6.1 Classification Results	42
6.2 Regression Results	44
6.3 Model Interpretation	46
6.4 Achievement of Research Objective	48
7. Conclusion and Future Work	48
Appendix (Full Code)	49-79

Introduction

Asia has over half of the world's population and over 50% of global CO₂ emissions, so it is the most crucial region for climate change understanding and addressing. But there is huge variability in emissions between Asian countries, with oil-rich Gulf states contributing more than 30 tons of CO₂ per person per year, and dense, developing countries contributing less than one ton. The differences are influenced by a variety of factors such as energy use, urbanisation, economic development, population size, and demographic composition. It is important to know which of these factors plays the largest role in CO₂ emissions in order to be able to formulate suitable climate policy.

However, traditional statistical methods assume linear relationships between the predictors and outcome, which cannot be applied when the data involves complex and non-linear relationships, as is the case in socioeconomic and environmental fields. Machine learning approaches can, however, capture these non-linear patterns, manage interactions between the features and generate explanations of the results, like SHAP (SHapley Additive exPlanations). In this project, we will use the entire data science process to forecast per capita CO₂ emissions for Asian nations, including data acquisition, exploratory data analysis, model training, model evaluation and interpretation. CO₂ per capita is measured and not total emissions as it is a better comparison of the emissions between countries with very large population sizes.

1. Research Objective Definition

The primary objective of this project is: To predict whether an Asian country in a given year is a High CO₂ Emitter or Low CO₂ Emitter using socioeconomic, demographic, urbanization, and energy-related features.

In this project, the main target variable is CO₂ emissions per capita. For the classification task, each country-year observation is labelled as:

- **High CO₂ Emitter:** if CO₂ emissions per capita is above the 75th percentile of the dataset.
- **Low CO₂ Emitter:** if CO₂ emissions per capita is equal to or below the 75th percentile.

This makes the objective specific, measurable, and data-driven, because the target class is created using a clear numerical threshold.

Along with classification, this project also includes a regression objective:

To predict the actual CO₂ emissions per capita value of an Asian country in a given year.

Therefore, the project performs two machine learning tasks:

- **Classification:** to identify whether a country-year belongs to the High or Low CO₂ emitter group.
- **Regression:** to estimate the exact CO₂ emissions per capita value.

The independent variables used in this project include:

- Log-transformed population
- Urban population percentage
- GDP growth rate

- Energy consumption per capita
- Log-transformed energy consumption
- Population density
- Fertility rate
- Median age
- Engineered interaction features, such as energy consumption combined with urbanization

These features have been chosen because they are logically related to CO₂ emissions. Increased energy use per capita, for instance, typically results in increased emissions, and urbanization and economic activity can affect transport, industry and housing and energy demand.

The objective makes the use of the machine learning models appropriate as the connection between CO₂ emissions and socioeconomic factors could be complicated and non-linear. A classification model is used to detect the high emission case and a regression model is used to estimate the actual emission. Moreover, the feature importance and SHAP explainability is applied to interpret the model results and determine the most influential features that drive the CO₂ emissions.

2. Data Collection (Web Scraping)

For this project, data was collected from publicly available online sources using web scraping and open CSV datasets. The purpose of data collection was to gather country-level information related to CO₂ emissions, population, urbanization, GDP growth, and energy consumption for Asian countries.

2.1 Data Sources

The data for this project was collected from four public sources. Worldometer was used to collect population-related data such as population, urban population, fertility rate, median age, and density. Worldometer CO₂ Emissions was used to collect total CO₂ emissions, CO₂ emissions per capita, and CO₂ change percentage, where CO₂ per capita is the main target variable of this project. World Bank was used to collect GDP annual growth rate, which represents economic activity. Our World in Data (OWID) was used to collect energy consumption per capita, which is an important factor related to CO₂ emissions. These sources were selected because they provide demographic, economic, energy, and environmental data needed for analyzing and predicting CO₂ emissions across Asian countries.

Ethical and Legal Considerations

This project uses public data sources, which means that no login or payment is required to access all data used in this project. Worldometer clearly provides the data for public use and allows scraping of historical tables. The World Bank publishes its GDP info in an open data portal as a freely downloadable csv file. Our World in Data is an open-access research platform designed to provide all the data publicly for download and use. At no time any personally identifiable information was gathered. The delay between the successive HTTP requests was also added to the scraping script with the use of `sys.sleep(1)`, thus avoiding the scraping script from overwhelming the server with fast repeated requests and causing responsible scraping

behaviour. These practices follow the broad guidelines for web scraping: public data should be scraped, the server should be loaded and all protected and private data should not be accessed.

2.2 Web Scraping Methodology

The web scraping process was performed using R programming language. The main libraries used were:

- **rvest** - to read web pages and extract HTML tables
- **dplyr** - to clean and filter data
- **stringr** - to clean text and column names
- **purrr** - to loop through multiple country links
- **readr / base R** - to read and write CSV files

First, the Worldometer Asia population page was accessed, then the links for the countries were retrieved from the html table. Then each country page was visited to gather historical information related to population. The same approach was taken with CO₂ emissions data, which table of CO₂ emissions was scraped from Worldometer for each country.

The code used for scraping looked for the appropriate HTML tables by the column names Year, Population, Fossil CO₂ emissions and CO₂ emissions per capita. These tables were then compiled into one data set.

To ensure ethical scraping, a short delay was added between requests using:

Sys.sleep(1)

This helped avoid sending too many requests to the website at once.

Code for Scraping Population data:

```
# 2. Scrape Population Data

wanted_cols <- c(
  "country", "Year", "Population", "Yearly % Change",
  "Yearly Change", "Migrants (net)", "Median Age",
  "Fertility Rate", "Density (P/Km²)", "Urban Pop %",
  "Urban Population", "Country's Share of World Pop",
  "World Population", "Global Rank"
)

asia_url <- "https://www.worldometers.info/population/countries-in-asia-by-population/"
webpage <- read_html(asia_url)

country_data <- data.frame(
  country = webpage %>% html_elements("table a") %>% html_text2(),
  link = webpage %>% html_elements("table a") %>% html_attr("href"),
  stringsAsFactors = FALSE
) %>%
  filter(!is.na(link)) %>%
  filter(str_detect(link, "/world-population/.+--population/?$")) %>%
  mutate(
    country = str_squish(country),
    link = paste0("https://www.worldometers.info", link)
  ) %>%
  distinct(link, .keep_all = TRUE)

print(country_data)

scrape_population_table <- function(country_name, country_url) {
  page <- read_html(country_url)
  tables <- page %>% html_elements("table") %>% html_table(fill = TRUE, convert = FALSE)
  target_table <- NULL
  for (tbl in tables) {
    names(tbl) <- str_squish(names(tbl))
    names(tbl) <- str_replace_all(names(tbl), "A+", "a")
    if (any(str_detect(names(tbl), regex("^Year$", ignore_case = TRUE))) &&
        any(str_detect(names(tbl), regex("^APopulation$", ignore_case = TRUE)))) {
      target_table <- tbl
      break
    }
  }
}
```

```

if (is.null(target_table)) return(NULL)
names(target_table) <- str_squish(names(target_table))
names(target_table) <- str_replace_all(names(target_table), "A^", "z")
rank_col <- names(target_table)[str_detect(names(target_table), "Global Rank")]
if (length(rank_col) == 1) names(target_table)[names(target_table) == rank_col] <- "Global Rank"
target_table %>%
  mutate(country = country_name, .before = 1) %>%
  select(any_of(wanted_cols)) %>%
  mutate(across(everything(), as.character))
}

all_tables <- map2(
  country_data$country,
  country_data$link,
  ~{
    cat("Scraping:", .x, "\n")
    tryCatch({ Sys.sleep(1); scrape_population_table(.x, .y) },
      error = function(e) { cat("Failed:", .x, "\n"); NULL })
  }
)

all_tables <- all_tables[!sapply(all_tables, is.null)]
population_data <- bind_rows(all_tables) %>% select(any_of(wanted_cols))
View(population_data)

write.csv(population_data, "D:/dsProj/worldometer_asia_population.csv", row.names = FALSE)

```

Code for Scraping CO2 Data:

```

# 3. Scrape CO2 Data
|
wanted_years <- c(2026,2025,2024,2023,2022,2020,2015,2010,
  2005,2000,1995,1990,1985,1980,1975,1970,1965,1960,1955)

wanted_cols_co2 <- c(
  "country", "Year",
  "Fossil CO2 emissions (tons)",
  "CO2 emissions change",
  "CO2 emissions per capita",
  "Share of World's CO2 emissions"
)

clean_names_custom <- function(x) {
  x %>%
    str_replace_all("CO_", "CO2") %>%
    str_replace_all("A^", "z") %>%
    str_replace_all("\u00a0", " ") %>%
    str_squish()
}

asia_co2_url <- "https://www.worldometers.info/co2-emissions/co2-emissions-by-country/?region=asia"
webpage_co2 <- read_html(asia_co2_url)

asia_country_data <- data.frame(
  country = webpage_co2 %>% html_elements("table a") %>% html_text2(),
  link = webpage_co2 %>% html_elements("table a") %>% html_attr("href"),
  stringsAsFactors = FALSE
) %>%
  filter(!is.na(link)) %>%
  filter(str_detect(link, "^/co2-emissions/.+co2-emissions/?$")) %>%
  mutate(
    country = str_squish(country),
    link = paste0("https://www.worldometers.info", link)
  ) %>%
  distinct(link, .keep_all = TRUE)

print(asia_country_data)

scrape_country_co2 <- function(country_name, country_url) {
  page <- read_html(country_url)
  tables <- page %>% html_elements("table") %>% html_table(fill = TRUE, convert = FALSE)
  target_table <- NULL
  for (tbl in tables) {
    names(tbl) <- clean_names_custom(names(tbl))
    if (any(str_detect(names(tbl), regex("Fossil CO2 emissions", ignore_case = TRUE))) &&
      any(str_detect(names(tbl), regex("CO2 emissions per capita", ignore_case = TRUE)))) {
      target_table <- tbl; break
    }
  }
}

```

```

if (is.null(target_table)) { cat("No CO2 table for:", country_name, "\n"); return(NULL) }
names(target_table) <- clean_names_custom(names(target_table))
names(target_table)[1] <- "Year"
names(target_table) <- names(target_table) %>%
  str_replace_all("Fossil CO2 Emissions \\\(tons\\)", "Fossil CO2 emissions (tons)") %>%
  str_replace_all("Fossil CO2 emissions \\\(tons\\)", "Fossil CO2 emissions (tons)") %>%
  str_replace_all("CO2 emissions Change", "CO2 emissions change") %>%
  str_replace_all("CO2 emissions Per Capita", "CO2 emissions per capita") %>%
  str_replace_all("Share of World's CO2 Emissions", "Share of World's CO2 emissions")
target_table %>%
  mutate(country = country_name, .before = 1) %>%
  mutate(across(everything(), as.character)) %>%
  mutate(Year = as.integer(str_extract(Year, "\\d{4}"))) %>%
  filter(Year %in% wanted_years) %>%
  select(any_of(wanted_cols_co2))
}

all_co2_tables <- map2(
  asia_country_data$country,
  asia_country_data$link,
  ~{
    cat("Scraping:", .x, "\n")
    tryCatch({ Sys.sleep(1); scrape_country_co2(.x, .y) },
      error = function(e) { cat("Failed:", .x, "-", conditionMessage(e), "\n"); NULL })
  }
)

all_co2_tables <- all_co2_tables[!isapply(all_co2_tables, is.null)]
asia_co2_selected_years <- bind_rows(all_co2_tables) %>%
  mutate(country = factor(country, levels = asia_country_data$country)) %>%
  arrange(country, desc(Year)) %>%
  mutate(country = as.character(country))

View(asia_co2_selected_years)
write.csv(asia_co2_selected_years, "D:/dsProj/asia_co2_selected_years.csv", row.names = FALSE)

```

GDP Data - Load & Clean:

4. GDP Data – Load & Clean

```

gdp_file <- "D:/dsProj/API_NY.GDP.MKTP.KD.ZG_DS2_en_csv_v2_121708.csv"
population_file <- "D:/dsProj/worldometer_asia_population.csv"
co2_file <- "D:/dsProj/asia_co2_selected_years.csv"
output_file <- "D:/dsProj/asia_gdp_growth_fixed.csv"

standardize_country <- function(x) {
  x <- str_squish(x)
  case_when(
    x %in% c("Korea, Rep.", "Republic of Korea") ~ "South Korea",
    x %in% c("Korea, Dem. People's Rep.") ~ "North Korea",
    x == "Viet Nam" ~ "Vietnam",
    x %in% c("Turkiye", "Türkiye") ~ "Turkey",
    x == "Iran, Islamic Rep." ~ "Iran",
    x == "Lao PDR" ~ "Laos",
    x == "Kyrgyz Republic" ~ "Kyrgyzstan",
    x == "Syrian Arab Republic" ~ "Syria",
    x == "Yemen, Rep." ~ "Yemen",
    x == "Brunei Darussalam" ~ "Brunei",
    x == "Hong Kong SAR, China" ~ "Hong Kong",
    x == "Macao SAR, China" ~ "Macao",
    x == "Russian Federation" ~ "Russia",
    x %in% c("West Bank and Gaza", "State of Palestine") ~ "Palestine",
    x == "Taiwan, China" ~ "Taiwan",
    TRUE ~ x
  )
}

population_data_raw <- read.csv(population_file, stringsAsFactors = FALSE)
asia_co2_selected_years <- read.csv(co2_file, stringsAsFactors = FALSE)

population_data_raw <- population_data_raw %>%
  mutate(country = standardize_country(country), Year = as.integer(Year))
asia_co2_selected_years <- asia_co2_selected_years %>%
  mutate(country = standardize_country(country), Year = as.integer(Year))

asia_countries <- intersect(unique(population_data_raw$country),
  unique(asia_co2_selected_years$country))
common_years <- intersect(unique(population_data_raw$Year),
  unique(asia_co2_selected_years$Year))
asia_countries <- sort(asia_countries)
common_years <- sort(common_years)

cat("\nCommon Asian countries:\n"); print(asia_countries)
cat("\nCommon years:\n"); print(common_years)

# Check countries in population but missing in CO2
missing_in_co2 <- setdiff(unique(population_data_raw$country),
  unique(asia_co2_selected_years$country))
cat("\nCountries in population data but missing in CO2 data:\n")
print(missing_in_co2)

```

```

# Check countries in CO2 but missing in population
missing_in_population <- setdiff(unique(asia_co2_selected_years$country),
                                unique(population_data_raw$country))
cat("\nCountries in CO2 data but missing in population data:\n")
print(missing_in_population)

gdp_raw <- read.csv(gdp_file, skip = 4, check.names = FALSE, stringsAsFactors = FALSE)
gdp_raw <- gdp_raw[, names(gdp_raw) != ""]
gdp_raw <- gdp_raw[, sapply(gdp_raw, function(col)
  !all(is.na(col) | str_squish(as.character(col)) == ""))]
names(gdp_raw) <- str_squish(names(gdp_raw))

gdp_clean <- gdp_raw %>%
  rename(country_original = 'Country Name', country_code = 'Country Code',
         indicator_name = 'Indicator Name', indicator_code = 'Indicator Code') %>%
  mutate(country_original = str_squish(country_original),
         country = standardize_country(country_original)) %>%
  pivot_longer(cols = matches("^\\d{4}$"), names_to = "Year",
              values_to = "GDP_growth_annual_percent") %>%
  mutate(Year = as.integer(Year),
         GDP_growth_annual_percent = as.numeric(GDP_growth_annual_percent))

asia_gdp_fixed <- gdp_clean %>%
  filter(country %in% asia_countries, Year %in% common_years) %>%
  select(country, country_original, country_code, Year, GDP_growth_annual_percent) %>%
  arrange(country, desc(Year))

# Check missing GDP countries after name fixing
missing_from_gdp <- setdiff(asia_countries, unique(gdp_clean$country))
cat("\nAsian countries missing from World Bank GDP after name fixing:\n")
print(missing_from_gdp)

cat("\nCountries with missing GDP values:\n")
print(asia_gdp_fixed %>% filter(is.na(GDP_growth_annual_percent)) %>%
      count(country, name = "missing_gdp_rows") %>% arrange(desc(missing_gdp_rows)))

View(asia_gdp_fixed)
write.csv(asia_gdp_fixed, output_file, row.names = FALSE)
cat("\nFixed GDP file saved:", output_file, "\n")

```

Energy Data - Load & Filter:

5. Energy Data – Load & Filter

```

asian_countries <- c(
  "India", "China", "Indonesia", "Pakistan", "Bangladesh", "Japan", "Philippines",
  "Vietnam", "Iran", "Turkey", "Thailand", "Myanmar", "South Korea", "Iraq",
  "Afghanistan", "Yemen", "Uzbekistan", "Malaysia", "Saudi Arabia", "Nepal",
  "North Korea", "Syria", "Sri Lanka", "Taiwan", "Kazakhstan", "Cambodia",
  "Jordan", "United Arab Emirates", "Tajikistan", "Azerbaijan", "Israel",
  "Laos", "Turkmenistan", "Kyrgyzstan", "Hong Kong", "Singapore", "Lebanon",
  "State of Palestine", "Oman", "Kuwait", "Georgia", "Mongolia", "Qatar",
  "Armenia", "Bahrain", "Timor-Leste", "Cyprus", "Bhutan", "Macao", "Maldives", "Brunei"
)

pop_years <- c(2026, 2025, 2024, 2023, 2022, 2020, 2015, 2010,
              2005, 2000, 1995, 1990, 1985, 1980, 1975, 1970, 1965, 1960, 1955)

energy_raw <- read.csv("D:/dsProj/per-capita-energy-use.csv", stringsAsFactors = FALSE)
cat("Raw energy dims:", dim(energy_raw), "\n")

energy_clean_full <- energy_raw %>%
  rename(country = entity, Year = year,
         Energy_Per_Capita_kWh = primary_energy_consumption_per_capita_kWh) %>%
  select(country, Year, Energy_Per_Capita_kWh)

energy_asia <- energy_clean_full %>%
  filter(country %in% asian_countries, Year %in% pop_years) %>%
  arrange(match(country, asian_countries), desc(Year))

cat("\nFiltered dims:", dim(energy_asia), "\n")

full_grid <- expand.grid(country = asian_countries, Year = pop_years,
                       stringsAsFactors = FALSE)
missing <- full_grid %>% anti_join(energy_asia, by = c("country", "Year")) %>%
  arrange(country, desc(Year))
cat("\nMissing country-year combinations:", nrow(missing), "\n")
if (nrow(missing) > 0) { cat("Missing countries:\n"); print(unique(missing$country)) }

write.csv(energy_asia, "D:/dsProj/asia_energy_per_capita.csv", row.names = FALSE)
cat("\n Saved: D:/dsProj/asia_energy_per_capita.csv \n")
cat("Rows:", nrow(energy_asia), "\n")
cat("Countries:", length(unique(energy_asia$country)), "\n")
cat("\nPreview:\n")
print(head(energy_asia, 20))

```


Data Frames:

	country	Year	Population	Yearly % Change	Yearly Change	Migrants (net)	Median Age	Fertility Rate	Density (P/Km²)	Urban Pop %	Urban Population	Country's Share of World Pop	World Population	Global Rank
1	India	2026	1,476,625,576	0.87%	12,760,051	-440,456	29.2	1.93	497	37.6%	555,299,296	17.79%	8,300,678,395	1
2	India	2025	1,463,865,525	0.89%	12,929,734	-495,753	28.8	1.94	492	37.1%	542,742,539	17.78%	8,231,613,070	1
3	India	2024	1,450,935,791	0.89%	12,866,195	-630,830	28.4	1.96	488	36.6%	530,387,142	17.78%	8,161,972,572	1
4	India	2023	1,438,069,596	0.89%	12,646,384	-979,179	28.1	1.98	484	36%	518,239,122	17.77%	8,091,734,930	1
5	India	2022	1,425,423,212	0.81%	11,402,759	-1,353,478	27.7	1.99	479	35.5%	506,304,869	17.77%	8,021,407,192	1
6	India	2020	1,402,617,695	1.1%	14,918,639	-73,806	27	2.05	472	34.4%	483,098,640	17.78%	7,887,001,292	2
7	India	2015	1,328,024,498	1.32%	16,908,587	-655,085	25.3	2.29	447	32.3%	429,069,459	17.78%	7,470,491,872	2
8	India	2010	1,243,481,564	1.49%	17,761,048	-50,905	23.6	2.6	418	30.6%	380,744,554	17.71%	7,021,732,148	2
9	India	2005	1,154,676,322	1.77%	19,350,718	-929,454	22.2	2.96	388	29%	334,479,406	17.53%	6,586,970,132	2
10	India	2000	1,057,922,733	1.96%	19,524,338	-143,960	21.2	3.35	356	27.5%	291,350,282	17.14%	6,171,702,993	2
11	India	1995	960,301,044	2.11%	19,065,765	-57,297	20.3	3.65	323	26.6%	255,558,824	16.68%	5,758,878,982	2
12	India	1990	864,972,221	2.28%	18,464,886	125,514	19.7	4.04	291	25.7%	222,296,728	16.24%	5,327,803,110	2
13	India	1985	772,647,793	2.37%	17,058,754	-89,991	19.3	4.43	260	24.6%	190,321,782	15.87%	4,868,943,465	2
14	India	1980	687,354,025	2.37%	15,208,898	210,914	18.9	4.78	231	23.4%	160,941,941	15.45%	4,447,606,236	2
15	India	1975	611,309,535	2.29%	13,089,953	434,205	18.4	5.2	206	21.7%	132,533,810	15.02%	4,070,735,277	2
16	India	1970	545,864,268	2.18%	11,144,824	233,782	18.1	5.62	184	20%	109,388,950	14.77%	3,694,683,794	2
17	India	1965	490,140,146	2.37%	10,829,962	-248,299	18.5	5.94	165	19.1%	93,493,844	14.70%	3,334,533,703	2
18	India	1960	435,990,338	2.38%	9,657,890	146,955	19.2	5.92	147	18.5%	80,565,723	14.46%	3,015,470,894	2
19	India	1955	387,700,887	2.29%	8,284,413	70,393	19.7	5.91	130	18.6%	71,958,495	14.15%	2,740,213,792	2
20	China	2026	1,412,914,089	-0.22%	-3,182,005	-232,107	40.6	1.03	150	68.7%	970,433,085	17.02%	8,300,678,395	2
21	China	2025	1,416,096,094	-0.23%	-3,225,184	-268,126	40.1	1.02	151	67.5%	956,553,854	17.20%	8,231,613,070	2
22	China	2024	1,419,321,278	-0.23%	-3,262,655	-318,992	39.6	1.01	151	66.4%	941,865,672	17.39%	8,161,972,572	2
23	China	2023	1,422,584,933	-0.18%	-2,594,636	-567,724	39.1	1	152	65.1%	926,375,811	17.58%	8,091,734,930	2

	country	country_original	country_code	Year	GDP_growth_annual_percent
1	Afghanistan	Afghanistan	AFG	2024	NA
2	Afghanistan	Afghanistan	AFG	2023	2.266943736
3	Afghanistan	Afghanistan	AFG	2022	-6.240171992
4	Afghanistan	Afghanistan	AFG	2020	-2.351100672
5	Afghanistan	Afghanistan	AFG	2015	1.451314661
6	Afghanistan	Afghanistan	AFG	2010	14.362441471
7	Afghanistan	Afghanistan	AFG	2005	11.229714827
8	Afghanistan	Afghanistan	AFG	2000	NA
9	Afghanistan	Afghanistan	AFG	1995	NA
10	Afghanistan	Afghanistan	AFG	1990	NA
11	Afghanistan	Afghanistan	AFG	1985	NA
12	Afghanistan	Afghanistan	AFG	1980	NA
13	Afghanistan	Afghanistan	AFG	1975	NA
14	Afghanistan	Afghanistan	AFG	1970	NA

	country	Year	Fossil.CO2.emissions..tons.	CO2.emissions.change	CO2.emissions.per.capita	Share.of.World.s.CO2.emissions
1	China	2024	13,124,727,993	0.79%	9.13	33.12%
2	China	2023	13,021,241,979	4.01%	9.08	33.29%
3	China	2022	12,518,684,441	-0.26%	8.74	32.48%
4	China	2020	11,964,128,536	1.28%	8.40	33.04%
5	China	2015	10,749,896,615	-1.21%	7.69	29.62%
6	China	2010	9,106,005,111	9.46%	6.70	26.81%
7	China	2005	6,235,685,203	13.45%	4.72	20.77%
8	China	2000	3,647,877,288	5.8%	2.84	14.20%
9	China	1995	3,365,775,036	10.46%	2.71	14.21%
10	China	1990	2,414,276,240	1.78%	2.06	10.65%
11	China	1985	1,878,471,978	1.07%	1.75	9.31%
12	China	1980	1,564,736,772	-2.14%	1.57	7.91%
13	China	1975	1,184,119,403	14.66%	1.29	6.83%
14	China	1970	911,501,251	-	1.11	5.79%
15	India	2024	3,153,829,130	4.62%	2.19	7.96%
16	India	2023	3,014,522,625	7.39%	2.12	7.71%
17	India	2022	2,807,080,475	8.05%	1.99	7.28%
18	India	2020	2,356,958,991	-7.62%	1.70	6.51%
19	India	2015	2,278,229,605	1.81%	1.74	6.28%
20	India	2010	1,730,054,152	6.72%	1.41	5.09%

	country	Year	Energy_Per_Capita_kWh
1	India	2024	7812.9277
2	India	2023	7529.4404
3	India	2022	7057.1865
4	India	2020	6268.4365
5	India	2015	5943.0303
6	India	2010	5024.5137
7	India	2005	3991.0898
8	India	2000	3521.4302
9	India	1995	3085.6135
10	India	1990	2659.8555
11	India	1985	2050.4330
12	India	1980	1772.8840
13	India	1975	1590.0476
14	India	1970	1399.1405
15	India	1965	1263.8252
16	China	2024	34514.4570
17	China	2023	33089.0700
18	China	2022	31235.7070
19	China	2020	29095.9360
20	China	2015	25166.6270
21	China	2010	21497.8090
22	China	2005	16050.9375
23	China	2000	9294.6280

2.3 Final Collected Dataset

After web scraping, downloading, the final datasets contained country-year observations for Asian countries. The dataset included variables related to:

- CO₂ emissions per capita
- Total CO₂ emissions
- Population
- Urbanization
- GDP growth
- Energy consumption per capita
- Fertility rate
- Median age
- Population density

This collected datasets was then used for data exploration, preprocessing, classification, regression, and model interpretation.

```
# Load all 4 datasets

population <- read.csv("worldometer_asia_population.csv",
  stringsAsFactors = FALSE, fileEncoding = "UTF-8")
gdp <- read.csv("asia_gdp_growth_fixed.csv",
  stringsAsFactors = FALSE, fileEncoding = "UTF-8")
co2 <- read.csv("asia_co2_selected_years.csv",
  stringsAsFactors = FALSE, fileEncoding = "UTF-8")
energy <- read.csv("asia_energy_per_capita.csv",
  stringsAsFactors = FALSE, fileEncoding = "UTF-8")
```

3. Data Understanding and Exploration

3.1 Loading and Initial Inspection of Collected Datasets

After collecting the data, the four datasets were loaded into R using the `read.csv()` function. These datasets were loaded separately so that their structure, column names, country coverage, year coverage, and missing values could be checked before cleaning and merging.

The four datasets loaded in this step were:

- Population dataset: `worldometer_asia_population.csv`
- GDP dataset: `asia_gdp_growth_fixed.csv`
- CO₂ dataset: `asia_co2_selected_years.csv`

- Energy dataset: asia_energy_per_capita.csv

After loading the datasets, the following information was checked for each dataset:

- Dataset dimensions, meaning number of rows and columns
- Column names available in each dataset
- Number of unique countries
- Available years in the dataset
- Total missing values in each dataset

This step was important because the datasets came from different sources and had different structures. Checking the datasets before merging helped identify differences in variables, years, country coverage, and missing values.

Code for Data Structure Check:

```
# Load all 4 datasets
population <- read.csv("worldometer_asia_population.csv",
                      stringsAsFactors = FALSE, fileEncoding = "UTF-8")
gdp <- read.csv("asia_gdp_growth_fixed.csv",
               stringsAsFactors = FALSE, fileEncoding = "UTF-8")
co2 <- read.csv("asia_co2_selected_years.csv",
               stringsAsFactors = FALSE, fileEncoding = "UTF-8")
energy <- read.csv("asia_energy_per_capita.csv",
                  stringsAsFactors = FALSE, fileEncoding = "UTF-8")

cat("\n Population \n")
cat("Dims:", dim(population), "\n")
cat("Columns:", paste(names(population), collapse = ", "), "\n")
cat("Countries:", length(unique(population$country)), "\n")
cat("Years:", paste(sort(unique(population$Year)), collapse = ", "), "\n")

cat("\n GDP \n")
cat("Dims:", dim(gdp), "\n")
cat("Columns:", paste(names(gdp), collapse = ", "), "\n")
cat("Countries:", length(unique(gdp$country)), "\n")
cat("Years:", paste(sort(unique(gdp$Year)), collapse = ", "), "\n")

cat("\n CO2 \n")
cat("Dims:", dim(co2), "\n")
cat("Columns:", paste(names(co2), collapse = ", "), "\n")
cat("Countries:", length(unique(co2$country)), "\n")
cat("Years:", paste(sort(unique(co2$Year)), collapse = ", "), "\n")

cat("\n Energy \n")
cat("Dims:", dim(energy), "\n")
cat("Columns:", paste(names(energy), collapse = ", "), "\n")
cat("Countries:", length(unique(energy$country)), "\n")
cat("Years:", paste(sort(unique(energy$Year)), collapse = ", "), "\n")

cat("\n Missing Values per Dataset \n")
cat("Population NAs:", sum(is.na(population)), "\n")
cat("GDP NAs:", sum(is.na(gdp)), "\n")
cat("CO2 NAs:", sum(is.na(co2)), "\n")
cat("Energy NAs:", sum(is.na(energy)), "\n")
```

Output:

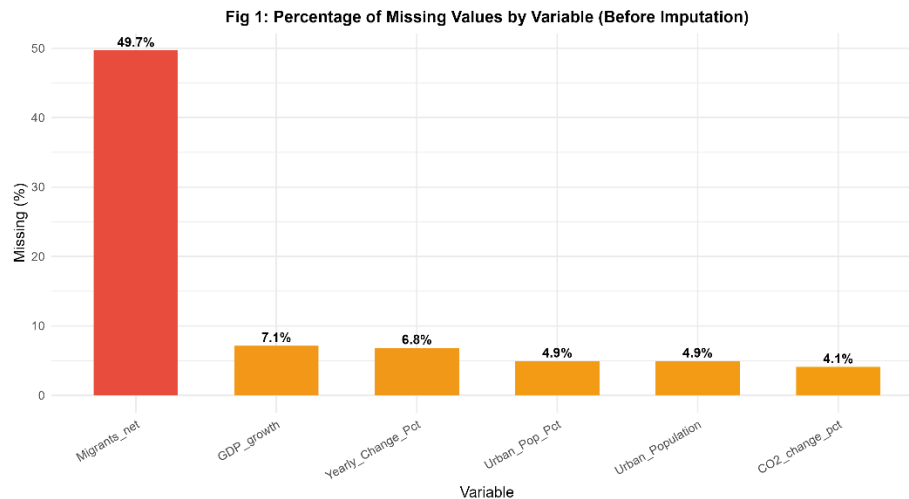
```
Population
> cat("Dims:", dim(population), "\n")
Dims: 969 14
> cat("Columns:", paste(names(population), collapse = ", "), "\n")
Columns: country, Year, Population, Yearly...Change, Yearly.Change, Migrants..net., Median.Age, Fertility.Rate, Density..P.Km., Urban.Pop., Urban.Population, Country
lation, Global.Rank
> cat("Countries:", length(unique(population$country)), "\n")
Countries: 51
> cat("Years:", paste(sort(unique(population$Year)), collapse = ", "), "\n")
Years: 1955, 1960, 1965, 1970, 1975, 1980, 1985, 1990, 1995, 2000, 2005, 2010, 2015, 2020, 2022, 2023, 2024, 2025, 2026
>
> cat("\n GDP \n")

GDP
> cat("Dims:", dim(gdp), "\n")
Dims: 686 5
> cat("Columns:", paste(names(gdp), collapse = ", "), "\n")
Columns: country, country_original, country_code, Year, GDP_growth_annual_percent
> cat("Countries:", length(unique(gdp$country)), "\n")
Countries: 49
> cat("Years:", paste(sort(unique(gdp$Year)), collapse = ", "), "\n")
Years: 1970, 1975, 1980, 1985, 1990, 1995, 2000, 2005, 2010, 2015, 2020, 2022, 2023, 2024
>
> cat("\n CO2 \n")

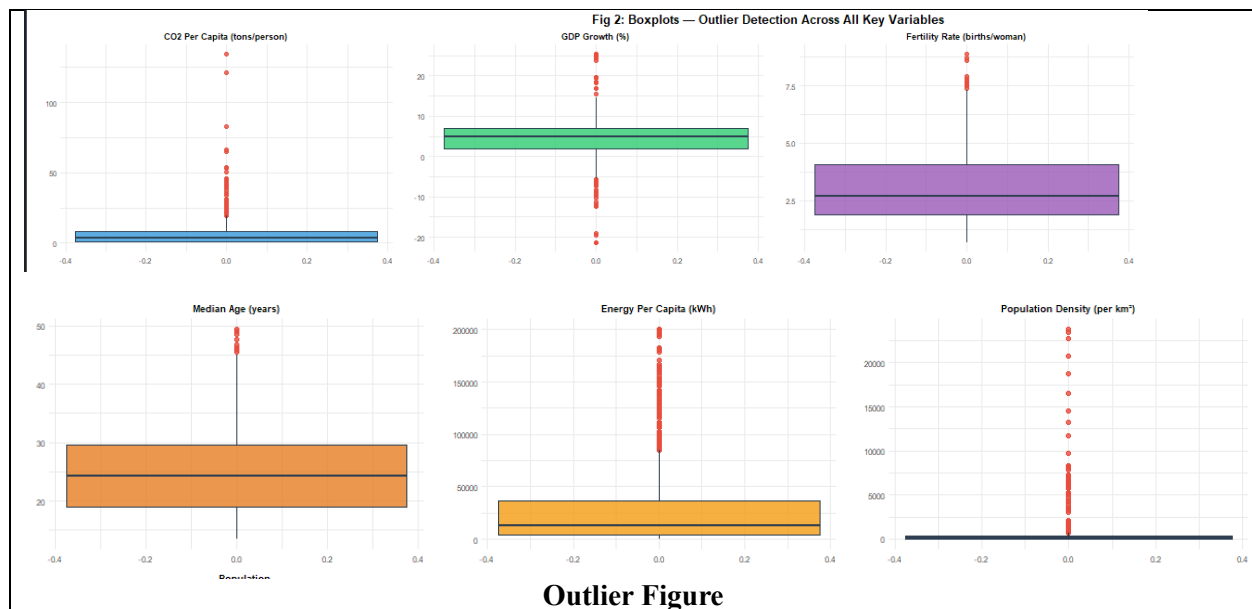
CO2
> cat("Dims:", dim(co2), "\n")
Dims: 700 6
> cat("Columns:", paste(names(co2), collapse = ", "), "\n")
Columns: country, Year, Fossil.CO2.emissions.tons., CO2.emissions.change, CO2.emissions.per.capita, Share.of.World.s.CO2.emissions
> cat("Countries:", length(unique(co2$country)), "\n")
Countries: 50
> cat("Years:", paste(sort(unique(co2$Year)), collapse = ", "), "\n")
Years: 1970, 1975, 1980, 1985, 1990, 1995, 2000, 2005, 2010, 2015, 2020, 2022, 2023, 2024

Energy
> cat("Dims:", dim(energy), "\n")
Dims: 627 3
> cat("Columns:", paste(names(energy), collapse = ", "), "\n")
Columns: country, Year, Energy_Per_Capita_kwh
> cat("Countries:", length(unique(energy$country)), "\n")
Countries: 49
> cat("Years:", paste(sort(unique(energy$Year)), collapse = ", "), "\n")
Years: 1965, 1970, 1975, 1980, 1985, 1990, 1995, 2000, 2005, 2010, 2015, 2020, 2022, 2023, 2024
>
> cat("\n Missing Values per Dataset \n")

Missing Values per Dataset
> cat("Population NAs:", sum(is.na(population)), "\n")
Population NAs: 0
> cat("GDP NAs:", sum(is.na(gdp)), "\n")
GDP NAs: 100
> cat("CO2 NAs:", sum(is.na(co2)), "\n")
CO2 NAs: 0
> cat("Energy NAs:", sum(is.na(energy)), "\n")
Energy NAs: 0
.
```



Missing Value Figure



3.2 Creating the Main Dataset by Merging

After collecting the data, the four datasets were cleaned and merged to create the main dataset for analysis. The datasets were:

- **Population dataset:** population, density, fertility rate, median age, urban population
- **GDP dataset:** annual GDP growth rate
- **CO₂ dataset:** total CO₂ emissions, CO₂ per capita, CO₂ change percentage
- **Energy dataset:** energy consumption per capita

Before merging, numeric columns were cleaned by removing commas, percentage signs, and unnecessary symbols. Then all datasets were merged using two common columns:

- **Country**
- **Year**

An inner join was used so that only the countries and years available in all four datasets were kept. The common years were:

1970, 1975, 1980, 1985, 1990, 1995, 2000, 2005, 2010, 2015, 2020, 2022, 2023, and 2024.

After merging, the final dataset contained:

- 588 observations
- 48 Asian countries
- 14 selected years

Code of Clean & Merge All 4 Datasets:

```
population_clean <- population %>%
  mutate(
    Population = as.numeric(gsub(",", "", Population)),
    Yearly_Change = as.numeric(gsub(",", "", Yearly_Change)),
    Migrants_net = as.numeric(gsub(",", "", Migrants..net.)),
    Density = as.numeric(gsub(",", "", Density..P.Km..)),
    Urban_Population = as.numeric(gsub(",", "", Urban.Population)),
    World_Population = as.numeric(gsub(",", "", World.Population)),
    Yearly_Change_Pct = as.numeric(gsub("%", "", Yearly...Change)),
    Urban_Pop_Pct = as.numeric(gsub("%", "", Urban.Pop..)),
    World_Share_Pct = as.numeric(gsub("%", "", Country.s.Share.of.World.Pop)),
    Median_Age = as.numeric(Median.Age),
    Fertility_Rate = as.numeric(Fertility.Rate),
    Global_Rank = as.numeric(Global.Rank),
    Year = as.integer(Year)
  ) %>%
  select(country, Year, Population, Yearly_Change_Pct,
    Migrants_net, Median_Age, Fertility_Rate, Density,
    Urban_Pop_Pct, Urban_Population, Global_Rank)

gdp_clean <- gdp %>%
  select(country, Year, GDP_growth = GDP_growth_annual_percent) %>%
  mutate(Year = as.integer(Year))

co2_clean <- co2 %>%
  mutate(
    CO2_total_tons = as.numeric(gsub(",", "", Fossil.CO2.emissions..tons.)),
    CO2_change_pct = as.numeric(gsub("[^0-9.\\-]", "",
      gsub("\\u2212", "- ", CO2.emissions.change))),
    World_CO2_share = as.numeric(gsub("%", "", Share.of.World.s.CO2.emissions)),
    CO2_per_capita = as.numeric(CO2.emissions.per.capita),
    Year = as.integer(Year)
  ) %>%
  select(country, Year, CO2_total_tons, CO2_per_capita, CO2_change_pct, World_CO2_share)

energy_clean <- energy %>%
  mutate(Year = as.integer(Year)) %>%
  select(country, Year, Energy_Per_Capita_kwh)

common_years <- Reduce(intersect, list(
  unique(population_clean$Year),
  unique(gdp_clean$Year),
  unique(co2_clean$Year),
  unique(energy_clean$Year)
))
cat("Common years across all 4 datasets:\n"); print(sort(common_years))

merged_data <- population_clean %>%
  filter(Year %in% common_years) %>%
  inner_join(gdp_clean %>% filter(Year %in% common_years), by = c("country", "Year")) %>%
  inner_join(co2_clean %>% filter(Year %in% common_years), by = c("country", "Year")) %>%
  inner_join(energy_clean %>% filter(Year %in% common_years), by = c("country", "Year"))

cat("\nMerged dataset dims:", dim(merged_data), "\n")
cat("Countries:", length(unique(merged_data$country)), "\n")
cat("Years:", paste(sort(unique(merged_data$Year)), collapse = ", "), "\n")
cat("\nMissing values per column:\n"); print(colSums(is.na(merged_data)))
cat("\nPreview:\n"); print(head(merged_data, 5))
```

Output:

```
> cat("Common years across all 4 datasets:\n"); print(sort(common_years))
Common years across all 4 datasets:
[1] 1970 1975 1980 1985 1990 1995 2000 2005 2010 2015 2020 2022 2023 2024
>
> merged_data <- population_clean %>%
+   filter(Year %in% common_years) %>%
+   inner_join(gdp_clean %>% filter(Year %in% common_years), by = c("country", "Year")) %>%
+   inner_join(co2_clean %>% filter(Year %in% common_years), by = c("country", "Year")) %>%
+   inner_join(energy_clean %>% filter(Year %in% common_years), by = c("country", "Year"))
>
> cat("\nMerged dataset dims:", dim(merged_data), "\n")

Merged dataset dims: 588 17
> cat("Countries:", length(unique(merged_data$country)), "\n")
Countries: 48
> cat("Years:", paste(sort(unique(merged_data$Year)), collapse = ", "), "\n")
Years: 1970, 1975, 1980, 1985, 1990, 1995, 2000, 2005, 2010, 2015, 2020, 2022, 2023, 2024
> cat("\nMissing values per column:\n"); print(colSums(is.na(merged_data)))

Missing values per column:
  country      Year      Population      Yearly_Change_Pct
      0         0          0             40
  Migrants_net  Median_Age      Fertility_Rate      Density
    292         0          0             0
  Urban_Pop_Pct  Urban_Population      Global_Rank      GDP_growth
    29         29          0             42
  CO2_total_tons  CO2_per_capita      CO2_change_pct      World_CO2_share
    0          0          24             0
  Energy_Per_Capita_kwh
    0
```

```
> cat("\nPreview:\n"); print(head(merged_data, 5))
```

Preview:

	country	Year	Population	Yearly_Change_Pct	Migrants_net	Median_Age	Fertility_Rate	Density
1	India	2024	1450935791	0.89	NA	28.4	1.96	488
2	India	2023	1438069596	0.89	NA	28.1	1.98	484
3	India	2022	1425423212	0.81	NA	27.7	1.99	479
4	India	2020	1402617695	1.10	NA	27.0	2.05	472
5	India	2015	1328024498	1.32	NA	25.3	2.29	447

	Urban_Pop_Pct	Urban_Population	Global_Rank	GDP_growth	CO2_total_tons	CO2_per_capita
1	36.6	530387142	1	6.494766	3153829130	2.19
2	36.0	518239122	1	9.190755	3014522625	2.12
3	35.5	506304869	1	7.609365	2807080475	1.99
4	34.4	483098640	2	-5.777725	2356958991	1.70
5	32.3	429069459	2	7.996254	2278229605	1.74

	CO2_change_pct	World_CO2_share	Energy_Per_Capita_kwh
1	4.62	7.96	7812.928
2	7.39	7.71	7529.440
3	8.05	7.28	7057.186
4	-7.62	6.51	6268.436
5	1.81	6.28	5943.030

```
> |
```

3.3 Exploratory Data Analysis Visualizations

Exploratory Data Analysis was performed to understand distributions, patterns, trends, and relationships in the dataset.

The EDA included:

- Histograms and density plots
- Scatter plots
- Violin plots
- Time-series trend plots
- Correlation heatmaps
- Pairs plot
- ANOVA and chi-square tests

Main EDA findings:

- CO₂ emissions per capita was highly right-skewed.
- Energy consumption per capita had a strong positive relationship with CO₂ emissions.
- Urban countries generally had higher CO₂ emissions per capita.
- GDP growth showed a weaker relationship with CO₂ emissions.
- Correlation analysis showed energy consumption as one of the strongest predictors of CO₂ per capita.

Descriptive Analysis:

```
> print(desc_stats)
```

```
# A tibble: 9 × 7
```

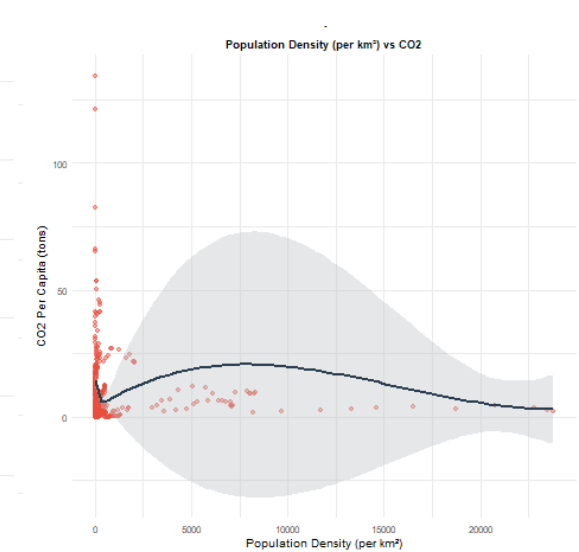
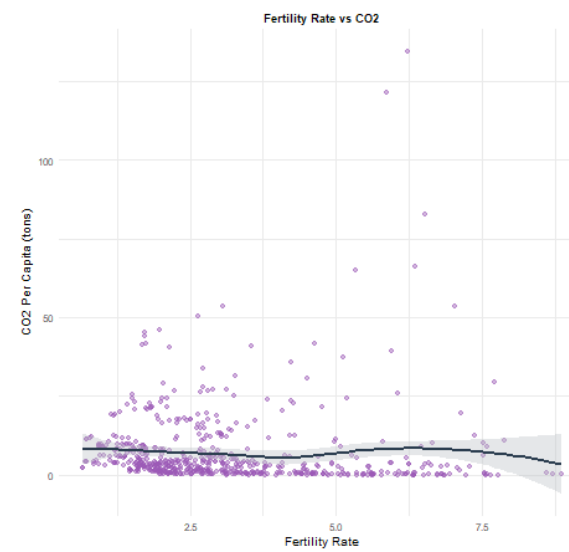
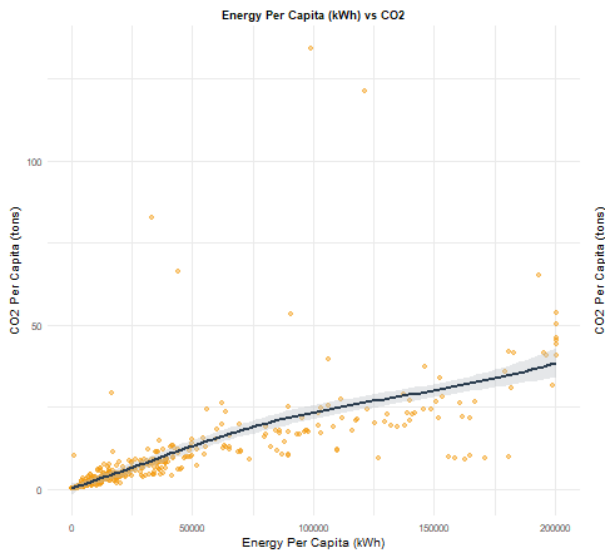
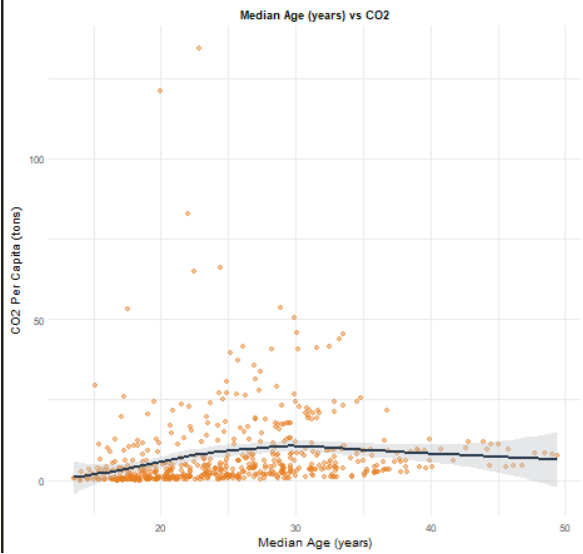
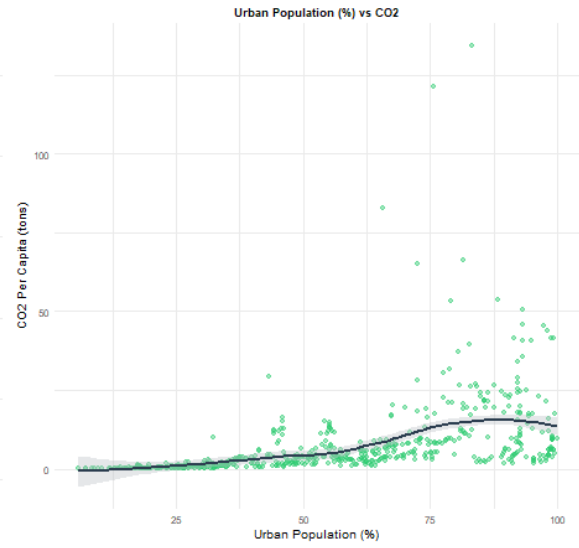
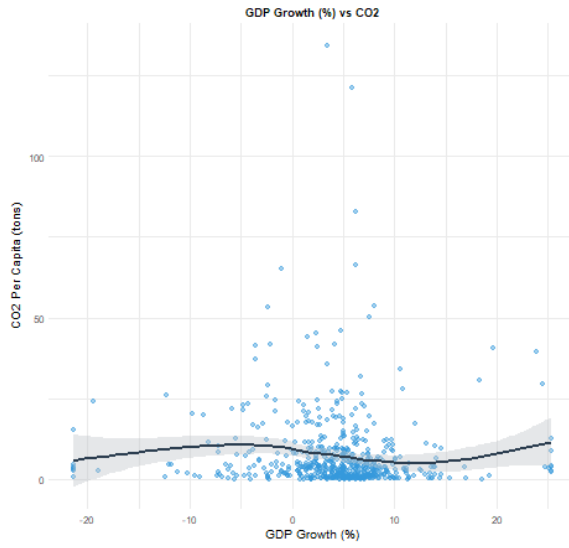
	Variable	Mean	Median	SD	Min	Max	Skew
	<chr>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	CO2_per_capita	7.32	3.52	12.2	0.04	1.34e 2	4.94
2	GDP_growth	4.31	4.96	6.26	-21.4	2.53e 1	-0.607
3	Urban_Pop_Pct	55.9	55.4	26.3	5.8	9.99e 1	0.065
4	Fertility_Rate	3.15	2.70	1.72	0.66	8.86e 0	1.01
5	Median_Age	25.1	24.3	7.17	13.5	4.94e 1	0.731
6	Density	782.	107	2661.	1	2.38e 4	5.79
7	Energy_Per_Capita_kWh	32297.	13365.	45550.	231.	2.00e 5	2.07
8	Population	87092288.	14194294	249852087.	116350	1.45e 9	4.33
9	CO2_total_tons	304307672.	36933595	1266788216.	15515	1.31e10	8.19

```
>
```

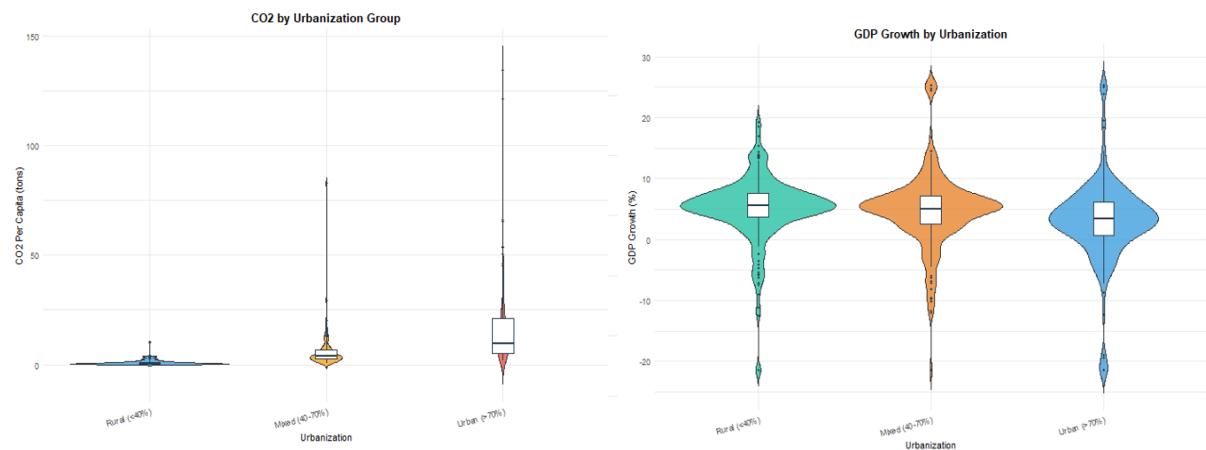
Fig 3: Distribution of Key Variables — Histogram + Density Curve



Histogram and density



Scatter Plots vs CO₂



Violin Plot

Fig 6b: Asian Average CO2, Energy & Urbanization Trends Over Time

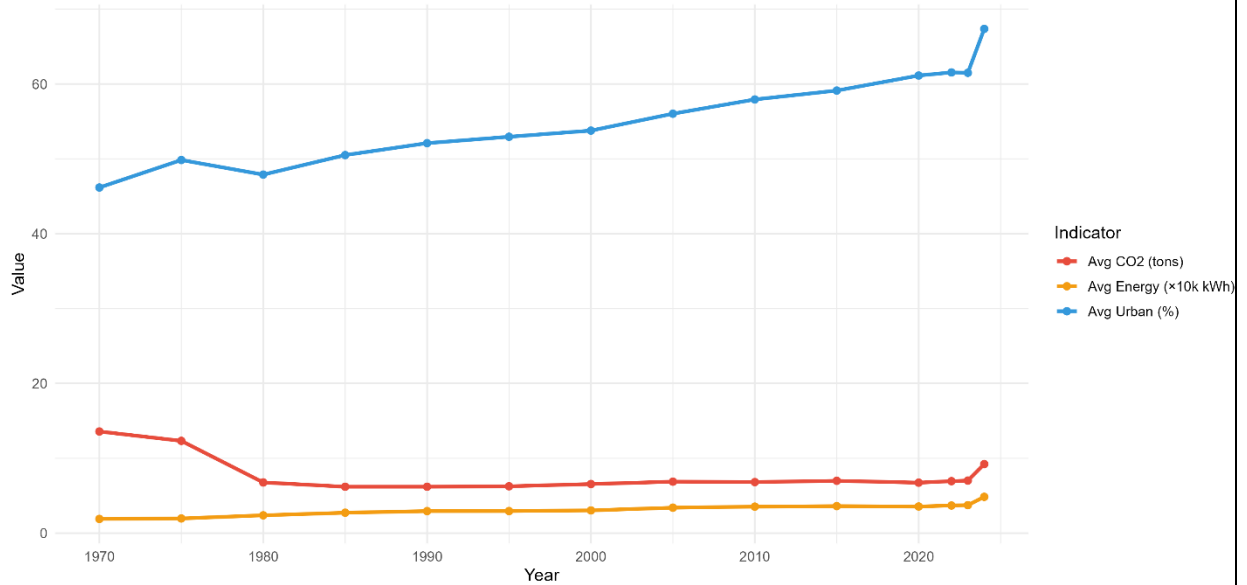


Fig of CO₂, Energy, Urban Trends

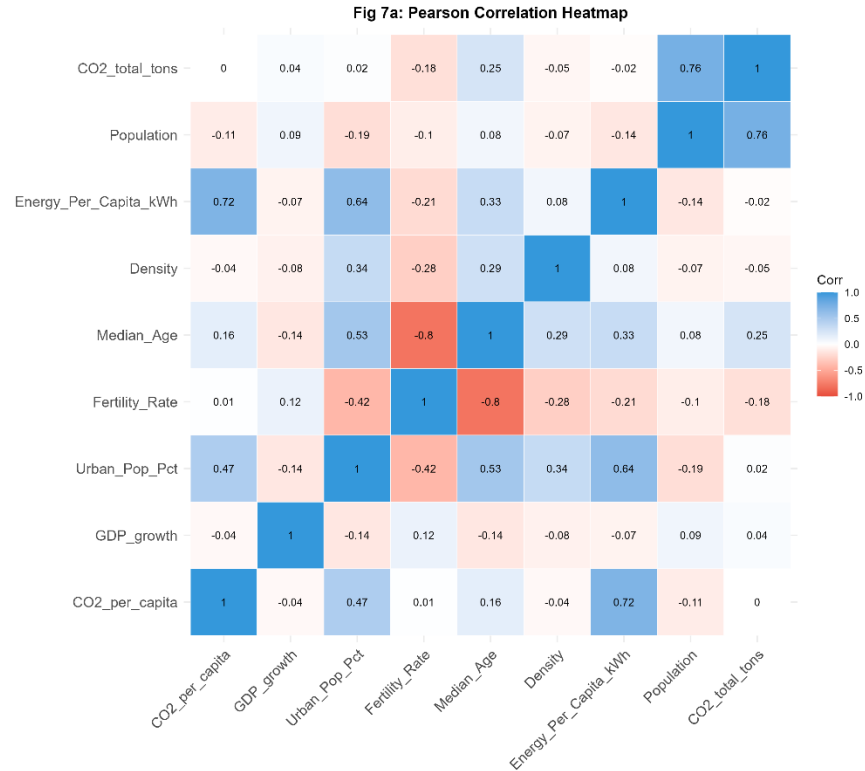


Fig of Pearson Correlation Heatmap

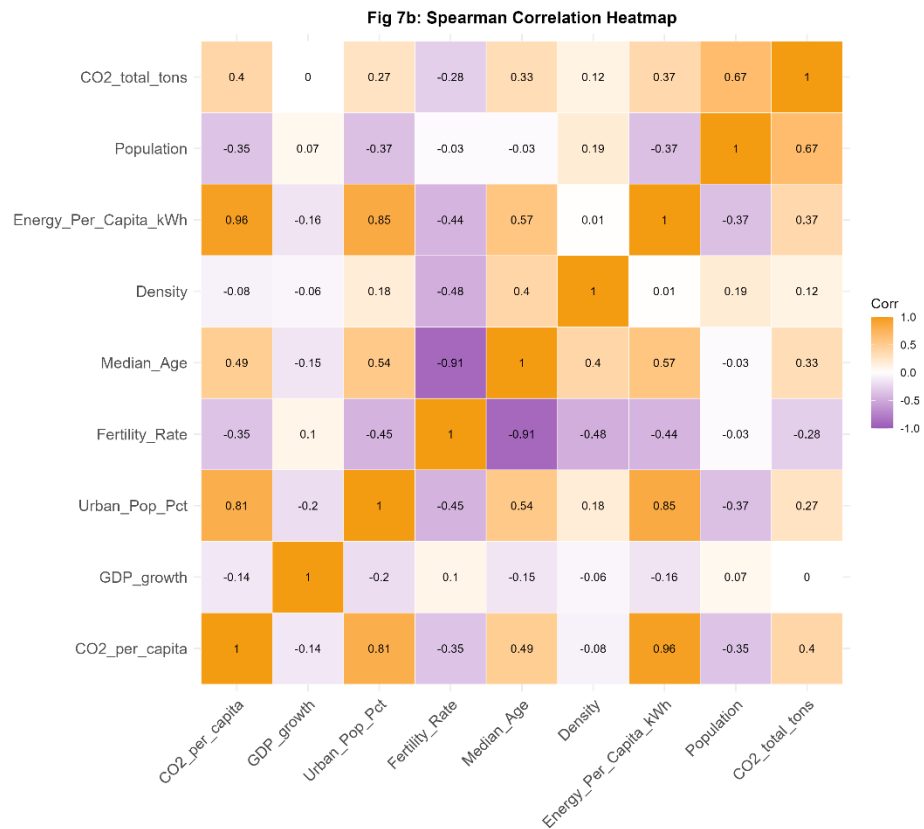


Fig of Spearman heatmap

Fig 10a: Mean CO2 Per Capita by Urbanization (One-Way ANOVA)

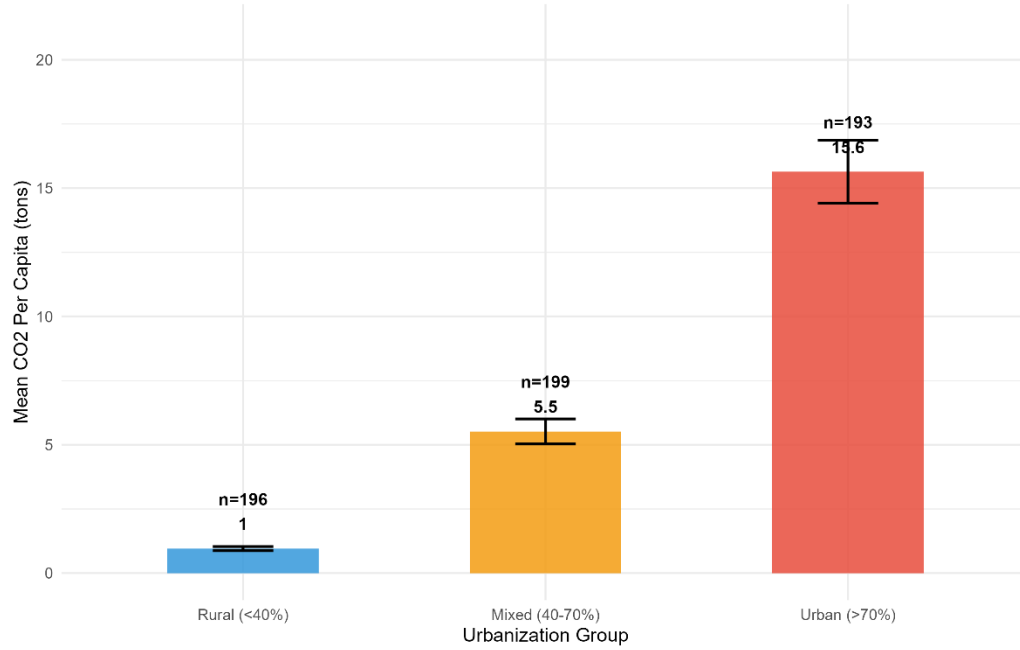


Fig of Anova

Fig 11: Chi-Square Test — CO2 Emission Group vs Categorical Variables

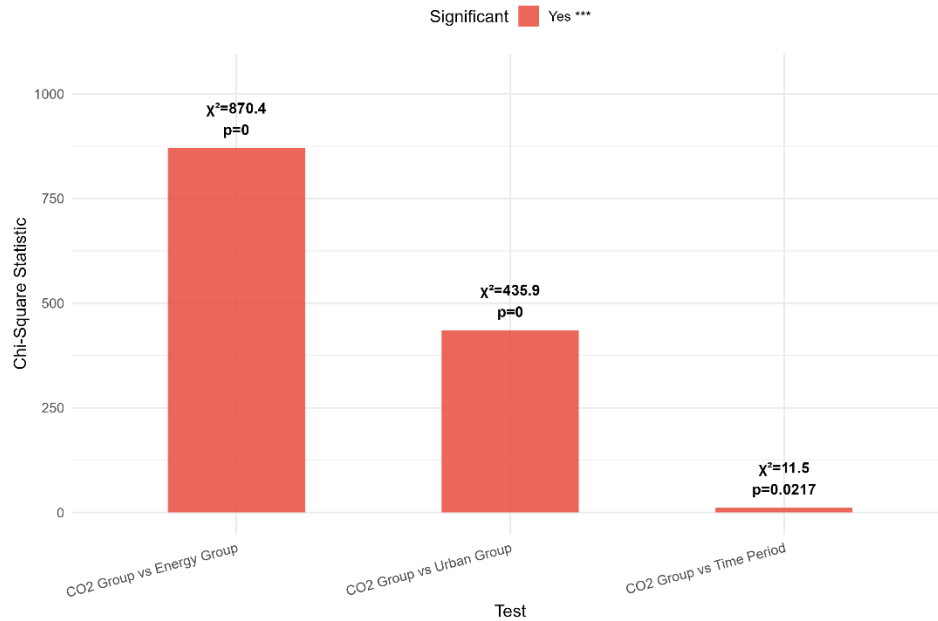


Fig of Chi-Square Test

Code of some Graphs:

```
# FIG 12: Pearson Feature Correlation Bar

cor_feature_df <- data.frame(
  Feature      = names(numeric_cols_df)[names(numeric_cols_df) != "CO2_per_capita"],
  Pearson      = cor_pearson["CO2_per_capita",
                             names(numeric_cols_df)[names(numeric_cols_df) != "CO2_per_capita"]
) %>% mutate(Abs_Pearson = abs(Pearson))

fig12 <- ggplot(cor_feature_df,
  aes(x=reorder(Feature,Abs_Pearson), y=Pearson, fill=Pearson>0)) +
  geom_bar(stat="identity",alpha=0.85) +
  coord_flip() +
  scale_fill_manual(values=c("TRUE"="#3498DB", "FALSE"="#E74C3C"),
    labels=c("Negative", "Positive"),name="Direction") +
  geom_hline(yintercept=c(-0.3,0.3),linetype="dashed",color="#2C3E50",alpha=0.5) +
  geom_text(aes(label=round(Pearson,3),hjust=ifelse(Pearson>0,-0.1,1.1)),
    size=3.8,fontface="bold") +
  theme_minimal(base_size=12) +
  theme(plot.title=element_text(face="bold",hjust=0.5),legend.position="right") +
  ylim(-0.8,1.0) +
  labs(title="Fig 12: Pearson Correlation of Features with CO2 Per Capita",
    x="Feature",y="Pearson Correlation Coefficient")
save_fig(fig12, "Fig12_feature_correlation_bar", width=10, height=7)

# FIG 10: ANOVA

anova_urban <- aov(CO2_per_capita ~ Urban_group, data=merged_final)
anova_period <- aov(CO2_per_capita ~ Period, data=merged_final)
cat("\n--- ANOVA: Urban Group ---\n"); print(summary(anova_urban))
cat("\n--- ANOVA: Time Period ---\n"); print(summary(anova_period))

anova_urban_sum <- summary(anova_urban)[[1]]
anova_period_sum <- summary(anova_period)[[1]]

anova_table <- data.frame(
  Test      = c("Urban Group vs CO2","Time Period vs CO2"),
  F_value    = round(c(anova_urban_sum$F value[1], anova_period_sum$F value[1]),3),
  p_value    = round(c(anova_urban_sum$Pr(>F)[1], anova_period_sum$Pr(>F)[1]),5),
  Significant = c(
    ifelse(anova_urban_sum$Pr(>F)[1] < 0.05,"Yes ***","No"),
    ifelse(anova_period_sum$Pr(>F)[1] < 0.05,"Yes ***","No")
  )
)
print(anova_table); save_table(anova_table, "anova_results")

urban_summary <- merged_final %>%
  group_by(Urban_group) %>%
  summarise(mean_CO2=mean(CO2_per_capita), se=sd(CO2_per_capita)/sqrt(n()), n=n())

fig10a <- ggplot(urban_summary, aes(x=Urban_group,y=mean_CO2,fill=Urban_group)) +
  geom_bar(stat="identity",width=0.5,alpha=0.85) +
  geom_errorbar(aes(ymin=mean_CO2-se,ymax=mean_CO2+se),width=0.2,linewidth=0.8) +
  geom_text(aes(label=paste0("n=",n,"\n",round(mean_CO2,1))),
    vjust=-0.5,size=3.8,fontface="bold") +
  scale_fill_manual(values=c("#3498DB", "#F39C12", "#E74C3C")) +
  theme_minimal(base_size=12) +
  theme(legend.position="none",plot.title=element_text(face="bold",hjust=0.5)) +
  labs(title="Fig 10a: Mean CO2 Per Capita by Urbanization (One-Way ANOVA)",
    x="Urbanization Group",y="Mean CO2 Per Capita (tons)" +
  ylim(0, max(urban_summary$mean_CO2)+1.35)
save_fig(fig10a, "Fig10a_anova_urban", width=9, height=6)

# FIG 5: Violin Plots

v1 <- ggplot(merged_final, aes(x=Urban_group,y=CO2_per_capita,fill=Urban_group)) +
  geom_violin(alpha=0.75,trim=FALSE) +
  geom_boxplot(width=0.12,fill="white",outlier.size=1,color="#2C3E50") +
  scale_fill_manual(values=c("#3498DB", "#F39C12", "#E74C3C")) +
  theme_minimal(base_size=11) +
  theme(legend.position="none",plot.title=element_text(face="bold",hjust=0.5),
    axis.text.x=element_text(angle=15,hjust=1)) +
  labs(title="CO2 by Urbanization Group",x="Urbanization",y="CO2 Per Capita (tons)")
```

```

# FIG 3: Histograms + Density Curves

hist_vars <- list(
  list(v="CO2_per_capita", fill="#3498DB", xlab="Tons/Person", sa=FALSE),
  list(v="GDP_growth", fill="#2ECC71", xlab="Annual %", sa=FALSE),
  list(v="Fertility_Rate", fill="#9B59B6", xlab="Births/woman", sa=FALSE),
  list(v="Urban_Pop_Pct", fill="#1AB394", xlab="Urban (%)", sa=FALSE),
  list(v="Median_Age", fill="#E67E22", xlab="Years", sa=FALSE),
  list(v="Energy_Per_Capita_kWh", fill="#F39C12", xlab="kWh/Person", sa=FALSE)
)

h_list <- lapply(hist_vars, function(x) {
  p <- ggplot(merged_final, aes_string(x = x$v)) +
    geom_histogram(aes(y = after_stat(density)), bins = 30,
                  fill = x$fill, color = "white", alpha = 0.8) +
    geom_density(color = "#E74C3C", linewidth = 1.2) +
    theme_minimal(base_size = 10) +
    labs(title = x$v, x = x$xlab, y = "Density") +
    theme(plot.title = element_text(face="bold",hjust=0.5,size=9))
  if (x$sa) p <- p + scale_x_continuous(labels = label_sa())
  p
})

fig3 <- grid.arrange(grobs = h_list, ncol = 3,
  top = grid::textGrob(
    "Fig 3: Distribution of Key Variables - Histogram + Density Curve"
    gp = grid::gpar(fontface="bold",fontsize=13)))
save_fig(NULL, "Fig03_histograms_density", width = 14, height = 8)

```

4. Data Preprocessing

Data preprocessing was performed after creating the merged dataset. This step was necessary to prepare the data for classification and regression modeling. The preprocessing stage included missing value handling, outlier treatment, feature engineering, target variable creation, feature selection, train-validation-test splitting, scaling, and SMOTE balancing. These steps match the project requirement that preprocessing should include cleaning, missing value handling, feature engineering, transformation, feature selection, encoding, and scaling.

4.1 Data Cleaning, Missing Value Handling, Outlier Treatment, and Dataset Saving

After merging the four datasets, preprocessing was performed to prepare the data for machine learning. First, missing values were checked using `colSums(is.na())`. The variable `Migrants_net` was removed because it had a high number of missing values and was not essential for predicting CO₂ emissions.

For the remaining missing values, country-level median imputation was applied to maintain each country's own data pattern. Missing values in `Yearly_Change_Pct`, `Urban_Pop_Pct`, `Urban_Population`, and `GDP_growth` were replaced using the median value of the same country. Any remaining missing GDP values were filled using the global median. Missing `CO2_change_pct` values were replaced with 0.

Outliers were checked using both the IQR method and the Z-score method. Since many extreme values represented real country-level differences, all outliers were not removed. Instead, Winsorization was applied only to `GDP_growth` and `Energy_Per_Capita_kWh`, where values below the 1st percentile and above the 99th percentile were capped. Finally, the cleaned dataset was saved as `asia_final_clean.csv` for feature engineering, scaling, SMOTE, and model training.

Code:

```
cat("Missing values BEFORE imputation:\n"); print(colSums(is.na(merged_data)))

merged_data <- merged_data %>% select(-Migrants_net)
cat("\nDropped Migrants_net\n")

merged_data <- merged_data %>%
  group_by(country) %>%
  mutate(Yearly_Change_Pct = ifelse(is.na(Yearly_Change_Pct),
                                   median(Yearly_Change_Pct, na.rm = TRUE),
                                   Yearly_Change_Pct)) %>%
  ungroup()

merged_data <- merged_data %>%
  group_by(country) %>%
  mutate(
    Urban_Pop_Pct = ifelse(is.na(Urban_Pop_Pct),
                           median(Urban_Pop_Pct, na.rm = TRUE), Urban_Pop_Pct),
    Urban_Population = ifelse(is.na(Urban_Population),
                              median(Urban_Population, na.rm = TRUE), Urban_Population)
  ) %>%
  ungroup()

merged_data <- merged_data %>%
  group_by(country) %>%
  mutate(GDP_growth = ifelse(is.na(GDP_growth),
                             median(GDP_growth, na.rm = TRUE), GDP_growth)) %>%
  ungroup()
global_gdp_median <- median(merged_data$GDP_growth, na.rm = TRUE)
merged_data$GDP_growth[is.na(merged_data$GDP_growth)] <- global_gdp_median

merged_data$CO2_change_pct[is.na(merged_data$CO2_change_pct)] <- 0

cat("\nMissing values AFTER imputation:\n"); print(colSums(is.na(merged_data)))

key_vars <- c("GDP_growth", "CO2_per_capita", "CO2_total_tons",
              "Population", "Urban_Pop_Pct", "Fertility_Rate",
              "Median_Age", "Density", "Energy_Per_Capita_kwh")

cat("\n=== IQR Outlier Detection ===\n")
iqr_results <- map_df(key_vars, function(v) {
  x <- merged_data[[v]]
  Q1 <- quantile(x, 0.25, na.rm = TRUE)
  Q3 <- quantile(x, 0.75, na.rm = TRUE)
  IQR_val <- Q3 - Q1
  lower <- Q1 - 1.5 * IQR_val
  upper <- Q3 + 1.5 * IQR_val
  n_out <- sum(x < lower | x > upper, na.rm = TRUE)
  data.frame(Variable=v, Q1=round(Q1,2), Q3=round(Q3,2), IQR=round(IQR_val,2),
             Lower_Fence=round(lower,2), Upper_Fence=round(upper,2),
             Outlier_Count=n_out, Outlier_Pct=round(n_out/length(x)*100,1))
})
print(iqr_results); save_table(iqr_results, "outlier_iqr_results")

cat("\n=== Z-Score Outlier Detection (threshold = 3) ===\n")
zscore_results <- map_df(key_vars, function(v) {
  x <- merged_data[[v]]
  z <- abs((x - mean(x, na.rm = TRUE)) / sd(x, na.rm = TRUE))
  n_out <- sum(z > 3, na.rm = TRUE)
  data.frame(Variable=v, Outlier_Count=n_out,
             Outlier_Pct=round(n_out/length(x)*100,1))
})
print(zscore_results); save_table(zscore_results, "outlier_zscore_results")

p01_gdp <- quantile(merged_data$GDP_growth, 0.01, na.rm = TRUE)
p99_gdp <- quantile(merged_data$GDP_growth, 0.99, na.rm = TRUE)
p01_en <- quantile(merged_data$Energy_Per_Capita_kwh, 0.01, na.rm = TRUE)
p99_en <- quantile(merged_data$Energy_Per_Capita_kwh, 0.99, na.rm = TRUE)

merged_data <- merged_data %>%
  mutate(
    GDP_growth = pmin(pmax(GDP_growth, p01_gdp), p99_gdp),
    Energy_Per_Capita_kwh = pmin(pmax(Energy_Per_Capita_kwh, p01_en), p99_en)
  )
cat("GDP_growth Winsorized: [", round(p01_gdp,2), ",", round(p99_gdp,2), "]\n")
cat("Energy Winsorized: [", round(p01_en,2), ",", round(p99_en,2), "]\n")

skew_kurt <- merged_data %>%
  select(all_of(key_vars)) %>%
  summarise(across(everything(), list(
    Skewness = ~round(skewness(., na.rm = TRUE), 3),
    Kurtosis = ~round(kurtosis(., na.rm = TRUE), 3)
  )), .names = "{col}_{.fn}") %>%
  pivot_longer(everything(), names_to = c("Variable", "Stat"), names_sep = "__") %>%
  pivot_wider(names_from = Stat, values_from = value)
print(skew_kurt); save_table(skew_kurt, "skewness_kurtosis")
```


Output:

```
> cat("\nMissing values AFTER imputation:\n"); print(colSums(is.na(merged_data)))

Missing values AFTER imputation:
      country      Year      Population      Yearly_Change_Pct
      0          0          0          0
      Median_Age      Fertility_Rate      Density      Urban_Pop_Pct
      0          0          0          0
      Urban_Population      Global_Rank      GDP_growth      CO2_total_tons
      0          0          0          0
      CO2_per_capita      CO2_change_pct      World_CO2_share      Energy_Per_Capita_kWh
      0          0          0          0
      ..
> print(zscore_results); save_table(zscore_results, "outlier_zscore_results")
  Variable Outlier_Count Outlier_Pct
1  GDP_growth          12          2.0
2  CO2_per_capita       11          1.9
3  CO2_total_tons        7          1.2
4  Population           23          3.9
5  Urban_Pop_Pct         0          0.0
6  Fertility_Rate        3          0.5
7  Median_Age            5          0.9
8  Density              10          1.7
9  Energy_Per_Capita_kWh 15          2.6
Saved: D:/dsProj/project_outputs/tables/outlier_zscore_results.csv
> cat("GDP_growth Winsorized: [", round(p01_gdp,2), ",", round(p99_gdp,2), "]\n")
GDP_growth Winsorized: [ -21.43 , 25.29 ]
> cat("Energy Winsorized: [", round(p01_en,2), ",", round(p99_en,2), "]\n")
Energy Winsorized: [ 231.27 , 200308.1 ]
>
```

4.2 Feature Engineering, Target Creation, and Feature Selection

After cleaning the dataset, feature engineering was performed to create more useful predictors for the machine learning models. Log transformations were applied to reduce skewness in large numerical variables, and interaction features were created to capture combined effects between variables.

The new engineered features were:

- **log_Population:** reduces skewness in population
- **log_Density:** reduces skewness in population density
- **log_Energy:** reduces skewness in energy consumption
- **Energy_x_Urban:** captures the combined effect of energy use and urbanization
- **GDP_x_Urban:** captures the combined effect of GDP growth and urbanization
- **Urban_x_Fert:** captures the interaction between urbanization and fertility rate

For classification, the target variable was created from CO₂ emissions per capita. A country-year observation was labelled as High Emitter if its CO₂ per capita was above the 75th percentile, and Low Emitter otherwise. The 75th percentile was selected because it identifies the top 25% of emitters while still keeping enough samples for model training. For regression, CO₂ per capita was transformed using $\log_{1p}()$ to create `log_CO2_target`, which helps reduce the effect of extreme values.

The selected features for modeling included energy consumption, urbanization, GDP growth, demographic variables, density, and engineered log/interaction features. These variables were selected because they are logically related to CO₂ emissions and supported by the EDA findings.

Code:

```
merged_final <- merged_final %>%
  mutate(
    log_Population = log1p(Population),
    log_Density = log1p(Density),
    log_Energy = log1p(Energy_Per_Capita_kWh),
    Energy_x_Urban = log1p(Energy_Per_Capita_kWh) * Urban_Pop_Pct,
    GDP_x_Urban = GDP_growth * Urban_Pop_Pct,
    Urban_x_Fert = Urban_Pop_Pct * Fertility_Rate
  )

t50 <- quantile(merged_final$CO2_per_capita, 0.50)
t75 <- quantile(merged_final$CO2_per_capita, 0.75)
t90 <- quantile(merged_final$CO2_per_capita, 0.90)

thresh_df <- data.frame(
  Threshold = c("50th pct", "75th pct (USED)", "90th pct"),
  Cutoff = round(c(t50, t75, t90), 3),
  Pct_High = round(c(mean(merged_final$CO2_per_capita > t50) * 100,
                        mean(merged_final$CO2_per_capita > t75) * 100,
                        mean(merged_final$CO2_per_capita > t90) * 100), 1)
)
print(thresh_df); save_table(thresh_df, "threshold_sensitivity")

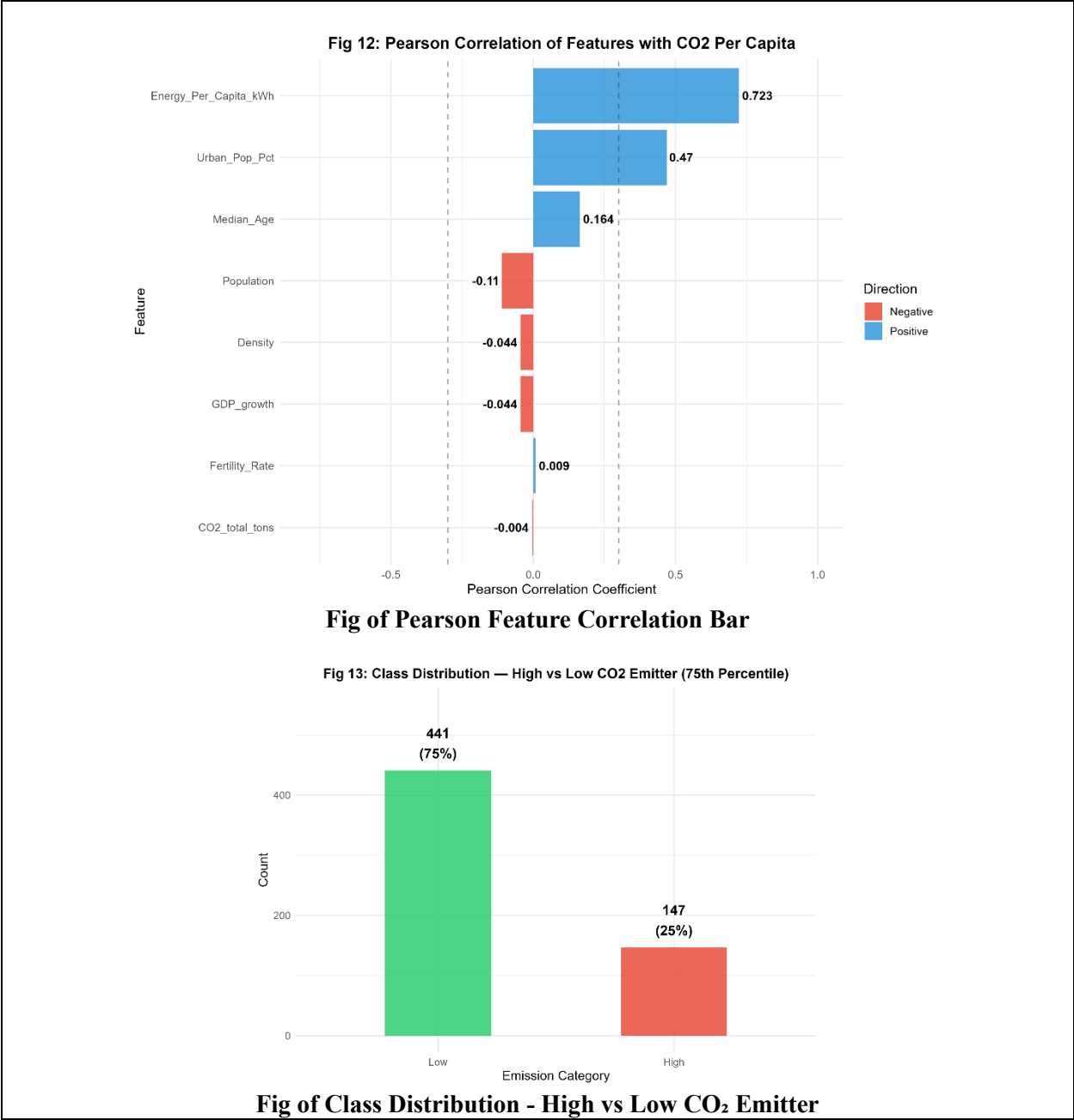
merged_final <- merged_final %>%
  mutate(
    High_Emitter = factor(ifelse(CO2_per_capita > t75, "High", "Low"),
                          levels=c("Low", "High")),
    log_CO2_target = log1p(CO2_per_capita)
  )
cat("\nClass distribution:\n"); print(table(merged_final$High_Emitter))

class_df <- as.data.frame(table(merged_final$High_Emitter)) %>%
  rename(Class=Var1, Count=Freq) %>%
  mutate(Pct=round(Count/sum(Count)*100, 1))

fig13 <- ggplot(class_df, aes(x=Class, y=Count, fill=Class)) +
  geom_bar(stat="identity", width=0.45, alpha=0.85) +
  geom_text(aes(label=paste0(Count, "\n(", Pct, "%)")), vjust=-0.4, size=5, fontface="bold") +
  scale_fill_manual(values=c("Low"="#2ECC71", "High"="#E74C3C")) +
  theme_minimal(base_size=13) +
  theme(legend.position="none", plot.title=element_text(face="bold", hjust=0.5, size=14)) +
  labs(title="Fig 13: Class Distribution – High vs Low CO2 Emitter (75th Percentile)",
       x="Emission Category", y="Count") +
  ylim(0, max(class_df$Count) * 1.25)
save_fig(fig13, "Fig13_class_distribution", width=8, height=6)

selected_features <- c(
  "Energy_Per_Capita_kWh", "Urban_Pop_Pct", "Yearly_Change_Pct", "Median_Age",
  "GDP_growth", "Fertility_Rate", "Density", "log_Population", "log_Energy",
  "log_Density", "Energy_x_Urban", "GDP_x_Urban", "Urban_x_Fert"
)

model_data <- merged_final %>%
  select(all_of(selected_features), High_Emitter, log_CO2_target, CO2_per_capita)
```



4.3 Model Matrix Creation and Data Splitting

After selecting the final features, the dataset was converted into a model-ready format using `model.matrix()`. This created the predictor matrix `X_all`, containing all selected input variables. The target variables were then separated into two parts: `High_Emitter` for classification and `log_CO2_target` for regression.

The dataset was split into training, validation, and test sets. A 70/15/15 split was used, where 70% of the data was used for training, 15% for validation, and 15% for final testing. A fixed random seed was used to make the split reproducible.

This split was important because the model needed to be trained and evaluated on separate data. The training set was used to build the models, the validation set was used for model tuning and comparison, and the test set was kept separate for final performance evaluation.

Code:

```
X_all <- model.matrix(
  High_Emitter ~ Energy_Per_Capita_kWh + Urban_Pop_Pct + Yearly_Change_Pct +
  Median_Age + GDP_growth + Fertility_Rate + Density + log_Population +
  log_Energy + log_Density + Energy_X_Urban + GDP_X_Urban + Urban_X_Fert - 1,
  data = model_data
) %>% as.data.frame()
names(X_all) <- make.names(names(X_all), unique=TRUE)
y_class <- model_data$High_Emitter
y_reg <- model_data$log_CO2_target

set.seed(123)
train_idx <- createDataPartition(y_class, p=0.70, list=FALSE)
remaining <- setdiff(seq_len(nrow(X_all)), train_idx)
val_idx <- sample(remaining, floor(0.15 * nrow(X_all)))
test_idx <- setdiff(remaining, val_idx)

X_train_raw <- X_all[train_idx,]; X_val_raw <- X_all[val_idx,]; X_test_raw <- X_all[test_
y_train <- y_class[train_idx]; y_val <- y_class[val_idx]; y_test <- y_class[test_
y_train_reg <- y_reg[train_idx]; y_val_reg <- y_reg[val_idx]; y_test_reg <- y_reg[test_

cat("\nSplit - Train:",nrow(X_train_raw),"| Val:",nrow(X_val_raw),"| Test:",nrow(X_test_r
```

Output:

```
> cat("\nSplit - Train:",nrow(X_train_raw),"| Val:",nrow(X_val_raw),"| Test:",nrow(X_test_r
w),"\n")
```

```
Split - Train: 412 | Val: 88 | Test: 88
```

```
> |
```

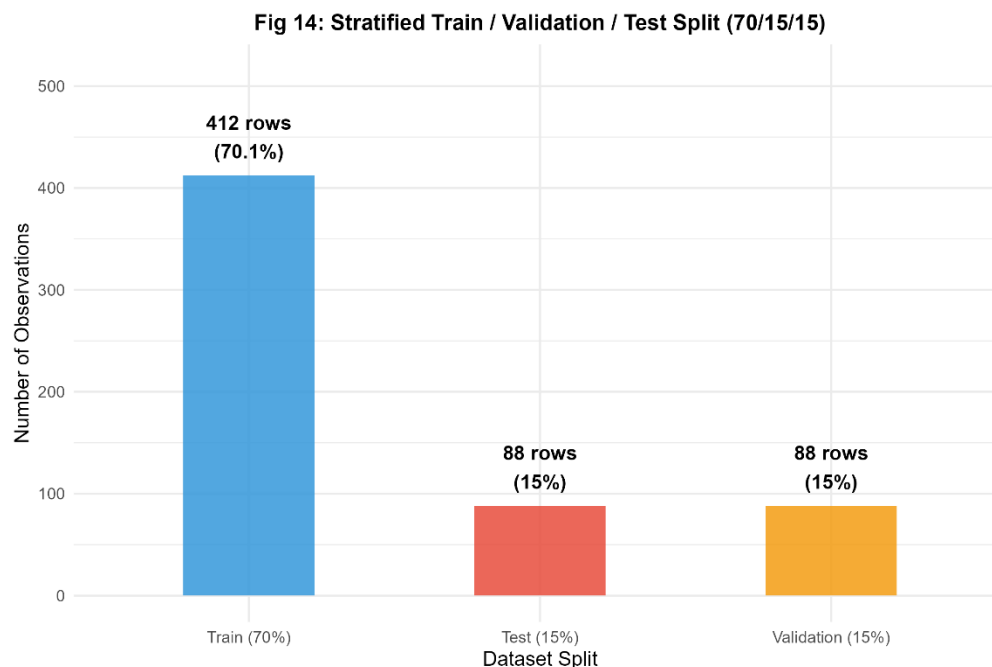


Fig of Stratified Train / Validation / Test Split

4.4 Scaling, SMOTE, and Cross-Validation Setup

After splitting the dataset, the selected features were scaled using Z-score standardization. The mean and standard deviation were calculated from the training set only, and the same values were applied to the validation and test sets. This step was important because models such as Logistic Regression and SVM are sensitive to feature scale.

After scaling, SMOTE was applied to handle class imbalance in the training data. Since the dataset had more Low CO₂ Emitter observations than High CO₂ Emitter observations, SMOTE generated synthetic samples for the minority class. SMOTE was applied only to the training set so that the validation and test sets remained realistic.

Finally, cross-validation was prepared to improve model reliability. The code compared 3-fold, 5-fold, and 10-fold cross-validation using AUC, sensitivity, and specificity. Based on the comparison, 5-fold cross-validation was selected as the final training setup because it provided a good balance between performance and computational cost.

Code:

```
train_means <- colMeans(X_train_raw, na.rm=TRUE)
train_sds   <- apply(X_train_raw, 2, sd, na.rm=TRUE)
train_sds[train_sds == 0] <- 1

scale_fn <- function(df, means, sds)
  as.data.frame(sweep(sweep(df, 2, means, "-"), 2, sds, "/"))

X_train <- scale_fn(X_train_raw, train_means, train_sds)
X_val   <- scale_fn(X_val_raw,   train_means, train_sds)
X_test  <- scale_fn(X_test_raw,  train_means, train_sds)

save_table(data.frame(Feature=names(train_means),
                      Mean=round(train_means,4), SD=round(train_sds,4)),
            "scaling_parameters")

cat("\n=== SMOTE ===\n")
cat("Before SMOTE:\n"); print(table(y_train))
smote_input <- data.frame(X_train, High_Emitter=y_train)
smote_res   <- SMOTE(X=smote_input[,!names(smote_input)%in%"High_Emitter"],
                    target=smote_input$High_Emitter, K=5, dup_size=0)
X_train_sm  <- smote_res$data[,!names(smote_res$data)%in%"class"]
y_train_sm  <- factor(smote_res$data$class, levels=c("Low","High"))
cat("After SMOTE:\n"); print(table(y_train_sm))
```

```

cv_comparison <- map_df(c(3,5,10), function(k) {
  ctrl <- trainControl(method="cv",number=k,classProbs=TRUE,
                        summaryFunction=twoClassSummary,savePredictions="final")
  set.seed(123)
  m <- train(High_Emitter~.,data=train_cv_df,method="glm",family="binomial",
             trControl=ctrl,metric="ROC")
  data.frame(CV_Method=paste0(k,"-Fold CV"),
             AUC_ROC=round(max(m$results$ROC),4),
             Sensitivity=round(max(m$results$Sens),4),
             Specificity=round(max(m$results$Spec),4))
})
print(cv_comparison); save_table(cv_comparison, "cv_comparison_3_5_10_fold")
--

```

Output:

```

> cat("Before SMOTE:\n"); print(table(y_train))
Before SMOTE:
y_train
Low High
309 103
> smote_input <- data.frame(X_train, High_Emitter=y_train)
> smote_res <- SMOTE(X=smote_input[,!names(smote_input)%in%"High_Emitter"],
+                   target=smote_input$High_Emitter, K=5, dup_size=0)
> X_train_sm <- smote_res$data[,!names(smote_res$data)%in%"class"]
> y_train_sm <- factor(smote_res$data$class, levels=c("Low","High"))
> cat("After SMOTE:\n"); print(table(y_train_sm))
After SMOTE:
y_train_sm
Low High
309 309
>
> print(cv_comparison); save_table(cv_comparison, "cv_comparison_3_5_10_fold")
  CV_Method AUC_ROC Sensitivity Specificity
1 3-Fold CV  0.9801      0.9709      0.9773
2 5-Fold CV  0.9848      0.9741      0.9870
3 10-Fold CV 0.9924      0.9709      0.9871

```

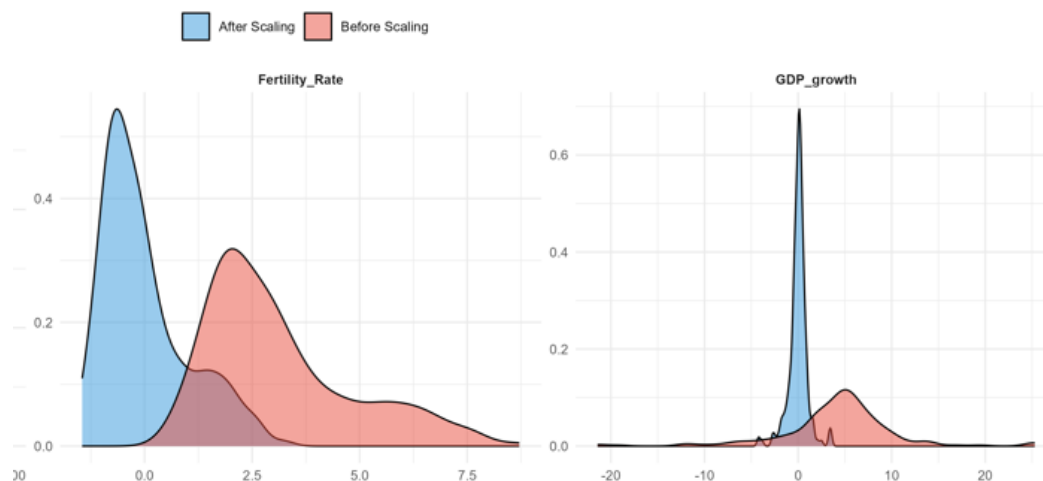


Fig of Feature distributions before and after Z-score standardization

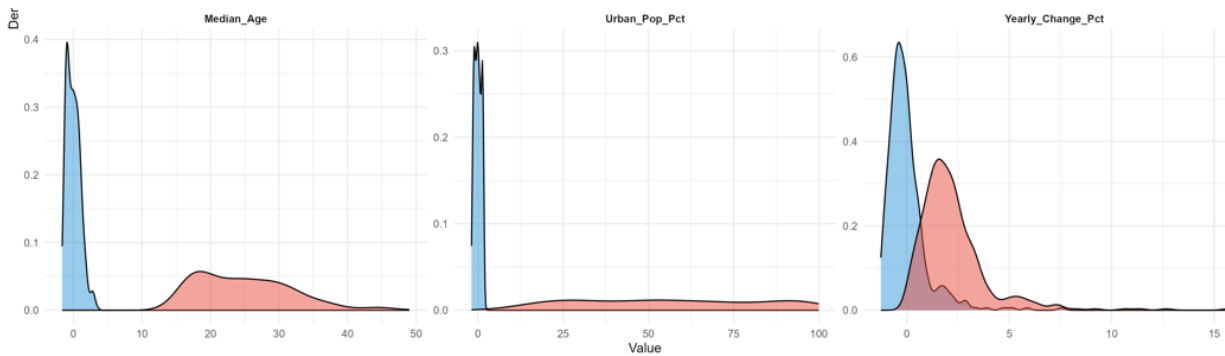


Fig of Feature distributions before and after Z-score standardization

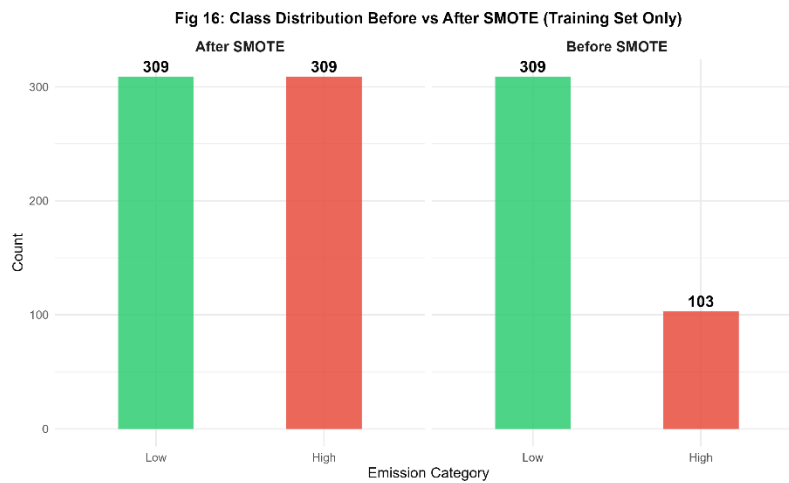


Fig of Class distribution before and after SMOTE

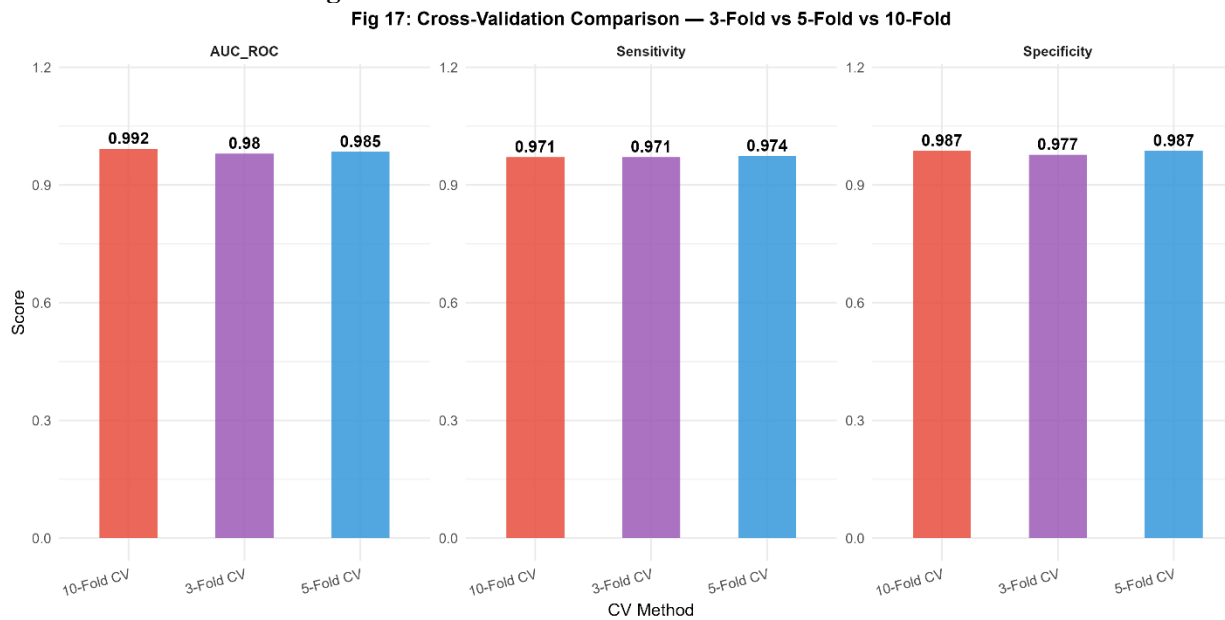


Fig of Cross-validation comparison between 3-fold, 5-fold, and 10-fold

5. Modeling and Analysis

Modeling was performed based on the project objective, which included both classification and regression tasks. The classification task predicted whether a country-year observation was a High CO₂ Emitter or Low CO₂ Emitter, while the regression task predicted the actual log-transformed CO₂ emissions per capita value. The code implemented five classification models and five regression models, then evaluated them using validation and test sets.

5.1 Classification Models

The classification task involved training five machine learning models to classify observations from Asian countries as either High CO₂ Emitter or Low CO₂ Emitter using a set of 13 features, such as energy consumption per capita, urbanization rate, GDP growth, demographic variables, and engineered interaction terms.

Logistic Regression has been added as a linear baseline. Based on the findings of the EDA, Logistic Regression was a logical choice to determine if a linear decision boundary would adequately separate the two emission classes as several features showed strong linear correlation with CO₂ per capita (e.g., Energy_Per_Capita_kWh, Pearson $r = 0.723$ and Urban_Pop_Pct, Pearson $r = 0.470$).

Decision Tree (CART) was added due to the curved, non-linear relationships between features and CO₂ per capita seen in the LOESS scatter plots in the EDA. It is also the structural unit of Random Forest, so its inclusion in itself shows the advantages of the ensemble over a single tree on this dataset.

Since the dataset is complex with feature interactions such as the engineered Energy_x_Urban variable, and highly skewed variables like CO₂ per capita and Energy_Per_Capita_kWh, Random Forest was chosen as the primary model. Random Forest constructs 500 trees on bootstrapped samples, smoothing out variance from the skewed distributions. Grid search was used to find the best mtry, which is 5 out of 13 features.

Support Vector Machine with RBF Kernel was chosen because for several features, Spearman correlations were significantly higher than Pearson correlations indicating non-linear monotonic relationships between them and CO₂ per capita. The RBF kernel maps features into a higher dimensional space to find a maximum margin boundary. The z-score standardisation had to be performed first. The model's optimal hyperparameters were $C = 5$ and $\sigma = 0.1$.

XGBoost was chosen because the dataset is multicollinear with some features having VIF values as high as 303 (Energy_x_Urban) and gradient boosting is known to work well with correlated features by iteratively correcting the errors in the previous model. To avoid overfitting on the training set (412 rows), early stopping on validation AUC was implemented.

Five models were trained on 618 rows of the training set after balancing the set with SMOTE and then evaluated on 88 rows of the validation set and 88 rows of the test set, using 5-fold cross validation. Accuracy, Precision, Recall, F1-score, AUC-ROC, Specificity and Balanced Accuracy were used as measures of performance. Recall was given a special emphasis because of the real world consequences of not identifying an actual High Emitter, as opposed to false alarms.

Code of Classification Models:

```
eval_caret <- function(model, X, y, model_name) {
  pred_class <- predict(model, X)
  pred_prob <- predict(model, X, type="prob")[,"High"]
  cm <- confusionMatrix(pred_class, y, positive="High")
  roc <- roc(as.numeric(y=="High"), pred_prob, quiet=TRUE)
  ci <- binom.test(sum(pred_class==y), length(y))$conf.int
  data.frame(Model=model_name,
    Accuracy=round(cm$overall["Accuracy"],4),
    Acc_CI_Low=round(ci[1],4), Acc_CI_High=round(ci[2],4),
    Precision=round(cm$byClass["Precision"],4),
    Recall=round(cm$byClass["Recall"],4),
    F1_Score=round(cm$byClass["F1"],4),
    AUC_ROC=round(auc(roc),4),
    Specificity=round(cm$byClass["Specificity"],4),
    Balanced_Acc=round(cm$byClass["Balanced Accuracy"],4))
}

eval_xgb <- function(cls, prob, y, name) {
  cm <- confusionMatrix(cls, y, positive="High")
  roc <- roc(as.numeric(y=="High"), prob, quiet=TRUE)
  ci <- binom.test(sum(cls==y), length(y))$conf.int
  data.frame(Model=name,
    Accuracy=round(cm$overall["Accuracy"],4),
    Acc_CI_Low=round(ci[1],4), Acc_CI_High=round(ci[2],4),
    Precision=round(cm$byClass["Precision"],4),
    Recall=round(cm$byClass["Recall"],4),
    F1_Score=round(cm$byClass["F1"],4),
    AUC_ROC=round(auc(roc),4),
    Specificity=round(cm$byClass["Specificity"],4),
    Balanced_Acc=round(cm$byClass["Balanced Accuracy"],4))
}

cat("\n Model 1: Logistic Regression \n")
set.seed(123)
model_lr <- train(High_Emitter~.,data=train_df,method="glm",family="binomial",
  trControl=ctrl_5fold,metric="ROC")
cat("LR CV AUC:", round(max(model_lr$results$ROC),4),"\n")

cat("\n Model 2: Decision Tree \n")
set.seed(123)
model_dt <- train(High_Emitter~.,data=train_df,method="rpart",
  trControl=ctrl_5fold,metric="ROC",tuneLength=10)
cat("DT CV AUC:", round(max(model_dt$results$ROC),4),"\n")

png(file.path(output_folder,"figures","Fig18_decision_tree.png"),
  width=1400,height=900,res=120)
rpart.plot(model_dt$finalModel,type=4,extra=101,
  main="Fig 18: Decision Tree - CO2 High Emitter Classification",
  cex=0.75,fallen.leaves=TRUE)
dev.off()

cat("\n Model 3: Random Forest \n")
set.seed(123)
model_rf <- train(High_Emitter~.,data=train_df,method="rf",
  trControl=ctrl_5fold,metric="ROC",
  tuneGrid=expand.grid(mtry=c(2,3,4,5,6)),ntree=500)
cat("RF best mtry:",model_rf$bestTune$mtry,"| CV AUC:",round(max(model_rf$results$ROC),4))

cat("\n Model 4: SVM (RBF Kernel) \n")
set.seed(123)
model_svm <- train(High_Emitter~.,data=train_df,method="svmRadial",
  trControl=ctrl_5fold,metric="ROC",
  tuneGrid=expand.grid(C=c(0.1,0.5,1,2,5),sigma=c(0.01,0.05,0.1,0.5)))
cat("SVM best C:",model_svm$bestTune$C,"| sigma:",model_svm$bestTune$sigma,"\n")
```

```

cat("\n Model 5: XGBoost \n")
X_tr_mat <- as.matrix(X_train_sm); X_va_mat <- as.matrix(X_val); X_te_mat <- as.matrix(X_val)
y_tr_xgb <- as.numeric(y_train_sm=="High")
y_va_xgb <- as.numeric(y_val=="High")
y_te_xgb <- as.numeric(y_test=="High")

dtrain <- xgb.DMatrix(data=X_tr_mat,label=y_tr_xgb)
dval <- xgb.DMatrix(data=X_va_mat,label=y_va_xgb)
dtest <- xgb.DMatrix(data=X_te_mat,label=y_te_xgb)

xgb_params <- list(objective="binary:logistic",eval_metric="auc",max_depth=4,
eta=0.05,subsample=0.8,colsample_bytree=0.8,
min_child_weight=1,gamma=0.1)
set.seed(123)
model_xgb <- xgb.train(params=xgb_params,data=dtrain,nrounds=300,
evals=list(train=dtrain,val=dval),verbose=0,
early_stopping_rounds=20)
cat("XGBoost best iteration:",model_xgb$best_iteration,"\n")

xgb_val_prob <- predict(model_xgb,dval)
xgb_test_prob <- predict(model_xgb,dtest)
xgb_val_cls <- factor(ifelse(xgb_val_prob > 0.5, "High", "Low"),levels=c("Low","High"))
xgb_test_cls <- factor(ifelse(xgb_test_prob > 0.5, "High", "Low"),levels=c("Low","High"))

```

Output:

```

=== Validation Results ===
> val_results <- bind_rows(
+   eval_caret(model_lr, X_val,y_val,"Logistic Regression"),
+   eval_caret(model_dt, X_val,y_val,"Decision Tree"),
+   eval_caret(model_rf, X_val,y_val,"Random Forest"),
+   eval_caret(model_svm,X_val,y_val,"SVM (RBF)"),
+   eval_xgb(xgb_val_cls,xgb_val_prob,y_val,"XGBoost")
+ )
> print(val_results); save_table(val_results,"validation_results_all_models")

```

	Model	Accuracy	Acc_CI_Low	Acc_CI_High	Precision	Recall	F1_Score
Accuracy...1	Logistic Regression	0.9318	0.8575	0.9746	0.7917	0.95	0.8636
Accuracy...2	Decision Tree	0.9659	0.9036	0.9929	0.9048	0.95	0.9268
Accuracy...3	Random Forest	0.9659	0.9036	0.9929	0.9048	0.95	0.9268
Accuracy...4	SVM (RBF)	0.9659	0.9036	0.9929	0.9048	0.95	0.9268
Accuracy...5	XGBoost	0.9659	0.9036	0.9929	0.8696	1.00	0.9302

```

+   AUC_ROC Specificity Balanced_Acc
Accuracy...1 0.9853 0.9265 0.9382
Accuracy...2 0.9904 0.9706 0.9603
Accuracy...3 0.9971 0.9706 0.9603
Accuracy...4 0.9934 0.9706 0.9603
Accuracy...5 0.9982 0.9559 0.9779
Saved: D:/dsProj/project_outputs/tables/validation_results_all_models.csv
>

=== Test Results ===
> test_results <- bind_rows(
+   eval_caret(model_lr, X_test,y_test,"Logistic Regression"),
+   eval_caret(model_dt, X_test,y_test,"Decision Tree"),
+   eval_caret(model_rf, X_test,y_test,"Random Forest"),
+   eval_caret(model_svm,X_test,y_test,"SVM (RBF)"),
+   eval_xgb(xgb_test_cls,xgb_test_prob,y_test,"XGBoost")
+ )
> print(test_results); save_table(test_results,"test_results_all_models")

```

	Model	Accuracy	Acc_CI_Low	Acc_CI_High	Precision	Recall	F1_Score
Accuracy...1	Logistic Regression	0.9318	0.8575	0.9746	0.8000	1.0000	0.8889
Accuracy...2	Decision Tree	0.9205	0.8430	0.9674	0.8400	0.8750	0.8571
Accuracy...3	Random Forest	0.9659	0.9036	0.9929	0.8889	1.0000	0.9412
Accuracy...4	SVM (RBF)	0.9432	0.8724	0.9813	0.8276	1.0000	0.9057
Accuracy...5	XGBoost	0.9205	0.8430	0.9674	0.8148	0.9167	0.8627

```

+   AUC_ROC Specificity Balanced_Acc
Accuracy...1 0.9880 0.9062 0.9531
Accuracy...2 0.9030 0.9375 0.9062
Accuracy...3 0.9915 0.9531 0.9766
Accuracy...4 0.9837 0.9219 0.9609
Accuracy...5 0.9844 0.9219 0.9193
Saved: D:/dsProj/project_outputs/tables/test_results_all_models.csv
>

```

Fig 18: Decision Tree — CO2 High Emitter Classification

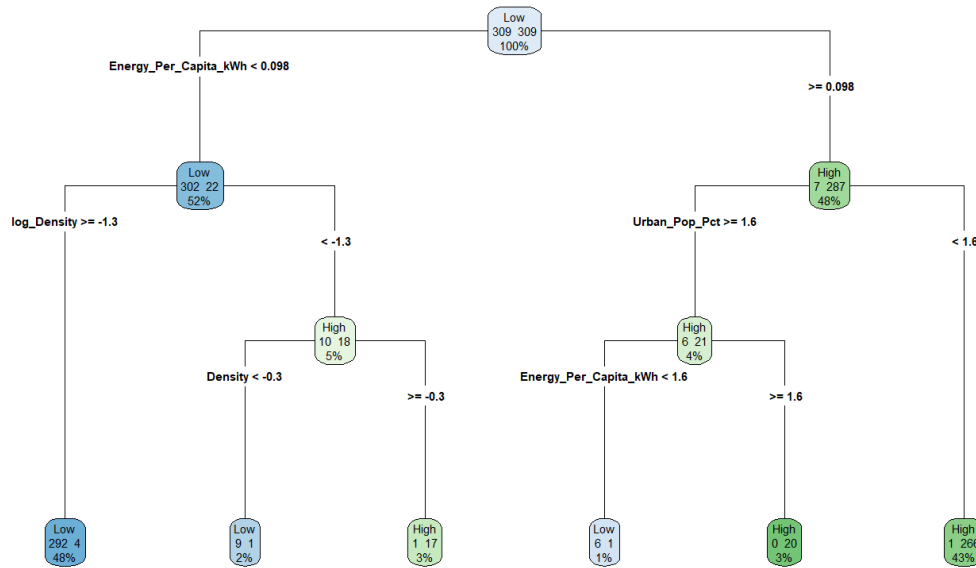


Fig of Decision tree model used for interpretable CO₂ emitter classification

Fig 19: Confusion Matrices — All Models (Validation Set)

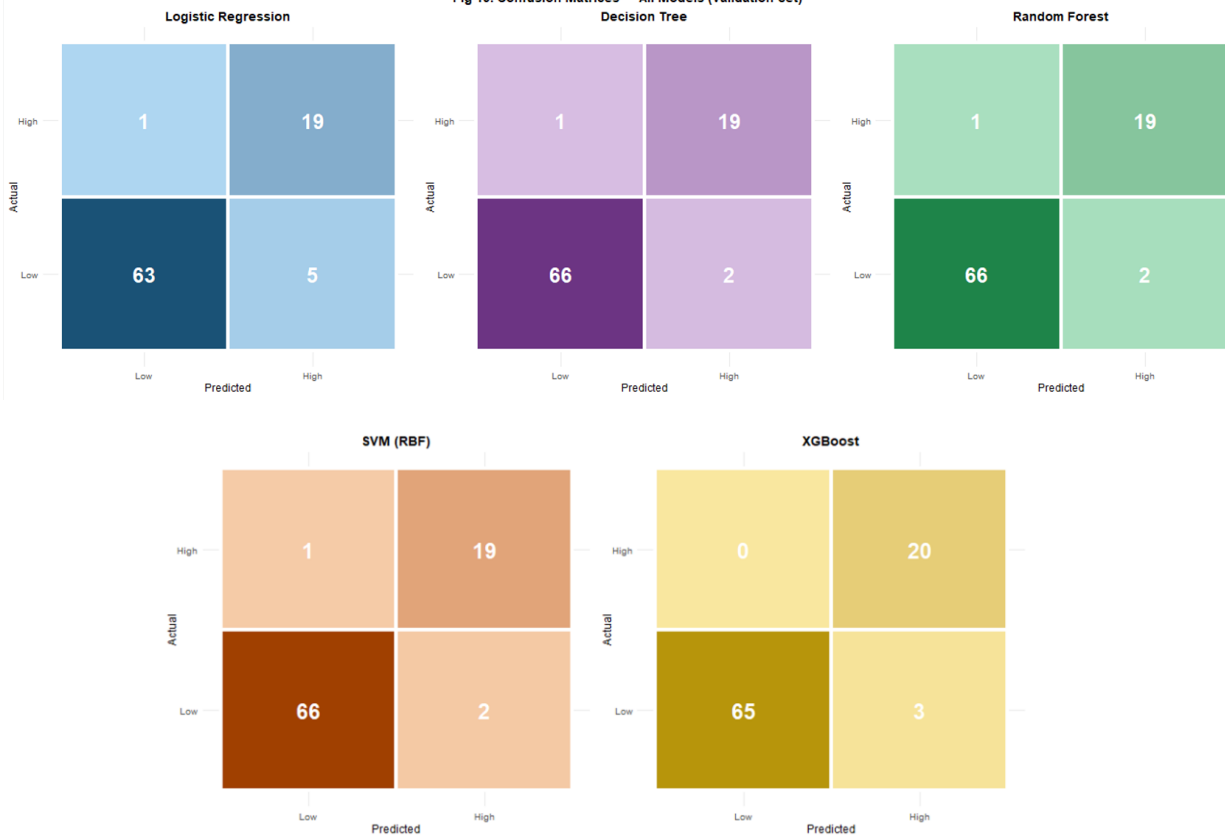


Fig of Confusion Matrices

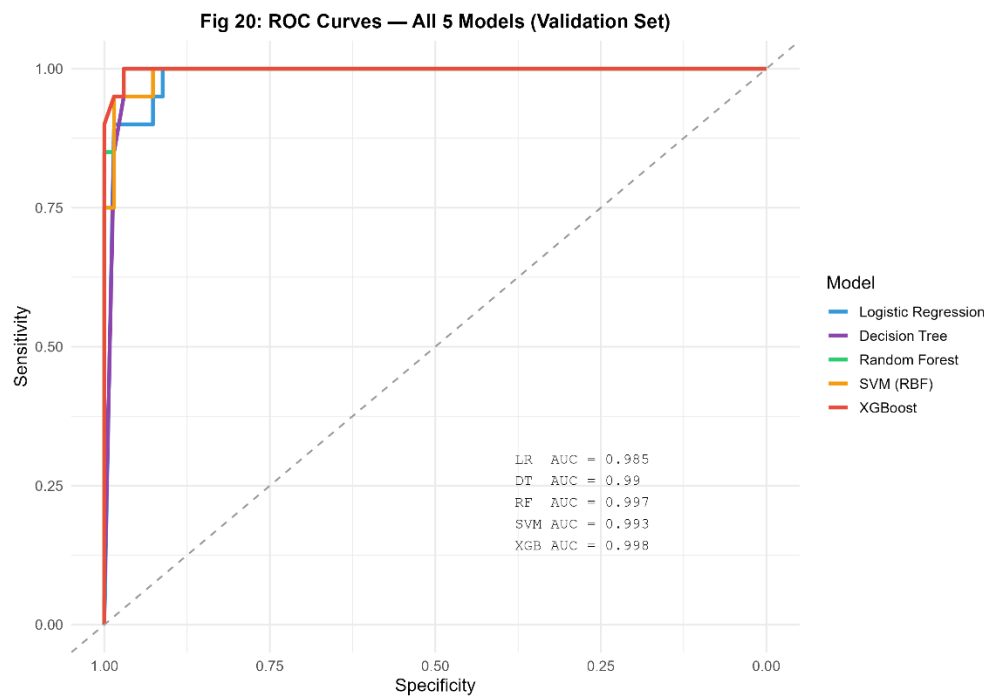


Fig of ROC Curves

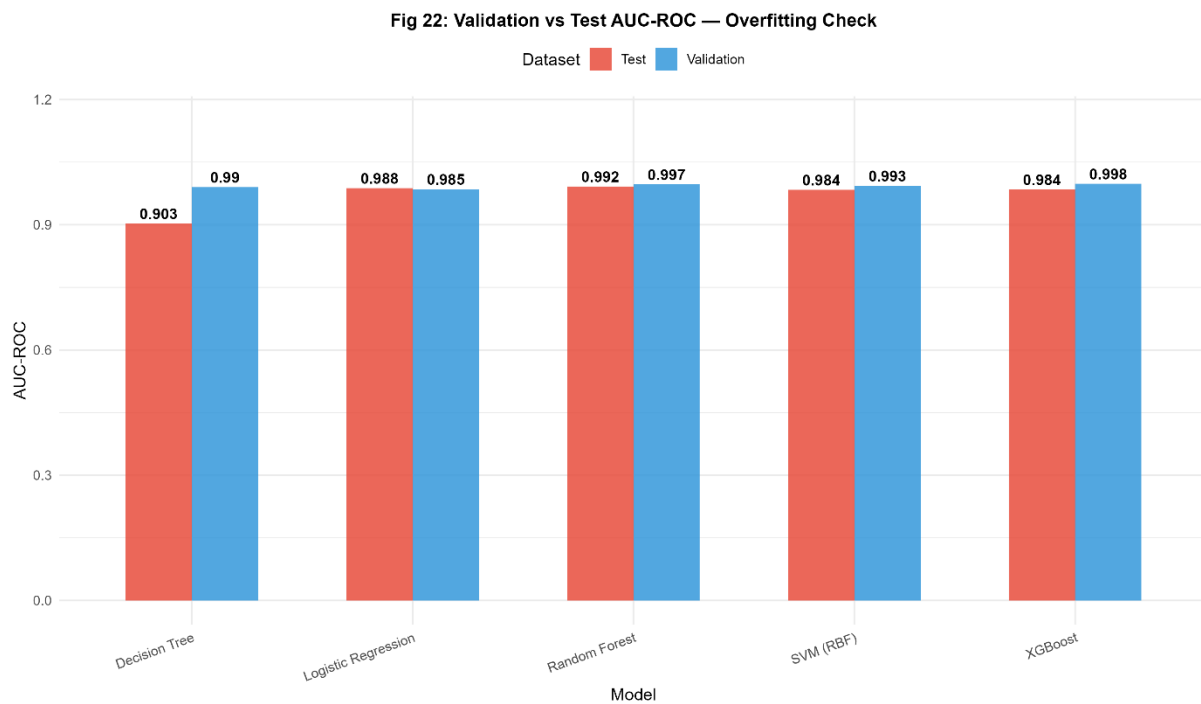
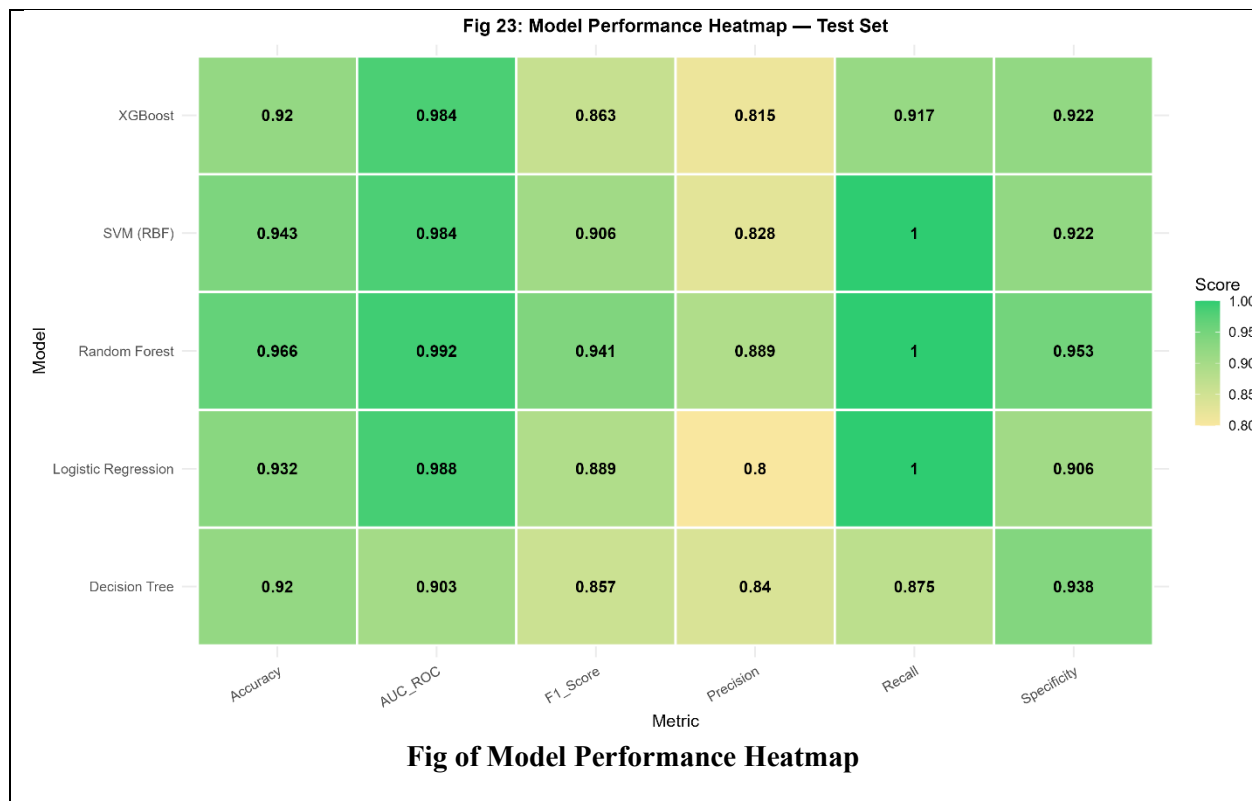


Fig of Validation vs Test AUC-ROC



5.2 Regression Models

Five models were trained for the regression task to predict `log_CO2_target` which is the log transformed CO₂ emissions per capita. A log transformation was applied as CO₂ per capita has a high right skew (skewness = 4.94) and predicting on the log scale limits the impact of extreme values in the Gulf states (e.g. Qatar and Kuwait) during model training.

Linear Regression was included as the baseline model. It assumes that there is a linear relationship between the 13 input features and log CO₂ per capita. The Shapiro-Wilk test of the residuals gave a p value of 0.0025 meaning that the residuals are not normally distributed, which is a violation of the assumption of OLS. This outcome appears to support the use of more flexible non-linear models for this data, theoretically.

Ridge Regression was added because of the high VIF values for some of the features: `Energy_x_Urban` (VIF of 303) and `Urban_Pop_Pct` (VIF of 225). Ridge uses regularisation L2 which is used to shrink all the coefficients towards zero without eliminating any feature, thereby stabilising the coefficient estimates in the presence of multicollinearity. The parameter of the regularisation lambda was optimised by cross-validation and the optimal value was 0.087.

Lasso Regression was added to the list of the Ridge options as L1 regularisation shrinks some coefficients to zero, thus doing automatic feature selection. This was applicable since there were 13 features included, which may be redundant. Lasso chose 11 out of 13 features, leaving out the `Urban_x_Fert` and `GDP_x_Urban`, and arrived at the highest coefficient being `log_Energy` (0.697), which again indicates that energy use is the most important linear predictor.

Random Forest Regression was chosen due to the scatter plots and residual plot analysis indicating a non-linear association between CO₂ per capita and the predictors. Random Forest is an ensemble tree learning

method that does not assume any distribution and can learn such non-linear patterns. Grid search with 500 trees was used to determine that the best value for mtry was 3.

For gradient boosting, XGBoost Regression was chosen. It recursively reduces the sum of the squares of the errors, each tree correcting the errors from the previous tree. To avoid overfitting on the training set, the regularisation parameters $\eta = 0.05$, $\gamma = 0.1$ and early stopping on the validation RMSE were used.

For regression, the original scaled training set (412 rows) was used for training, and the validation and test sets (88 rows each) were used to test all the regression models, with 5-fold cross validation. RMSE, MAE, and R^2 were used for measuring the performance. These three metrics together give us a full picture: RMSE is more sensitive to large errors, MAE gives us the average absolute error, and R^2 tells us the amount of variance in log CO₂ per capita that is explained by the model.

Code:

```
# STEP 7: Regression Models + SHAP Explainability

eval_reg <- function(pred, actual, model_name) {
  rmse <- sqrt(mean((pred-actual)^2))
  mae <- mean(abs(pred-actual))
  r2 <- 1 - sum((pred-actual)^2)/sum((actual-mean(actual))^2)
  data.frame(Model=model_name, RMSE=round(rmse,4), MAE=round(mae,4), R2=round(r2,4))
}

ctrl_reg <- trainControl(method="cv",number=5)

cat("\n Reg Model 1: Linear Regression \n")
set.seed(123)
model_lm <- train(x=X_train,y=y_train_reg,method="lm",trControl=ctrl_reg)
cat("LM CV R^2:",round(max(model_lm$results$Rsquared),4),"\n")
pred_lm_val <- predict(model_lm,X_val)
pred_lm_test <- predict(model_lm,X_test)

cat("\n VIF Check \n")
vif_df <- data.frame(X_train, y=y_train_reg)
lm_vif <- lm(y~.,data=vif_df)
vif_vals <- tryCatch({
  v <- car::vif(lm_vif)
  data.frame(Feature=names(v),VIF=round(v,3)) %>% arrange(desc(VIF))
}, error=function(e) { cat("VIF error:",e$message,"\n"); NULL })
if (!is.null(vif_vals)) {
  print(vif_vals); save_table(vif_vals,"vif_results")
  high_vif <- vif_vals %>% filter(VIF>10)
  if (nrow(high_vif)>0) { cat("High VIF Features:\n"); print(high_vif) }
}

lm_resid <- pred_lm_test - y_test_reg
sw_test <- shapiro.test(sample(lm_resid, min(5000,length(lm_resid))))
cat("Shapiro-Wilk: W=",round(sw_test$statistic,4),"| p=",round(sw_test$p.value,4),"\n")
save_table(data.frame(Test="Shapiro-Wilk",W=round(sw_test$statistic,4),
  p_value=round(sw_test$p.value,4),
  Normal=ifelse(sw_test$p.value>=0.05,"Yes","No")),
  "residual_normality_test")

cat("\n Reg Model 2: Ridge Regression \n")
set.seed(123)
model_ridge <- train(x=X_train,y=y_train_reg,method="glmnet",trControl=ctrl_reg,
  tuneGrid=expand.grid(alpha=0,lambda=10^seq(-4,2,length=50)))
cat("Ridge best lambda:",round(model_ridge$bestTune$lambda,5),"\n")
pred_ridge_val <- predict(model_ridge,X_val)
pred_ridge_test <- predict(model_ridge,X_test)

cat("\n Reg Model 3: Lasso Regression \n")
set.seed(123)
model_lasso <- train(x=X_train,y=y_train_reg,method="glmnet",trControl=ctrl_reg,
  tuneGrid=expand.grid(alpha=1,lambda=10^seq(-4,2,length=50)))
cat("Lasso best lambda:",round(model_lasso$bestTune$lambda,5),"\n")
lasso_coef <- coef(model_lasso$finalModel, s=model_lasso$bestTune$lambda)
lasso_selected <- data.frame(Feature=row.names(lasso_coef),
  Coefficient=round(as.numeric(lasso_coef),5)) %>%
  filter(abs(Coefficient)>0, Feature!="(Intercept)") %>%
  arrange(desc(abs(Coefficient)))
cat("Lasso selected",nrow(lasso_selected),"Features:\n"); print(lasso_selected)
save_table(lasso_selected,"lasso_selected_features")
pred_lasso_val <- predict(model_lasso,X_val)
pred_lasso_test <- predict(model_lasso,X_test)
```

```

cat("\n Reg Model 4: Random Forest Regression \n")
set.seed(123)
model_rfr <- train(x=X_train,y=y_train_reg,method="rf",trControl=ctrl_reg,
                  tuneGrid=expand.grid(mtry=c(2,3,4,5,6)),ntree=500)
cat("RF Reg best mtry:",model_rfr$bestTune$mtry,"\n")
pred_rfr_val <- predict(model_rfr,X_val)
pred_rfr_test <- predict(model_rfr,X_test)

cat("\n Reg Model 5: XGBoost Regression \n")
dtrain_reg <- xgb.DMatrix(data=as.matrix(X_train),label=y_train_reg)
dval_reg <- xgb.DMatrix(data=as.matrix(X_val), label=y_val_reg)
dtest_reg <- xgb.DMatrix(data=as.matrix(X_test), label=y_test_reg)

xgb_reg_params <- list(objective="reg:squarederror",eval_metric="rmse",max_depth=4,
                      eta=0.05,subsample=0.8,colsample_bytree=0.8,
                      min_child_weight=1,gamma=0.1)

set.seed(123)
model_xgbr <- xgb.train(params=xgb_reg_params,data=dtrain_reg,nrounds=300,
                      evals=list(train=dtrain_reg,val=dval_reg),verbose=0,
                      early_stopping_rounds=20)
cat("XGBoost Reg best iteration:",model_xgbr$best_iteration,"\n")
pred_xgbr_val <- predict(model_xgbr,dval_reg)
pred_xgbr_test <- predict(model_xgbr,dtest_reg)

```

Output:

```

VIF Check
> vif_df <- data.frame(X_train, y=y_train_reg)
> lm_vif <- lm(y~.,data=vif_df)
> vif_vals <- tryCatch({
+   v <- car::vif(lm_vif)
+   data.frame(Feature=names(v),VIF=round(v,3)) %>% arrange(desc(VIF))
+ }, error=function(e) { cat("VIF error:",e$message,"\n"); NULL })
> if (!is.null(vif_vals)) {
+   print(vif_vals); save_table(vif_vals,"vif_results")
+   high_vif <- vif_vals %>% filter(VIF>10)
+   if (nrow(high_vif)>0) { cat("High VIF Features:\n"); print(high_vif) }
+ }

```

Feature	VIF
Energy_x_Urban	303.681
Urban_Pop_Pct	225.322
Energy_Per_Capita_kwh	11.665
log_Energy	10.513
Urban_x_Fert	8.477
Fertility_Rate	7.918
GDP_x_Urban	6.535
GDP_growth	6.386
Median_Age	3.917
log_Density	2.503
Density	2.247
Yearly_Change_Pct	2.182
log_Population	1.496

Saved: D:/dsProj/project_outputs/tables/vif_results.csv

High VIF features:

Feature	VIF
Energy_x_Urban	303.681
Urban_Pop_Pct	225.322
Energy_Per_Capita_kwh	11.665
log_Energy	10.513

```

Lasso selected 11 features:

```

	Feature	Coefficient
1	log_Energy	0.69717
2	Energy_Per_Capita_kwh	0.15699
3	log_Density	-0.15176
4	Yearly_Change_Pct	0.12544
5	Energy_x_Urban	0.09322
6	Median_Age	0.08311
7	Fertility_Rate	0.06158
8	GDP_x_Urban	-0.04739
9	GDP_growth	0.03349
10	log_Population	-0.01459
11	Density	-0.01444

```

> save_table(lasso_selected,"lasso_selected_features")

```

```

=== Regression Results - Validation ===
> reg_val <- bind_rows(
+   eval_reg(pred_lm_val, y_val_reg, "Linear Regression"),
+   eval_reg(pred_ridge_val, y_val_reg, "Ridge Regression"),
+   eval_reg(pred_lasso_val, y_val_reg, "Lasso Regression"),
+   eval_reg(pred_rfr_val, y_val_reg, "RF Regression"),
+   eval_reg(pred_xgbr_val, y_val_reg, "XGBoost Regression")
+ )
> print(reg_val); save_table(reg_val, "regression_validation_results")
  Model RMSE MAE R2
1 Linear Regression 0.2393 0.1811 0.9409
2 Ridge Regression 0.2497 0.1897 0.9356
3 Lasso Regression 0.2338 0.1748 0.9436
4 RF Regression 0.1689 0.1041 0.9705
5 XGBoost Regression 0.1551 0.1029 0.9752
Saved: D:/dsProj/project_outputs/tables/regression_validation_results.csv
>
> cat("\n=== Regression Results - Test ===\n")

=== Regression Results - Test ===
> reg_test <- bind_rows(
+   eval_reg(pred_lm_test, y_test_reg, "Linear Regression"),
+   eval_reg(pred_ridge_test, y_test_reg, "Ridge Regression"),
+   eval_reg(pred_lasso_test, y_test_reg, "Lasso Regression"),
+   eval_reg(pred_rfr_test, y_test_reg, "RF Regression"),
+   eval_reg(pred_xgbr_test, y_test_reg, "XGBoost Regression")
+ )
> print(reg_test); save_table(reg_test, "regression_test_results")
  Model RMSE MAE R2
1 Linear Regression 0.2359 0.1893 0.9450
2 Ridge Regression 0.2385 0.1914 0.9437
3 Lasso Regression 0.2302 0.1852 0.9476
4 RF Regression 0.1127 0.0834 0.9874
5 XGBoost Regression 0.1406 0.1038 0.9804
Saved: D:/dsProj/project_outputs/tables/regression_test_results.csv
>

```

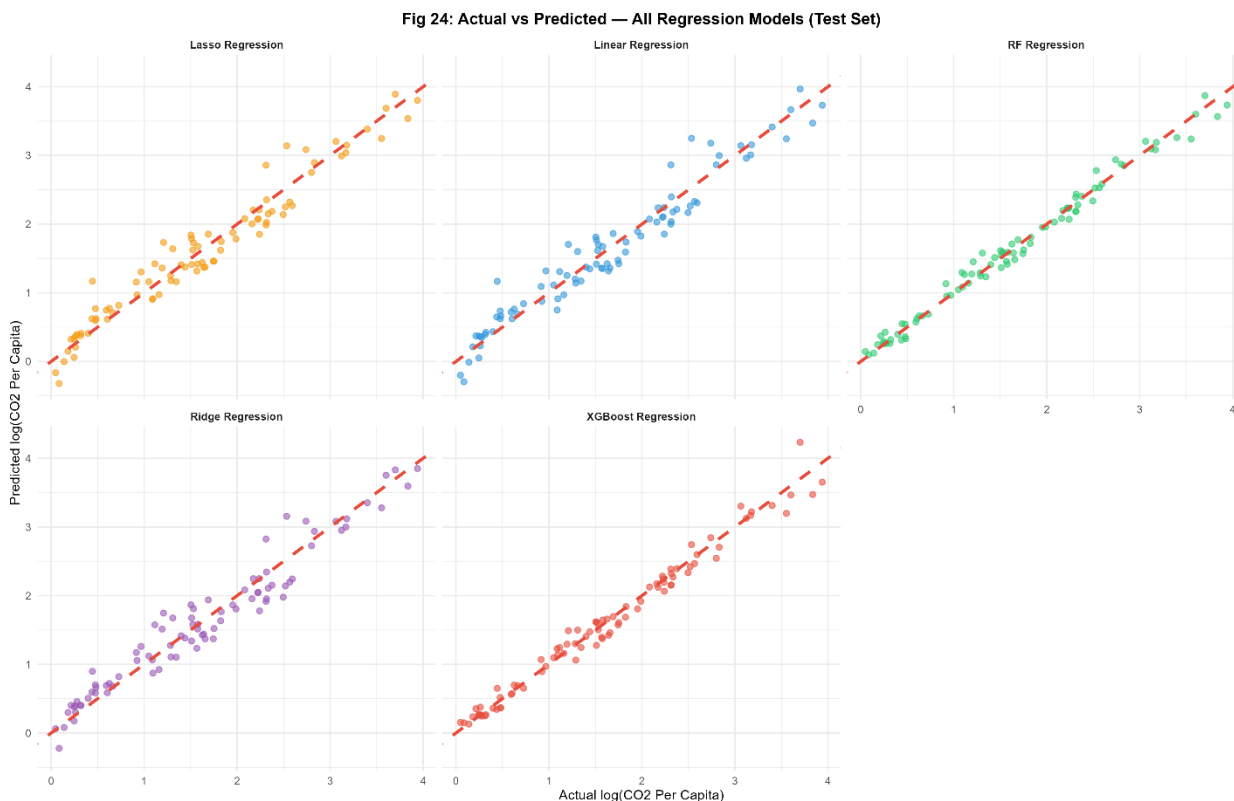


Fig of Actual vs Predicted in Regression Models

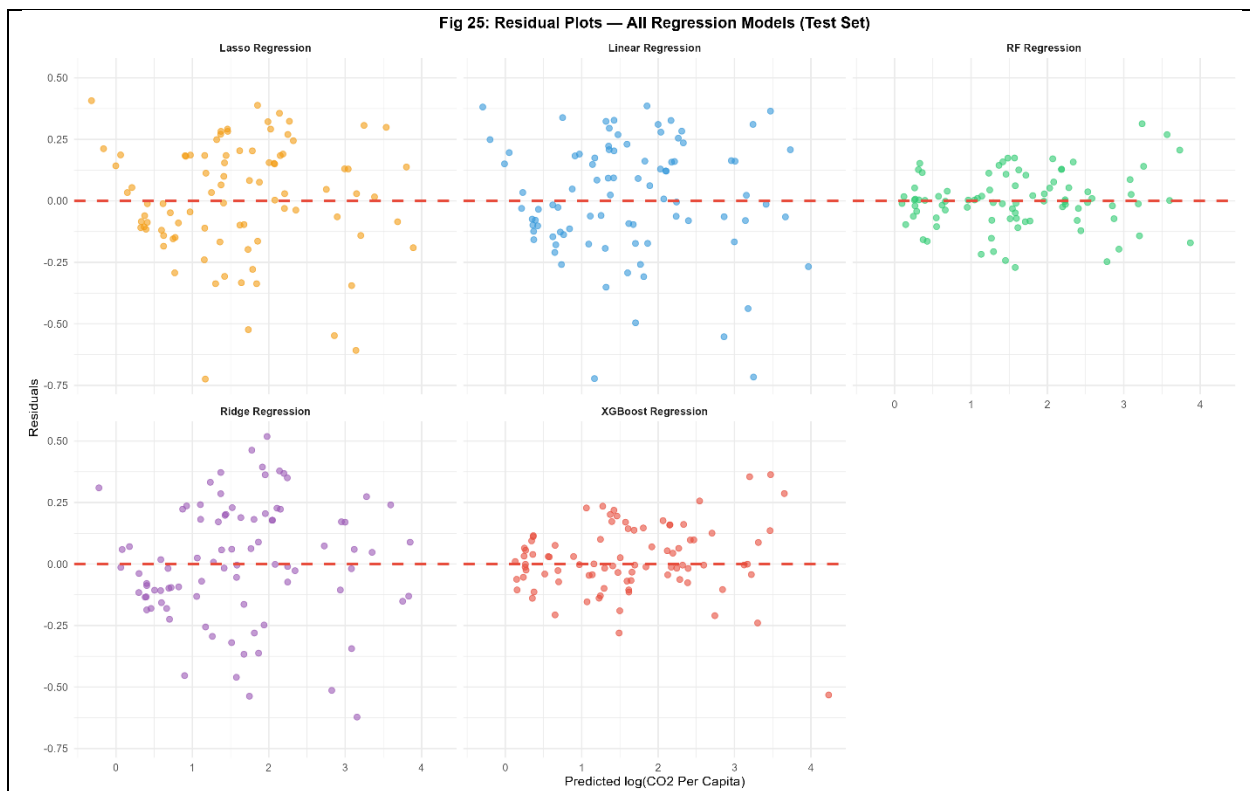


Fig of Residual Plots for Regression Models

Fig 26: Q-Q Plot — Linear Regression Residuals (Test Set)

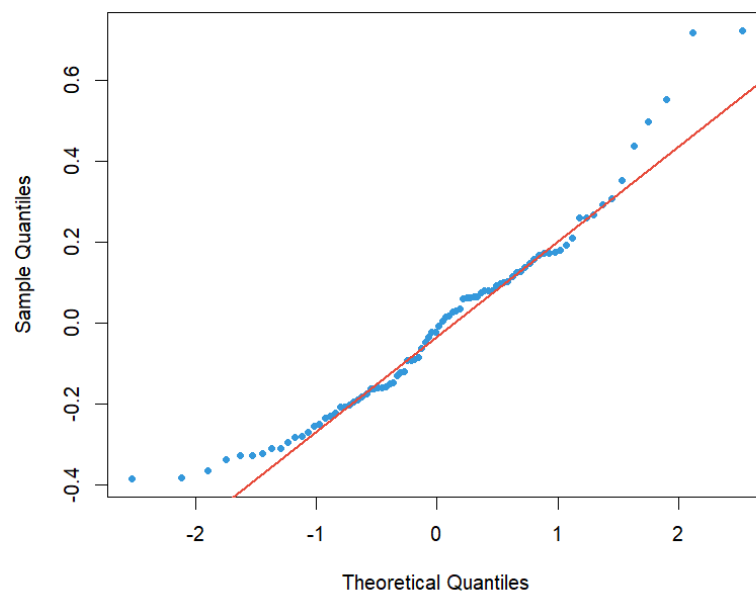
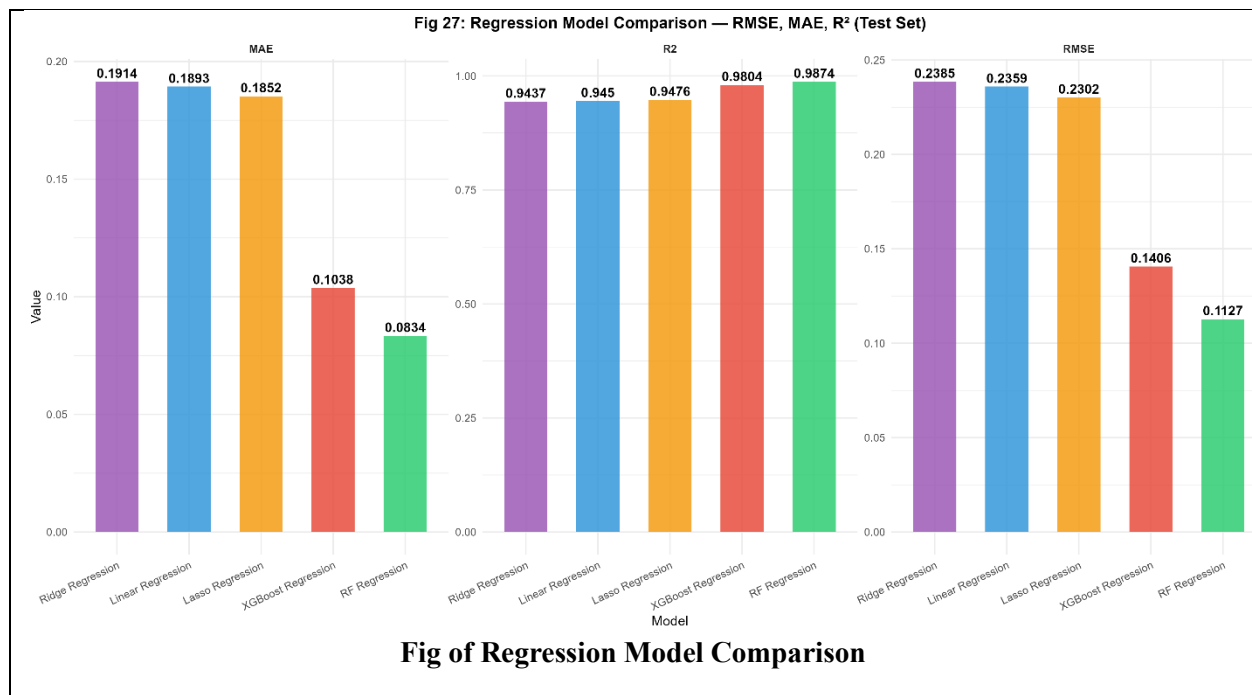


Fig of Q-Q Plot for Linear Regression Residuals



5.3 Summary of Modeling and Analysis

In this section, both classification and regression models were implemented according to the research objective.

- Classification models predicted High or Low CO₂ Emitter.
- Regression models predicted the actual log-transformed CO₂ per capita value.
- Five classification models and five regression models were trained.
- Models were evaluated using validation and test datasets.
- Classification performance was measured using accuracy, precision, recall, F1-score, AUC-ROC, specificity, and balanced accuracy.
- Regression performance was measured using RMSE, MAE, and R².
- Validation and test results were compared to check model generalization and possible overfitting.

6. Results and Interpretation

6.1 Classification Results

The classification models were evaluated using validation and test data. Among the five classification models, Random Forest performed the best overall. It achieved the highest test performance, with strong accuracy, AUC-ROC, F1-score, and recall.

The Random Forest classifier achieved:

- **Accuracy:** 96.6%
- **AUC-ROC:** 0.992
- **F1-score:** 0.941
- **Recall:** 1.000

This means the model was highly effective at identifying High CO₂ Emitters. The recall value of 1.000 is especially important because it means the model did not miss actual High Emitter cases in the test set. This is important for the research objective because missing a high-emission country would be a more serious issue in environmental analysis.

Table 1: Classification Model Performance for Test Set

Model	Accuracy	AUC-ROC	F1-Score	Precision	Recall	Specificity
Random Forest	0.966	0.992	0.941	0.889	1.000	0.953
Logistic Regression	0.932	0.988	0.889	0.800	1.000	0.906
SVM (RBF)	0.943	0.984	0.906	0.828	1.000	0.922
XGBoost	0.920	0.984	0.863	0.815	0.917	0.922
Decision Tree	0.920	0.903	0.857	0.840	0.875	0.938

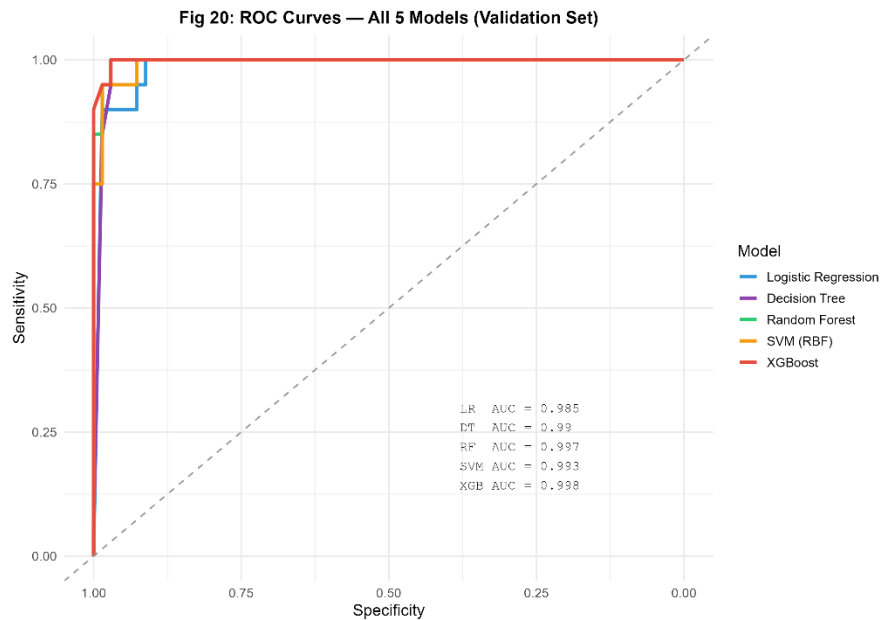


Fig of ROC Curves for All Classification Models

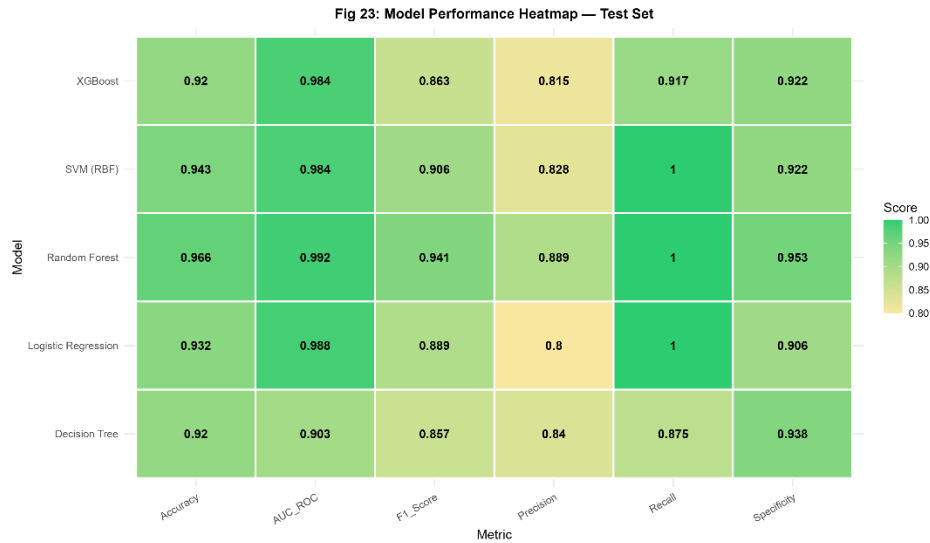


Fig of Test Set Performance Heatmap

6.2 Regression Results

The regression models were evaluated using RMSE, MAE, and R^2 . Among the regression models, Random Forest Regression achieved the best result.

The Random Forest Regression model achieved:

- **RMSE:** 0.1127
- **MAE:** 0.0834
- **R^2 :** 0.9874

This means the model explained about 98.7% of the variation in log-transformed CO₂ emissions per capita. The low RMSE and MAE values show that the model made accurate predictions. Random Forest Regression performed better than the linear models because CO₂ emissions are influenced by non-linear relationships between energy use, urbanization, population, and other socioeconomic variables.

Table 2: Regression Model Performance for Test Set

Model	RMSE	MAE	R^2
RF Regression	0.1127	0.0834	0.9874
XGBoost Regression	0.1406	0.1038	0.9804
Lasso Regression	0.2302	0.1852	0.9476
Linear Regression	0.2359	0.1893	0.9450
Ridge Regression	0.2385	0.1914	0.9437

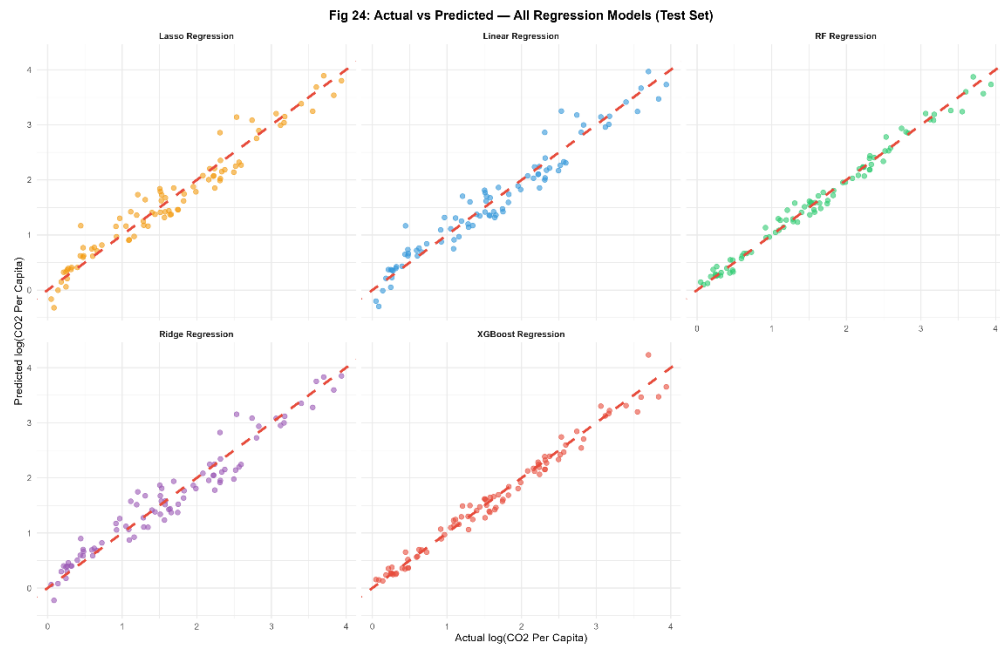


Fig of Actual vs Predicted for Regression Models



Fig of Regression Model Comparison for RMSE, MAE, R²

6.3 Model Interpretation

SHAP (SHapley Additive exPlanations) analysis was used to interpret the predictions of the best-performing Random Forest models, both for the classification and regression models. The SHAP value indicates how much each feature contributes to the prediction (either towards High Emitter or Low Emitter).

The most influential predictor for the classification model was Energy_Per_Capita_kWh with a mean absolute SHAP value of 0.2304, significantly higher than any other variable. The second most important variable was log_Energy with a score of 0.1734, reinforcing the importance of energy consumption. The fourth most important interaction was between energy and urbanization (Energy_x_Urban); in other words, high energy use and high urbanization gives rise to a higher CO₂ prediction than each factor individually.

For the regression model, the log_Energy and Energy_Per_Capita_kWh were almost equal with 0.2792 and 0.2782 respectively, which proves that energy usage also influences the exact amount of CO₂ per capita. Urban_Pop_Pct was 0.0717, the fourth of the secondary drivers, confirming the importance of urbanization. GDP_growth also had a very small contribution to the SHAP values across both of the models, which is also in line with the near-zero value of the Pearson correlation coefficient (scoring: -0.044) when analyzing features. The SHAP values for GDP_growth were also very small in both models, and the Pearson correlation coefficient (scoring: -0.044) was also near zero when analyzing features.

The ranking of the top three variables by the Random Forest variable importance based on Gini impurity was the same as that by SHAP, which is another measure of reliability in explainability analysis.

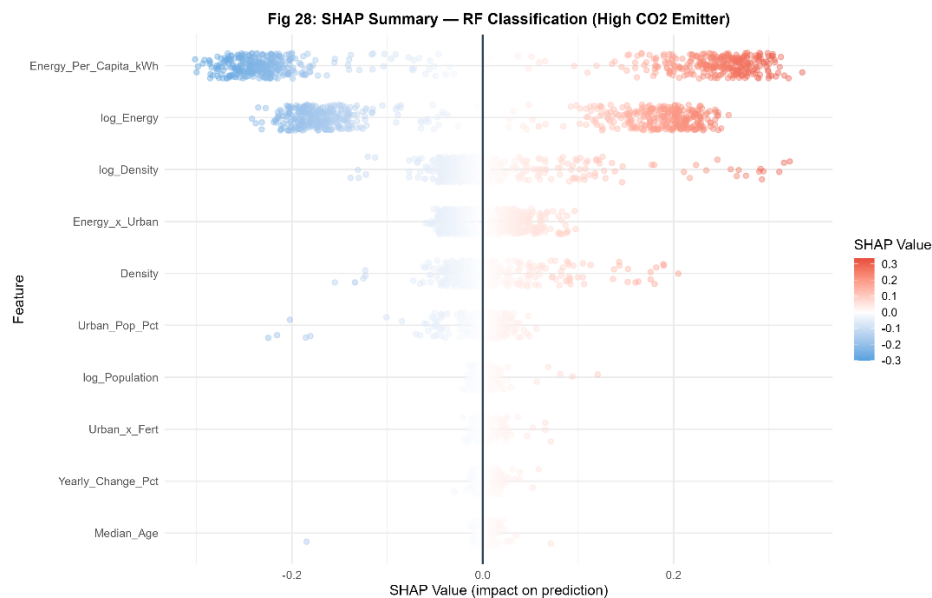


Fig of shap_summary_classification

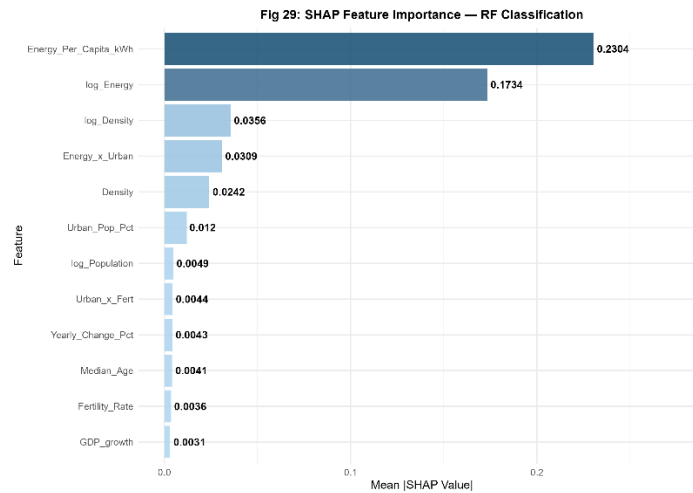


Fig of SHAP Feature Importance for Classification

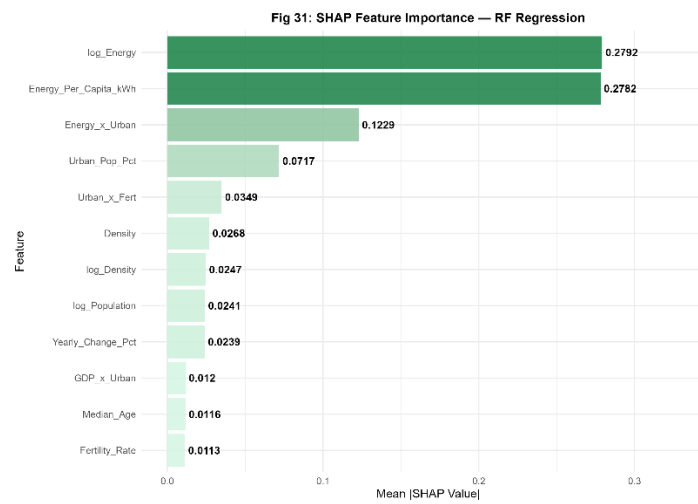


Fig of SHAP Feature Importance for Regression

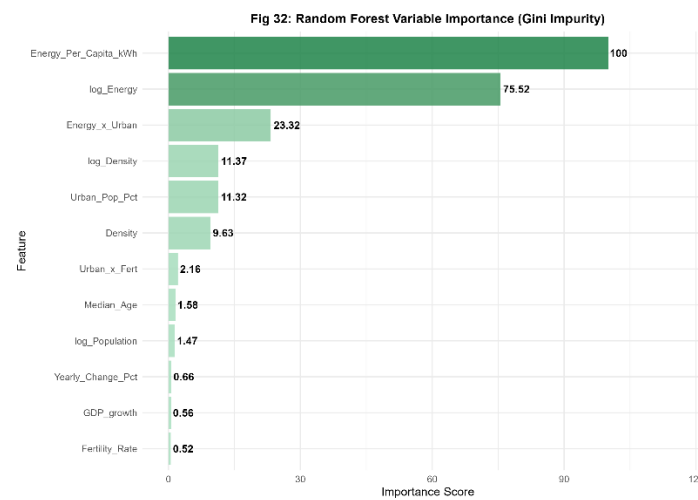


Fig of Random Forest Variable Importance

6.4 Achievement of Research Objective

The research objective was achieved successfully. The project aimed to predict whether an Asian country in a given year is a High or Low CO₂ Emitter and also predict the actual CO₂ emissions per capita value.

The classification objective was achieved because the Random Forest classifier produced very strong test results with high accuracy and AUC-ROC. The regression objective was also achieved because Random Forest Regression produced a very high R² value and low prediction error.

The results show that machine learning models can effectively predict CO₂ emission patterns using socioeconomic, demographic, urbanization, and energy-related features. The findings also show that energy consumption per capita is the most important factor influencing CO₂ emissions across Asian countries.

7. Conclusion and Future Work

Conclusion

This project was able to create a machine learning pipeline to predict CO₂ emissions per capita across 48 Asian countries, based on data scraped from four public sources: Worldometer, World Bank, and Our World in Data, and aggregated into 588 observations from 1970 to 2024. Following extensive pre-processing, such as filling in missing values, outlier detection (IQR and Z-score) and Winsorisation, SMOTE balanced and Z-score scaled, five classification and five regression models were trained and tested on the train-validation-test set with a stratified 70/15/15 split and 5-fold cross validation.

Random Forest out performed all other classifiers with AUC-ROC of 0.992 and F1-score of 0.941 on the test set, while having a Recall of 100%: no actual high emitter was missed. Random Forest Regression was the best regressor, accounting for 98.7% of the variance in CO₂ per capita with R² = 0.987 and RMSE = 0.113. SHAP explainability validated that energy use per capita, urbanization, and the interaction between energy use per capita and urbanization are the key factors influencing CO₂ emissions in Asia with marginal predictive power of the GDP growth rate. The goal of the research was completely accomplished.

Limitations

The data is not continuous over the years, so that temporal analysis is limited and time-series forecasting techniques cannot be used. While the 75th percentile threshold for the High Emitter classification is almost meaningful, it is not based on an internationally agreed policy threshold, and changing the threshold would change the class balance and the result of the model. There was evidence of serious multicollinearity, with the VIFs as high as 303 for Energy_x_Urban and 225 for Urban_Pop_Pct, which suggests that the results of Linear Regression are not reliable for making causal inferences and that the models should be replaced by Ridge, Lasso and ensemble methods. In addition, some potentially confounding factors were missing from the data sets scrapped, such as energy mix (renewable vs. fossil fuel) and industrial structure and government climate policy. Therefore, none of the results of this project are causal relationships per se, but rather are statistical associations between the measured features and CO₂ emissions.

Future Work

In future extensions of this work, energy mix attributes including percentages of renewable energy and dependency on coal could be added, offering a considerable increase in the predictive power and policy interpretability. The Panel regression models with fixed or random country effects may be more suitable to capture the unobserved country-specific heterogeneity. Being able to have the data available as continuous annual information would allow time-series forecasting with ARIMA or LSTM models to forecast future CO2 trajectories. A natural experiment technique (pre and post Paris Agreement (2015) or pre and post COVID-19 disruption (2020)) could give causal information on the impact of policy intervention on the drivers of emissions in Asia.

Appendix

Full Code :

```
# 1. Install & Load Libraries

packages <- c(
  "tidyverse", "skimr", "DataExplorer", "naniar",
  "corrplot", "ggcorrplot", "RColorBrewer", "gridExtra",
  "ggpubr", "scales", "viridis", "reshape2",
  "caret", "randomForest", "xgboost", "e1071",
  "smotefamily", "ROSE", "pROC", "MLmetrics",
  "fastshap", "vip", "car", "nortest",
  "moments", "GGally", "gggridges", "knitr",
  "kableExtra", "kernlab", "tibble", "rpart",
  "rpart.plot", "rvest", "dplyr", "stringr",
  "purrr", "tidyr", "readr"
)

new_packages <- packages[!(packages %in% installed.packages()[,"Package"])]
if(length(new_packages)) install.packages(new_packages)

library(tidyverse); library(skimr); library(DataExplorer); library(naniar)
library(corrplot); library(ggcorrplot); library(RColorBrewer); library(gridExtra)
library(ggpubr); library(scales); library(viridis); library(reshape2)
library(caret); library(randomForest); library(xgboost); library(e1071)
library(smotefamily); library(ROSE); library(pROC); library(MLmetrics)
library(fastshap); library(vip); library(car); library(nortest)
library(moments); library(GGally); library(gggridges); library(knitr)
library(kableExtra); library(kernlab); library(tibble)
library(rpart); library(rpart.plot)
library(rvest); library(dplyr); library(stringr); library(purrr)
library(tidyr); library(readr)

cat("All libraries loaded.\n")

# SOUTH ASIAN NUMBER FORMATTING HELPERS

format_sa <- function(x, digits = 2) {
  dplyr::case_when(
    abs(x) >= 1e7 ~ paste0(round(x / 1e7, digits), " Cr"),
```

```

abs(x) >= 1e5 ~ paste0(round(x / 1e5, digits), " L"),
abs(x) >= 1e3 ~ paste0(round(x / 1e3, digits), " K"),
TRUE      ~ as.character(round(x, digits))
)
}

label_sa <- function(digits = 2) {
  function(x) format_sa(x, digits)
}

cat("South Asian formatters ready: format_sa() and label_sa()\n")

# 2. Scrape Population Data

wanted_cols <- c(
  "country", "Year", "Population", "Yearly % Change",
  "Yearly Change", "Migrants (net)", "Median Age",
  "Fertility Rate", "Density (P/Km²)", "Urban Pop %",
  "Urban Population", "Country's Share of World Pop",
  "World Population", "Global Rank"
)

asia_url <- "https://www.worldometers.info/population/countries-in-asia-by-population/"
webpage <- read_html(asia_url)

country_data <- data.frame(
  country = webpage %>% html_elements("table a") %>% html_text2(),
  link    = webpage %>% html_elements("table a") %>% html_attr("href"),
  stringsAsFactors = FALSE
) %>%
  filter(!is.na(link)) %>%
  filter(str_detect(link, "^/world-population/.+-population/?$")) %>%
  mutate(
    country = str_squish(country),
    link    = paste0("https://www.worldometers.info", link)
  ) %>%
  distinct(link, .keep_all = TRUE)

print(country_data)

scrape_population_table <- function(country_name, country_url) {
  page <- read_html(country_url)
  tables <- page %>% html_elements("table") %>% html_table(fill = TRUE, convert = FALSE)
  target_table <- NULL
  for (tbl in tables) {
    names(tbl) <- str_squish(names(tbl))
    names(tbl) <- str_replace_all(names(tbl), "Â²", "2")
    if (any(str_detect(names(tbl), regex("^Year$", ignore_case = TRUE))) &&
        any(str_detect(names(tbl), regex("^Population$", ignore_case = TRUE)))) {
      target_table <- tbl
      break
    }
  }
  if (is.null(target_table)) return(NULL)
  names(target_table) <- str_squish(names(target_table))
  names(target_table) <- str_replace_all(names(target_table), "Â²", "2")
}

```

```

rank_col <- names(target_table)[str_detect(names(target_table), "Global Rank")]
if (length(rank_col) == 1) names(target_table)[names(target_table) == rank_col] <- "Global Rank"
target_table %>%
  mutate(country = country_name, .before = 1) %>%
  select(any_of(wanted_cols)) %>%
  mutate(across(everything(), as.character))
}

all_tables <- map2(
  country_data$country,
  country_data$link,
  ~{
    cat("Scraping:", .x, "\n")
    tryCatch({ Sys.sleep(1); scrape_population_table(.x, .y) },
      error = function(e) { cat("Failed:", .x, "\n"); NULL })
  }
)

all_tables <- all_tables[!sapply(all_tables, is.null)]
population_data <- bind_rows(all_tables) %>% select(any_of(wanted_cols))
View(population_data)

write.csv(population_data, "D:/dsProj/worldometer_asia_population.csv", row.names = FALSE)

# 3. Scrape CO2 Data

wanted_years <- c(2026,2025,2024,2023,2022,2020,2015,2010,
  2005,2000,1995,1990,1985,1980,1975,1970,1965,1960,1955)

wanted_cols_co2 <- c(
  "country", "Year",
  "Fossil CO2 emissions (tons)",
  "CO2 emissions change",
  "CO2 emissions per capita",
  "Share of World's CO2 emissions"
)

clean_names_custom <- function(x) {
  x %>%
    str_replace_all("CO₂", "CO2") %>%
    str_replace_all("Â²", "2") %>%
    str_replace_all("\u00a0", " ") %>%
    str_squish()
}

asia_co2_url <- "https://www.worldometers.info/co2-emissions/co2-emissions-by-country/?region=asia"
webpage_co2 <- read_html(asia_co2_url)

asia_country_data <- data.frame(
  country = webpage_co2 %>% html_elements("table a") %>% html_text2(),
  link = webpage_co2 %>% html_elements("table a") %>% html_attr("href"),
  stringsAsFactors = FALSE
) %>%
  filter(!is.na(link)) %>%
  filter(str_detect(link, "^/co2-emissions/.+co2-emissions/?$")) %>%
  mutate(

```

```

country = str_squish(country),
link = paste0("https://www.worldometers.info", link)
) %>%
distinct(link, .keep_all = TRUE)

print(asia_country_data)

scrape_country_co2 <- function(country_name, country_url) {
  page <- read_html(country_url)
  tables <- page %>% html_elements("table") %>% html_table(fill = TRUE, convert = FALSE)
  target_table <- NULL
  for (tbl in tables) {
    names(tbl) <- clean_names_custom(names(tbl))
    if (any(str_detect(names(tbl), regex("Fossil CO2 emissions", ignore_case = TRUE))) &&
        any(str_detect(names(tbl), regex("CO2 emissions per capita", ignore_case = TRUE)))) {
      target_table <- tbl; break
    }
  }
  if (is.null(target_table)) { cat("No CO2 table for:", country_name, "\n"); return(NULL) }
  names(target_table) <- clean_names_custom(names(target_table))
  names(target_table)[1] <- "Year"
  names(target_table) <- names(target_table) %>%
    str_replace_all("Fossil CO2 Emissions \\(tons\\)", "Fossil CO2 emissions (tons)") %>%
    str_replace_all("Fossil CO2 emissions \\(tons\\)", "Fossil CO2 emissions (tons)") %>%
    str_replace_all("CO2 emissions Change", "CO2 emissions change") %>%
    str_replace_all("CO2 emissions Per Capita", "CO2 emissions per capita") %>%
    str_replace_all("Share of World's CO2 Emissions", "Share of World's CO2 emissions")
  target_table %>%
    mutate(country = country_name, .before = 1) %>%
    mutate(across(everything(), as.character)) %>%
    mutate(Year = as.integer(str_extract(Year, "\\d{4}"))) %>%
    filter(Year %in% wanted_years) %>%
    select(any_of(wanted_cols_co2))
}

all_co2_tables <- map2(
  asia_country_data$country,
  asia_country_data$link,
  ~{
    cat("Scraping:", .x, "\n")
    tryCatch({ Sys.sleep(1); scrape_country_co2(.x, .y) },
      error = function(e) { cat("Failed:", .x, "-", conditionMessage(e), "\n"); NULL } )
  }
)

all_co2_tables <- all_co2_tables[!sapply(all_co2_tables, is.null)]
asia_co2_selected_years <- bind_rows(all_co2_tables) %>%
  mutate(country = factor(country, levels = asia_country_data$country)) %>%
  arrange(country, desc(Year)) %>%
  mutate(country = as.character(country))

View(asia_co2_selected_years)
write.csv(asia_co2_selected_years, "D:/dsProj/asia_co2_selected_years.csv", row.names = FALSE)

# 4. GDP Data — Load & Clean

```

```

gdp_file      <- "D:/dsProj/API_NY.GDP.MKTP.KD.ZG_DS2_en_csv_v2_121708.csv"
population_file <- "D:/dsProj/worldometer_asia_population.csv"
co2_file      <- "D:/dsProj/asia_co2_selected_years.csv"
output_file   <- "D:/dsProj/asia_gdp_growth_fixed.csv"

```

```

standardize_country <- function(x) {
  x <- str_squish(x)
  case_when(
    x %in% c("Korea, Rep.", "Republic of Korea") ~ "South Korea",
    x %in% c("Korea, Dem. People's Rep.") ~ "North Korea",
    x == "Viet Nam" ~ "Vietnam",
    x %in% c("Turkiye", "Türkiye") ~ "Turkey",
    x == "Iran, Islamic Rep." ~ "Iran",
    x == "Lao PDR" ~ "Laos",
    x == "Kyrgyz Republic" ~ "Kyrgyzstan",
    x == "Syrian Arab Republic" ~ "Syria",
    x == "Yemen, Rep." ~ "Yemen",
    x == "Brunei Darussalam" ~ "Brunei",
    x == "Hong Kong SAR, China" ~ "Hong Kong",
    x == "Macao SAR, China" ~ "Macao",
    x == "Russian Federation" ~ "Russia",
    x %in% c("West Bank and Gaza", "State of Palestine") ~ "Palestine",
    x == "Taiwan, China" ~ "Taiwan",
    TRUE ~ x
  )
}

```

```

population_data_raw <- read.csv(population_file, stringsAsFactors = FALSE)
asia_co2_selected_years <- read.csv(co2_file, stringsAsFactors = FALSE)

```

```

population_data_raw <- population_data_raw %>%
  mutate(country = standardize_country(country), Year = as.integer(Year))
asia_co2_selected_years <- asia_co2_selected_years %>%
  mutate(country = standardize_country(country), Year = as.integer(Year))

```

```

asia_countries <- intersect(unique(population_data_raw$country),
  unique(asia_co2_selected_years$country))
common_years <- intersect(unique(population_data_raw$Year),
  unique(asia_co2_selected_years$Year))
asia_countries <- sort(asia_countries)
common_years <- sort(common_years)

```

```

cat("\nCommon Asian countries:\n"); print(asia_countries)
cat("\nCommon years:\n"); print(common_years)

```

```

# Check countries in population but missing in CO2
missing_in_co2 <- setdiff(unique(population_data_raw$country),
  unique(asia_co2_selected_years$country))
cat("\nCountries in population data but missing in CO2 data:\n")
print(missing_in_co2)

```

```

# Check countries in CO2 but missing in population
missing_in_population <- setdiff(unique(asia_co2_selected_years$country),
  unique(population_data_raw$country))
cat("\nCountries in CO2 data but missing in population data:\n")
print(missing_in_population)

```

```

gdp_raw <- read.csv(gdp_file, skip = 4, check.names = FALSE, stringsAsFactors = FALSE)
gdp_raw <- gdp_raw[, names(gdp_raw) != ""]
gdp_raw <- gdp_raw[, sapply(gdp_raw, function(col)
  !all(is.na(col) | str_squish(as.character(col)) == ""))]
names(gdp_raw) <- str_squish(names(gdp_raw))

gdp_clean <- gdp_raw %>%
  rename(country_original = `Country Name`, country_code = `Country Code`,
    indicator_name = `Indicator Name`, indicator_code = `Indicator Code`) %>%
  mutate(country_original = str_squish(country_original),
    country = standardize_country(country_original)) %>%
  pivot_longer(cols = matches("^\\d{4}$"), names_to = "Year",
    values_to = "GDP_growth_annual_percent") %>%
  mutate(Year = as.integer(Year),
    GDP_growth_annual_percent = as.numeric(GDP_growth_annual_percent))

asia_gdp_fixed <- gdp_clean %>%
  filter(country %in% asia_countries, Year %in% common_years) %>%
  select(country, country_original, country_code, Year, GDP_growth_annual_percent) %>%
  arrange(country, desc(Year))

# Check missing GDP countries after name fixing
missing_from_gdp <- setdiff(asia_countries, unique(gdp_clean$country))
cat("\nAsian countries missing from World Bank GDP after name fixing:\n")
print(missing_from_gdp)

cat("\nCountries with missing GDP values:\n")
print(asia_gdp_fixed %>% filter(is.na(GDP_growth_annual_percent)) %>%
  count(country, name = "missing_gdp_rows") %>% arrange(desc(missing_gdp_rows)))

View(asia_gdp_fixed)
write.csv(asia_gdp_fixed, output_file, row.names = FALSE)
cat("\nFixed GDP file saved:", output_file, "\n")

# 5. Energy Data — Load & Filter

asian_countries <- c(
  "India", "China", "Indonesia", "Pakistan", "Bangladesh", "Japan", "Philippines",
  "Vietnam", "Iran", "Turkey", "Thailand", "Myanmar", "South Korea", "Iraq",
  "Afghanistan", "Yemen", "Uzbekistan", "Malaysia", "Saudi Arabia", "Nepal",
  "North Korea", "Syria", "Sri Lanka", "Taiwan", "Kazakhstan", "Cambodia",
  "Jordan", "United Arab Emirates", "Tajikistan", "Azerbaijan", "Israel",
  "Laos", "Turkmenistan", "Kyrgyzstan", "Hong Kong", "Singapore", "Lebanon",
  "State of Palestine", "Oman", "Kuwait", "Georgia", "Mongolia", "Qatar",
  "Armenia", "Bahrain", "Timor-Leste", "Cyprus", "Bhutan", "Macao", "Maldives", "Brunei"
)

pop_years <- c(2026, 2025, 2024, 2023, 2022, 2020, 2015, 2010,
  2005, 2000, 1995, 1990, 1985, 1980, 1975, 1970, 1965, 1960, 1955)

energy_raw <- read.csv("D:/dsProj/per-capita-energy-use.csv", stringsAsFactors = FALSE)
cat("Raw energy dims:", dim(energy_raw), "\n")

energy_clean_full <- energy_raw %>%
  rename(country = entity, Year = year,

```

```

    Energy_Per_Capita_kWh = primary_energy_consumption_per_capita_kwh) %>%
select(country, Year, Energy_Per_Capita_kWh)

energy_asia <- energy_clean_full %>%
  filter(country %in% asian_countries, Year %in% pop_years) %>%
  arrange(match(country, asian_countries), desc(Year))

cat("\nFiltered dims:", dim(energy_asia), "\n")

full_grid <- expand.grid(country = asian_countries, Year = pop_years,
  stringsAsFactors = FALSE)
missing <- full_grid %>% anti_join(energy_asia, by = c("country", "Year")) %>%
  arrange(country, desc(Year))
cat("\nMissing country-year combinations:", nrow(missing), "\n")
if (nrow(missing) > 0) { cat("Missing countries:\n"); print(unique(missing$country)) }

write.csv(energy_asia, "D:/dsProj/asia_energy_per_capita.csv", row.names = FALSE)
cat("\n Saved: D:/dsProj/asia_energy_per_capita.csv \n")
cat("Rows:", nrow(energy_asia), "\n")
cat("Countries:", length(unique(energy_asia$country)), "\n")
cat("\nPreview:\n")
print(head(energy_asia, 20))

# Working directory & output folders

setwd("D:/dsProj")
output_folder <- "D:/dsProj/project_outputs"
dir.create(output_folder, showWarnings = FALSE)
dir.create(file.path(output_folder, "figures"), showWarnings = FALSE)
dir.create(file.path(output_folder, "tables"), showWarnings = FALSE)

save_fig <- function(plot_obj = NULL, filename, width = 10, height = 7) {
  path <- file.path(output_folder, "figures", paste0(filename, ".png"))
  if (!is.null(plot_obj)) {
    ggsave(path, plot = plot_obj, width = width, height = height, dpi = 300)
  } else {
    dev.copy(png, path, width = width * 100, height = height * 100)
    dev.off()
  }
  cat("Saved:", path, "\n")
}

save_table <- function(df, filename) {
  path <- file.path(output_folder, "tables", paste0(filename, ".csv"))
  write.csv(df, path, row.names = FALSE)
  cat("Saved:", path, "\n")
}

set.seed(123)

# Load all 4 datasets

population <- read.csv("worldometer_asia_population.csv",
  stringsAsFactors = FALSE, fileEncoding = "UTF-8")
gdp <- read.csv("asia_gdp_growth_fixed.csv",
  stringsAsFactors = FALSE, fileEncoding = "UTF-8")

```

```

co2      <- read.csv("asia_co2_selected_years.csv",
                    stringsAsFactors = FALSE, fileEncoding = "UTF-8")
energy   <- read.csv("asia_energy_per_capita.csv",
                    stringsAsFactors = FALSE, fileEncoding = "UTF-8")

cat("\n Population \n")
cat("Dims:", dim(population), "\n")
cat("Columns:", paste(names(population), collapse = ", "), "\n")
cat("Countries:", length(unique(population$country)), "\n")
cat("Years:", paste(sort(unique(population$Year)), collapse = ", "), "\n")

cat("\n GDP \n")
cat("Dims:", dim(gdp), "\n")
cat("Columns:", paste(names(gdp), collapse = ", "), "\n")
cat("Countries:", length(unique(gdp$country)), "\n")
cat("Years:", paste(sort(unique(gdp$Year)), collapse = ", "), "\n")

cat("\n CO2 \n")
cat("Dims:", dim(co2), "\n")
cat("Columns:", paste(names(co2), collapse = ", "), "\n")
cat("Countries:", length(unique(co2$country)), "\n")
cat("Years:", paste(sort(unique(co2$Year)), collapse = ", "), "\n")

cat("\n Energy \n")
cat("Dims:", dim(energy), "\n")
cat("Columns:", paste(names(energy), collapse = ", "), "\n")
cat("Countries:", length(unique(energy$country)), "\n")
cat("Years:", paste(sort(unique(energy$Year)), collapse = ", "), "\n")

cat("\n Missing Values per Dataset \n")
cat("Population NAs:", sum(is.na(population)), "\n")
cat("GDP NAs:",      sum(is.na(gdp)),      "\n")
cat("CO2 NAs:",      sum(is.na(co2)),      "\n")
cat("Energy NAs:",   sum(is.na(energy)),   "\n")

# STEP 2: Clean & Merge All 4 Datasets

population_clean <- population %>%
  mutate(
    Population      = as.numeric(gsub(",", "", Population)),
    Yearly.Change    = as.numeric(gsub(",", "", Yearly.Change)),
    Migrants_net     = as.numeric(gsub(",", "", Migrants.net.)),
    Density          = as.numeric(gsub(".", "", Density..P.Km..)),
    Urban_Population = as.numeric(gsub(",", "", Urban.Population)),
    World_Population = as.numeric(gsub(",", "", World.Population)),
    Yearly_Change_Pct = as.numeric(gsub("%", "", Yearly...Change)),
    Urban_Pop_Pct    = as.numeric(gsub("%", "", Urban.Pop..)),
    World_Share_Pct  = as.numeric(gsub("%", "", Country.s.Share.of.World.Pop)),
    Median_Age       = as.numeric(Median.Age),
    Fertility_Rate    = as.numeric(Fertility.Rate),
    Global_Rank       = as.numeric(Global.Rank),
    Year             = as.integer(Year)
  ) %>%
  select(country, Year, Population, Yearly_Change_Pct,
         Migrants_net, Median_Age, Fertility_Rate, Density,
         Urban_Pop_Pct, Urban_Population, Global_Rank)

```



```

gdp_clean <- gdp %>%
  select(country, Year, GDP_growth = GDP_growth_annual_percent) %>%
  mutate(Year = as.integer(Year))

co2_clean <- co2 %>%
  mutate(
    CO2_total_tons = as.numeric(gsub(",", "", Fossil.CO2.emissions..tons.)),
    CO2_change_pct = as.numeric(gsub("[^0-9\\.\\-]", "",
                                     gsub("\\u2212", "-", CO2.emissions.change))),
    World_CO2_share = as.numeric(gsub("%", "", Share.of.World.s.CO2.emissions)),
    CO2_per_capita = as.numeric(CO2.emissions.per.capita),
    Year = as.integer(Year)
  ) %>%
  select(country, Year, CO2_total_tons, CO2_per_capita, CO2_change_pct, World_CO2_share)

energy_clean <- energy %>%
  mutate(Year = as.integer(Year)) %>%
  select(country, Year, Energy_Per_Capita_kWh)

common_years <- Reduce(intersect, list(
  unique(population_clean$Year),
  unique(gdp_clean$Year),
  unique(co2_clean$Year),
  unique(energy_clean$Year)
))
cat("Common years across all 4 datasets:\n"); print(sort(common_years))

merged_data <- population_clean %>%
  filter(Year %in% common_years) %>%
  inner_join(gdp_clean %>% filter(Year %in% common_years), by = c("country", "Year")) %>%
  inner_join(co2_clean %>% filter(Year %in% common_years), by = c("country", "Year")) %>%
  inner_join(energy_clean %>% filter(Year %in% common_years), by = c("country", "Year"))

cat("\nMerged dataset dims:", dim(merged_data), "\n")
cat("Countries:", length(unique(merged_data$country)), "\n")
cat("Years:", paste(sort(unique(merged_data$Year)), collapse = ", "), "\n")
cat("\nMissing values per column:\n"); print(colSums(is.na(merged_data)))
cat("\nPreview:\n"); print(head(merged_data, 5))

# STEP 3: Missing Values + Outlier Detection & Handling + Save

cat("Missing values BEFORE imputation:\n"); print(colSums(is.na(merged_data)))

merged_data <- merged_data %>% select(-Migrants_net)
cat("\nDropped Migrants_net\n")

merged_data <- merged_data %>%
  group_by(country) %>%
  mutate(Yearly_Change_Pct = ifelse(is.na(Yearly_Change_Pct),
                                   median(Yearly_Change_Pct, na.rm = TRUE),
                                   Yearly_Change_Pct)) %>%
  ungroup()

merged_data <- merged_data %>%
  group_by(country) %>%

```

```

mutate(
  Urban_Pop_Pct = ifelse(is.na(Urban_Pop_Pct),
    median(Urban_Pop_Pct, na.rm = TRUE), Urban_Pop_Pct),
  Urban_Population = ifelse(is.na(Urban_Population),
    median(Urban_Population, na.rm = TRUE), Urban_Population)
) %>%
ungroup()

merged_data <- merged_data %>%
  group_by(country) %>%
  mutate(GDP_growth = ifelse(is.na(GDP_growth),
    median(GDP_growth, na.rm = TRUE), GDP_growth)) %>%
  ungroup()
global_gdp_median <- median(merged_data$GDP_growth, na.rm = TRUE)
merged_data$GDP_growth[is.na(merged_data$GDP_growth)] <- global_gdp_median

merged_data$CO2_change_pct[is.na(merged_data$CO2_change_pct)] <- 0

cat("\nMissing values AFTER imputation:\n"); print(colSums(is.na(merged_data)))

key_vars <- c("GDP_growth", "CO2_per_capita", "CO2_total_tons",
  "Population", "Urban_Pop_Pct", "Fertility_Rate",
  "Median_Age", "Density", "Energy_Per_Capita_kWh")

cat("\n IQR Outlier Detection \n")
iqr_results <- map_df(key_vars, function(v) {
  x <- merged_data[[v]]
  Q1 <- quantile(x, 0.25, na.rm = TRUE)
  Q3 <- quantile(x, 0.75, na.rm = TRUE)
  IQR_val <- Q3 - Q1
  lower <- Q1 - 1.5 * IQR_val
  upper <- Q3 + 1.5 * IQR_val
  n_out <- sum(x < lower | x > upper, na.rm = TRUE)
  data.frame(Variable=v, Q1=round(Q1,2), Q3=round(Q3,2), IQR=round(IQR_val,2),
    Lower_Fence=round(lower,2), Upper_Fence=round(upper,2),
    Outlier_Count=n_out, Outlier_Pct=round(n_out/length(x)*100,1))
})
print(iqr_results); save_table(iqr_results, "outlier_iqr_results")

cat("\n Z-Score Outlier Detection (threshold = 3) \n")
zscore_results <- map_df(key_vars, function(v) {
  x <- merged_data[[v]]
  z <- abs((x - mean(x, na.rm = TRUE)) / sd(x, na.rm = TRUE))
  n_out <- sum(z > 3, na.rm = TRUE)
  data.frame(Variable=v, Outlier_Count=n_out,
    Outlier_Pct=round(n_out/length(x)*100,1))
})
print(zscore_results); save_table(zscore_results, "outlier_zscore_results")

p01_gdp <- quantile(merged_data$GDP_growth, 0.01, na.rm = TRUE)
p99_gdp <- quantile(merged_data$GDP_growth, 0.99, na.rm = TRUE)
p01_en <- quantile(merged_data$Energy_Per_Capita_kWh, 0.01, na.rm = TRUE)
p99_en <- quantile(merged_data$Energy_Per_Capita_kWh, 0.99, na.rm = TRUE)

merged_data <- merged_data %>%
  mutate(

```

```

    GDP_growth      = pmin(pmax(GDP_growth,      p01_gdp), p99_gdp),
    Energy_Per_Capita_kWh = pmin(pmax(Energy_Per_Capita_kWh, p01_en), p99_en)
  )
cat("GDP_growth Winsorized: [", round(p01_gdp,2), ", ", round(p99_gdp,2), "]\n")
cat("Energy Winsorized: [", round(p01_en,2), ", ", round(p99_en,2), "]\n")

skew_kurt <- merged_data %>%
  select(all_of(key_vars)) %>%
  summarise(across(everything(), list(
    Skewness = ~round(skewness(., na.rm = TRUE), 3),
    Kurtosis = ~round(kurtosis(., na.rm = TRUE), 3)
  )), .names = "{.col}_{.fn}") %>%
  pivot_longer(everything(), names_to = c("Variable", "Stat"), names_sep = "__") %>%
  pivot_wider(names_from = Stat, values_from = value)
print(skew_kurt); save_table(skew_kurt, "skewness_kurtosis")

write.csv(merged_data, "asia_final_clean.csv", row.names = FALSE)
save_table(merged_data, "asia_final_clean")
cat("\nFinal clean dataset saved. Dims:", dim(merged_data), "\n")

# STEP 4: EDA — Full Publication-Quality Visualizations

merged_final <- merged_data %>%
  mutate(
    CO2_group = case_when(
      CO2_per_capita <= quantile(CO2_per_capita, 0.33) ~ "Low Emitter",
      CO2_per_capita <= quantile(CO2_per_capita, 0.67) ~ "Mid Emitter",
      TRUE ~ "High Emitter"),
    CO2_group = factor(CO2_group, levels = c("Low Emitter", "Mid Emitter", "High Emitter")),
    Urban_group = case_when(
      Urban_Pop_Pct < 40 ~ "Rural (<40%)",
      Urban_Pop_Pct < 70 ~ "Mixed (40-70%)",
      TRUE ~ "Urban (>70%)"),
    Urban_group = factor(Urban_group, levels = c("Rural (<40%)", "Mixed (40-70%)", "Urban (>70%)")),
    Energy_group = case_when(
      Energy_Per_Capita_kWh <= quantile(Energy_Per_Capita_kWh, 0.33) ~ "Low Energy",
      Energy_Per_Capita_kWh <= quantile(Energy_Per_Capita_kWh, 0.67) ~ "Mid Energy",
      TRUE ~ "High Energy"),
    Energy_group = factor(Energy_group, levels = c("Low Energy", "Mid Energy", "High Energy")),
    Period = case_when(
      Year < 1990 ~ "Pre-1990",
      Year < 2010 ~ "1990-2009",
      TRUE ~ "2010-2024"),
    Period = factor(Period, levels = c("Pre-1990", "1990-2009", "2010-2024"))
  )

# Descriptive statistics
desc_stats <- merged_final %>%
  select(CO2_per_capita, GDP_growth, Urban_Pop_Pct, Fertility_Rate,
    Median_Age, Density, Energy_Per_Capita_kWh, Population, CO2_total_tons) %>%
  summarise(across(everything(), list(
    Mean = ~round(mean(., na.rm = TRUE), 3),
    Median = ~round(median(., na.rm = TRUE), 3),
    SD = ~round(sd(., na.rm = TRUE), 3),
    Min = ~round(min(., na.rm = TRUE), 3),
    Max = ~round(max(., na.rm = TRUE), 3),

```

```

  Skew = ~round(skewness(., na.rm = TRUE), 3)
), .names = "{.col}_{.fn}") %>%
pivot_longer(everything(), names_to = c("Variable", "Stat"), names_sep = "_") %>%
pivot_wider(names_from = Stat, values_from = value)
save_table(desc_stats, "descriptive_statistics")
print(desc_stats)

# FIG 1: Missing Value Heatmap

pre_imp <- population_clean %>%
  filter(Year %in% common_years) %>%
  inner_join(gdp_clean %>% filter(Year %in% common_years), by = c("country", "Year")) %>%
  inner_join(co2_clean %>% filter(Year %in% common_years), by = c("country", "Year")) %>%
  inner_join(energy_clean %>% filter(Year %in% common_years), by = c("country", "Year"))

missing_pct <- colSums(is.na(pre_imp)) / nrow(pre_imp) * 100
missing_df <- data.frame(Variable = names(missing_pct), Missing_Pct = missing_pct) %>%
  filter(Missing_Pct > 0) %>% arrange(desc(Missing_Pct))

fig1 <- ggplot(missing_df, aes(x = reorder(Variable, -Missing_Pct),
                                y = Missing_Pct, fill = Missing_Pct)) +
  geom_bar(stat = "identity", width = 0.6) +
  geom_text(aes(label = paste0(round(Missing_Pct, 1), "%"),
                    vjust = -0.4, size = 4, fontface = "bold")) +
  scale_fill_gradient(low = "#F39C12", high = "#E74C3C") +
  theme_minimal(base_size = 13) +
  theme(plot.title = element_text(face="bold", hjust=0.5, size=14),
        axis.text.x = element_text(angle=30, hjust=1), legend.position="none") +
  labs(title = "Fig 1: Percentage of Missing Values by Variable (Before Imputation)",
        x = "Variable", y = "Missing (%)")
save_fig(fig1, "Fig01_missing_values", width = 11, height = 6)

# FIG 2: Boxplots — Outlier Detection
# SA FORMAT applied to CO2_total_tons and Population boxes

bp_vars <- list(
  list(v="CO2_per_capita", fill="#3498DB", title="CO2 Per Capita (tons/person)", sa=FALSE),
  list(v="GDP_growth", fill="#2ECC71", title="GDP Growth (%)", sa=FALSE),
  list(v="Fertility_Rate", fill="#9B59B6", title="Fertility Rate (births/woman)", sa=FALSE),
  list(v="Urban_Pop_Pct", fill="#1ABC9C", title="Urban Population (%)", sa=FALSE),
  list(v="Median_Age", fill="#E67E22", title="Median Age (years)", sa=FALSE),
  list(v="Energy_Per_Capita_kWh", fill="#F39C12", title="Energy Per Capita (kWh)", sa=FALSE),
  list(v="Density", fill="#E74C3C", title="Population Density (per km²)", sa=FALSE),
  list(v="CO2_total_tons", fill="#8E44AD", title="Total CO2 Emissions (tons)", sa=TRUE),
  list(v="Population", fill="#2980B9", title="Population", sa=TRUE)
)

bp_list <- lapply(bp_vars, function(x) {
  p <- ggplot(merged_final, aes_string(y = x$v)) +
    geom_boxplot(fill = x$fill, color = "#2C3E50",
                 outlier.color = "#E74C3C", outlier.size = 1.8, alpha = 0.8) +
    theme_minimal(base_size = 10) +
    labs(title = x$title, y = "") +
    theme(plot.title = element_text(face="bold", hjust=0.5, size=9))
  # SA FORMAT: apply label_sa() on y-axis for big-number columns
  if (x$sa) p <- p + scale_y_continuous(labels = label_sa())
})

```

```

    p
  })

fig2 <- grid.arrange(grobs = bp_list, ncol = 4,
  top = grid::textGrob(
    "Fig 2: Boxplots — Outlier Detection Across All Key Variables",
    gp = grid::gpar(fontface="bold",fontsize=13)))
save_fig(NULL, "Fig02_boxplots_outliers", width = 16, height = 8)

# FIG 3: Histograms + Density Curves

hist_vars <- list(
  list(v="CO2_per_capita", fill="#3498DB", xlab="Tons/Person", sa=FALSE),
  list(v="GDP_growth", fill="#2ECC71", xlab="Annual %", sa=FALSE),
  list(v="Fertility_Rate", fill="#9B59B6", xlab="Births/Woman", sa=FALSE),
  list(v="Urban_Pop_Pct", fill="#1ABC9C", xlab="Urban (%)", sa=FALSE),
  list(v="Median_Age", fill="#E67E22", xlab="Years", sa=FALSE),
  list(v="Energy_Per_Capita_kWh", fill="#F39C12", xlab="kWh/Person", sa=FALSE)
)

h_list <- lapply(hist_vars, function(x) {
  p <- ggplot(merged_final, aes_string(x = x$v)) +
    geom_histogram(aes(y = after_stat(density)), bins = 30,
      fill = x$fill, color = "white", alpha = 0.8) +
    geom_density(color = "#E74C3C", linewidth = 1.2) +
    theme_minimal(base_size = 10) +
    labs(title = x$v, x = x$xlabel, y = "Density") +
    theme(plot.title = element_text(face="bold",hjust=0.5,size=9))
  if (x$sa) p <- p + scale_x_continuous(labels = label_sa())
  p
})

fig3 <- grid.arrange(grobs = h_list, ncol = 3,
  top = grid::textGrob(
    "Fig 3: Distribution of Key Variables — Histogram + Density Curve",
    gp = grid::gpar(fontface="bold",fontsize=13)))
save_fig(NULL, "Fig03_histograms_density", width = 14, height = 8)

# FIG 4: Scatter Plots — Features vs CO2 Per Capita

sc_vars <- list(
  list(v="GDP_growth", col="#3498DB", xlab="GDP Growth (%)", sa=FALSE),
  list(v="Urban_Pop_Pct", col="#2ECC71", xlab="Urban Population (%)", sa=FALSE),
  list(v="Fertility_Rate", col="#9B59B6", xlab="Fertility Rate", sa=FALSE),
  list(v="Median_Age", col="#E67E22", xlab="Median Age (years)", sa=FALSE),
  list(v="Energy_Per_Capita_kWh", col="#F39C12", xlab="Energy Per Capita (kWh)", sa=FALSE),
  list(v="Density", col="#E74C3C", xlab="Population Density (per km²)", sa=FALSE)
)

sc_list <- lapply(sc_vars, function(x) {
  p <- ggplot(merged_final, aes_string(x = x$v, y = "CO2_per_capita")) +
    geom_point(alpha = 0.45, color = x$col, size = 1.8) +
    geom_smooth(method = "loess", color = "#2C3E50",
      se = TRUE, linewidth = 1, fill = "#BDC3C7") +
    theme_minimal(base_size = 10) +
    labs(title = paste(x$xlabel, "vs CO2"),

```

```

      x = x$xlabel, y = "CO2 Per Capita (tons)") +
      theme(plot.title = element_text(face="bold",hjust=0.5,size=9))
    if (x$sa) p <- p + scale_x_continuous(labels = label_sa())
  p
})

fig4 <- grid.arrange(grobs = sc_list, ncol = 3,
  top = grid::textGrob(
    "Fig 4: Scatter Plots — Key Features vs CO2 Per Capita (LOESS fit)",
    gp = grid::gpar(fontface="bold",fontsize=13)))
save_fig(NULL, "Fig04_scatter_plots", width = 14, height = 9)

# FIG 5: Violin Plots

v1 <- ggplot(merged_final, aes(x=Urban_group,y=CO2_per_capita,fill=Urban_group)) +
  geom_violin(alpha=0.75,trim=FALSE) +
  geom_boxplot(width=0.12,fill="white",outlier.size=1,color="#2C3E50") +
  scale_fill_manual(values=c("#3498DB","#F39C12","#E74C3C")) +
  theme_minimal(base_size=11) +
  theme(legend.position="none",plot.title=element_text(face="bold",hjust=0.5),
    axis.text.x=element_text(angle=15,hjust=1)) +
  labs(title="CO2 by Urbanization Group",x="Urbanization",y="CO2 Per Capita (tons)")

v2 <- ggplot(merged_final, aes(x=Energy_group,y=CO2_per_capita,fill=Energy_group)) +
  geom_violin(alpha=0.75,trim=FALSE) +
  geom_boxplot(width=0.12,fill="white",outlier.size=1,color="#2C3E50") +
  scale_fill_manual(values=c("#2ECC71","#F39C12","#E74C3C")) +
  theme_minimal(base_size=11) +
  theme(legend.position="none",plot.title=element_text(face="bold",hjust=0.5)) +
  labs(title="CO2 by Energy Group",x="Energy Consumption",y="CO2 Per Capita (tons)")

v3 <- ggplot(merged_final, aes(x=Period,y=CO2_per_capita,fill=Period)) +
  geom_violin(alpha=0.75,trim=FALSE) +
  geom_boxplot(width=0.12,fill="white",outlier.size=1,color="#2C3E50") +
  scale_fill_manual(values=c("#9B59B6","#2ECC71","#E74C3C")) +
  theme_minimal(base_size=11) +
  theme(legend.position="none",plot.title=element_text(face="bold",hjust=0.5)) +
  labs(title="CO2 by Time Period",x="Period",y="CO2 Per Capita (tons)")

v4 <- ggplot(merged_final, aes(x=Urban_group,y=GDP_growth,fill=Urban_group)) +
  geom_violin(alpha=0.75,trim=FALSE) +
  geom_boxplot(width=0.12,fill="white",outlier.size=1,color="#2C3E50") +
  scale_fill_manual(values=c("#1ABC9C","#E67E22","#3498DB")) +
  theme_minimal(base_size=11) +
  theme(legend.position="none",plot.title=element_text(face="bold",hjust=0.5),
    axis.text.x=element_text(angle=15,hjust=1)) +
  labs(title="GDP Growth by Urbanization",x="Urbanization",y="GDP Growth (%)")

fig5 <- grid.arrange(v1,v2,v3,v4,ncol=2,
  top=grid::textGrob(
    "Fig 5: Violin Plots — CO2 and GDP Distributions Across Categories",
    gp=grid::gpar(fontface="bold",fontsize=13)))
save_fig(NULL, "Fig05_violin_plots", width=14, height=10)

# FIG 6: Time-Series Line Plots

```

```

top8 <- merged_final %>%
  group_by(country) %>%
  summarise(avg_co2 = mean(CO2_per_capita, na.rm=TRUE)) %>%
  top_n(8, avg_co2) %>% pull(country)

fig6a <- merged_final %>%
  filter(country %in% top8) %>%
  ggplot(aes(x=Year,y=CO2_per_capita,color=country,group=country)) +
  geom_line(linewidth=1.1) + geom_point(size=2) +
  scale_color_brewer(palette="Set2") +
  theme_minimal(base_size=12) +
  theme(plot.title=element_text(face="bold",hjust=0.5),legend.position="right") +
  labs(title="Fig 6a: CO2 Per Capita Trend — Top 8 Asian Emitters",
       x="Year",y="CO2 Per Capita (tons)",color="Country")
save_fig(fig6a, "Fig06a_co2_trend_top8", width=12, height=6)

fig6b <- merged_final %>%
  group_by(Year) %>%
  summarise(avg_CO2 = mean(CO2_per_capita, na.rm=TRUE),
            avg_Energy = mean(Energy_Per_Capita_kWh, na.rm=TRUE) / 10000,
            avg_Urban = mean(Urban_Pop_Pct, na.rm=TRUE)) %>%
  pivot_longer(-Year, names_to="Indicator", values_to="Value") %>%
  ggplot(aes(x=Year,y=Value,color=Indicator,group=Indicator)) +
  geom_line(linewidth=1.2) + geom_point(size=2) +
  scale_color_manual(
    values=c("avg_CO2"="#E74C3C","avg_Energy"="#F39C12","avg_Urban"="#3498DB"),
    labels=c("Avg CO2 (tons)","Avg Energy (×10k kWh)","Avg Urban (%)")) +
  theme_minimal(base_size=12) +
  theme(plot.title=element_text(face="bold",hjust=0.5)) +
  labs(title="Fig 6b: Asian Average CO2, Energy & Urbanization Trends Over Time",
       x="Year",y="Value",color="Indicator")
save_fig(fig6b, "Fig06b_multi_trend", width=12, height=6)

fig6c <- merged_final %>%
  group_by(Year) %>%
  summarise(avg_GDP = mean(GDP_growth, na.rm=TRUE)) %>%
  ggplot(aes(x=Year,y=avg_GDP)) +
  geom_line(color="#2ECC71",linewidth=1.3) +
  geom_point(color="#27AE60",size=2.5) +
  geom_hline(yintercept=0,linetype="dashed",color="#E74C3C",linewidth=0.8) +
  theme_minimal(base_size=12) +
  theme(plot.title=element_text(face="bold",hjust=0.5)) +
  labs(title="Fig 6c: Average GDP Growth Across Asia Over Time",
       x="Year",y="Average GDP Growth (%)")
save_fig(fig6c, "Fig06c_gdp_trend", width=10, height=6)

# FIG 7: Pearson & Spearman Correlation Heatmaps

numeric_cols_df <- merged_final %>%
  select(CO2_per_capita, GDP_growth, Urban_Pop_Pct, Fertility_Rate,
         Median_Age, Density, Energy_Per_Capita_kWh, Population, CO2_total_tons)

cor_pearson <- cor(numeric_cols_df, method="pearson", use="complete.obs")
cor_spearman <- cor(numeric_cols_df, method="spearman", use="complete.obs")
save_table(as.data.frame(round(cor_pearson, 3)), "pearson_correlation_matrix")
save_table(as.data.frame(round(cor_spearman, 3)), "spearman_correlation_matrix")

```

```
fig7a <- ggcorrplot(cor_pearson, method="square", type="full", lab=TRUE, lab_size=3.2,
  colors=c("#E74C3C","white","#3498DB"), outline.color="white",
  ggtheme=theme_minimal(base_size=11),
  title="Fig 7a: Pearson Correlation Heatmap") +
  theme(plot.title=element_text(face="bold",hjust=0.5,size=13),
    axis.text.x=element_text(angle=45,hjust=1))
save_fig(fig7a, "Fig07a_pearson_heatmap", width=10, height=9)
```

```
fig7b <- ggcorrplot(cor_spearman, method="square", type="full", lab=TRUE, lab_size=3.2,
  colors=c("#9B59B6","white","#F39C12"), outline.color="white",
  ggtheme=theme_minimal(base_size=11),
  title="Fig 7b: Spearman Correlation Heatmap") +
  theme(plot.title=element_text(face="bold",hjust=0.5,size=13),
    axis.text.x=element_text(angle=45,hjust=1))
save_fig(fig7b, "Fig07b_spearman_heatmap", width=10, height=9)
```

FIG 8: Ridge Plot

```
fig8 <- ggplot(merged_final, aes(x=CO2_per_capita,y=Period,fill=Period)) +
  geom_density_ridges(alpha=0.7,scale=1.5,quantile_lines=TRUE,quantiles=2) +
  scale_fill_manual(values=c("#3498DB","#E67E22","#E74C3C")) +
  theme_minimal(base_size=12) +
  theme(legend.position="none",plot.title=element_text(face="bold",hjust=0.5)) +
  labs(title="Fig 8: Ridge Plot — CO2 Per Capita Distribution by Time Period",
    x="CO2 Per Capita (tons)",y="Time Period")
save_fig(fig8, "Fig08_ridge_plot", width=10, height=6)
```

FIG 9: Pairs Plot

```
pairs_data <- merged_final %>%
  select(CO2_per_capita, GDP_growth, Urban_Pop_Pct,
    Fertility_Rate, Energy_Per_Capita_kWh, Median_Age) %>%
  sample_n(min(300, nrow(merged_final)))

fig9 <- ggpairs(pairs_data,
  upper = list(continuous=wrap("cor",size=3.5,color="#2C3E50")),
  lower = list(continuous=wrap("points",alpha=0.4,size=1.2,color="#3498DB")),
  diag = list(continuous=wrap("densityDiag",fill="#3498DB",alpha=0.6)),
  title = "Fig 9: Pairs Plot — Multivariate Relationships") +
  theme_minimal(base_size=10) +
  theme(plot.title=element_text(face="bold",hjust=0.5))
save_fig(fig9, "Fig09_pairs_plot", width=12, height=10)
```

FIG 10: ANOVA

```
anova_urban <- aov(CO2_per_capita ~ Urban_group, data=merged_final)
anova_period <- aov(CO2_per_capita ~ Period, data=merged_final)
cat("\n--- ANOVA: Urban Group ---\n"); print(summary(anova_urban))
cat("\n--- ANOVA: Time Period ---\n"); print(summary(anova_period))

anova_urban_sum <- summary(anova_urban)[[1]]
anova_period_sum <- summary(anova_period)[[1]]

anova_table <- data.frame(
```



```

Test      = c("Urban Group vs CO2", "Time Period vs CO2"),
F_value   = round(c(anova_urban_sum$`F value`[1], anova_period_sum$`F value`[1]),3),
p_value   = round(c(anova_urban_sum$`Pr(>F)`[1], anova_period_sum$`Pr(>F)`[1]),5),
Significant = c(
  ifelse(anova_urban_sum$`Pr(>F)`[1] < 0.05, "Yes ***", "No"),
  ifelse(anova_period_sum$`Pr(>F)`[1] < 0.05, "Yes ***", "No"))
)
print(anova_table); save_table(anova_table, "anova_results")

urban_summary <- merged_final %>%
  group_by(Urban_group) %>%
  summarise(mean_CO2=mean(CO2_per_capita), se=sd(CO2_per_capita)/sqrt(n()), n=n())

fig10a <- ggplot(urban_summary, aes(x=Urban_group, y=mean_CO2, fill=Urban_group)) +
  geom_bar(stat="identity", width=0.5, alpha=0.85) +
  geom_errorbar(aes(ymin=mean_CO2-se, ymax=mean_CO2+se), width=0.2, linewidth=0.8) +
  geom_text(aes(label=paste0("n=", n, "\n", round(mean_CO2, 1))),
    vjust=-0.5, size=3.8, fontface="bold") +
  scale_fill_manual(values=c("#3498DB", "#F39C12", "#E74C3C")) +
  theme_minimal(base_size=12) +
  theme(legend.position="none", plot.title=element_text(face="bold", hjust=0.5)) +
  labs(title="Fig 10a: Mean CO2 Per Capita by Urbanization (One-Way ANOVA)",
    x="Urbanization Group", y="Mean CO2 Per Capita (tons)") +
  ylim(0, max(urban_summary$mean_CO2)*1.35)
save_fig(fig10a, "Fig10a_anova_urban", width=9, height=6)

period_summary <- merged_final %>%
  group_by(Period) %>%
  summarise(mean_CO2=mean(CO2_per_capita), se=sd(CO2_per_capita)/sqrt(n()), n=n())

fig10b <- ggplot(period_summary, aes(x=Period, y=mean_CO2, fill=Period)) +
  geom_bar(stat="identity", width=0.5, alpha=0.85) +
  geom_errorbar(aes(ymin=mean_CO2-se, ymax=mean_CO2+se), width=0.2, linewidth=0.8) +
  geom_text(aes(label=paste0("n=", n, "\n", round(mean_CO2, 1))),
    vjust=-0.5, size=3.8, fontface="bold") +
  scale_fill_manual(values=c("#9B59B6", "#2ECC71", "#E74C3C")) +
  theme_minimal(base_size=12) +
  theme(legend.position="none", plot.title=element_text(face="bold", hjust=0.5)) +
  labs(title="Fig 10b: Mean CO2 Per Capita by Time Period (One-Way ANOVA)",
    x="Time Period", y="Mean CO2 Per Capita (tons)") +
  ylim(0, max(period_summary$mean_CO2)*1.35)
save_fig(fig10b, "Fig10b_anova_period", width=9, height=6)

```

FIG 11: Chi-Square Tests

```

chi_urban <- chisq.test(table(merged_final$CO2_group, merged_final$Urban_group))
chi_period <- chisq.test(table(merged_final$CO2_group, merged_final$Period))
chi_energy <- chisq.test(table(merged_final$CO2_group, merged_final$Energy_group))

chi_summary <- data.frame(
  Test      = c("CO2 Group vs Urban Group", "CO2 Group vs Time Period", "CO2 Group vs Energy Group"),
  Chi_Square = round(c(chi_urban$statistic, chi_period$statistic, chi_energy$statistic), 3),
  df        = c(chi_urban$parameter, chi_period$parameter, chi_energy$parameter),
  p_value   = round(c(chi_urban$p.value, chi_period$p.value, chi_energy$p.value), 5),
  Significant = ifelse(

```

```

    c(chi_urban$p.value, chi_period$p.value, chi_energy$p.value) < 0.05, "Yes ***", "No")
  )
print(chi_summary); save_table(chi_summary, "chi_square_results")

fig11 <- chi_summary %>%
  ggplot(aes(x=reorder(Test,-Chi_Square), y=Chi_Square, fill=Significant)) +
  geom_bar(stat="identity", width=0.5, alpha=0.85) +
  geom_text(aes(label=paste0("χ²=", round(Chi_Square, 1), "\np=", round(p_value, 4))),
    vjust=-0.4, size=4, fontface="bold") +
  scale_fill_manual(values=c("Yes ***"="#E74C3C", "No"="#95A5A6"), name="Significant") +
  theme_minimal(base_size=13) +
  theme(plot.title=element_text(face="bold", hjust=0.5, size=14),
    axis.text.x=element_text(angle=15, hjust=1), legend.position="top") +
  labs(title="Fig 11: Chi-Square Test — CO2 Emission Group vs Categorical Variables",
    x="Test", y="Chi-Square Statistic") +
  ylim(0, max(chi_summary$Chi_Square)*1.2)
save_fig(fig11, "Fig11_chi_square_results", width=10, height=7)

# FIG 12: Pearson Feature Correlation Bar

cor_feature_df <- data.frame(
  Feature = names(numeric_cols_df)[names(numeric_cols_df) != "CO2_per_capita"],
  Pearson = cor_pearson["CO2_per_capita",
    names(numeric_cols_df)[names(numeric_cols_df) != "CO2_per_capita"]]
) %>% mutate(Abs_Pearson = abs(Pearson))

fig12 <- ggplot(cor_feature_df,
  aes(x=reorder(Feature, Abs_Pearson), y=Pearson, fill=Pearson>0)) +
  geom_bar(stat="identity", alpha=0.85) +
  coord_flip() +
  scale_fill_manual(values=c("TRUE"="#3498DB", "FALSE"="#E74C3C"),
    labels=c("Negative", "Positive"), name="Direction") +
  geom_hline(yintercept=c(-0.3, 0.3), linetype="dashed", color="#2C3E50", alpha=0.5) +
  geom_text(aes(label=round(Pearson, 3), hjust=ifelse(Pearson>0, -0.1, 1.1)),
    size=3.8, fontface="bold") +
  theme_minimal(base_size=12) +
  theme(plot.title=element_text(face="bold", hjust=0.5), legend.position="right") +
  ylim(-0.8, 1.0) +
  labs(title="Fig 12: Pearson Correlation of Features with CO2 Per Capita",
    x="Feature", y="Pearson Correlation Coefficient")
save_fig(fig12, "Fig12_feature_correlation_bar", width=10, height=7)

cat("\n=== STEP 4 COMPLETE — All EDA figures saved ===\n")

# STEP 5: Feature Engineering + Target + Scaling + Split + SMOTE + CV

merged_final <- merged_final %>%
  mutate(
    log_Population = log1p(Population),
    log_Density = log1p(Density),
    log_Energy = log1p(Energy_Per_Capita_kWh),
    Energy_x_Urban = log1p(Energy_Per_Capita_kWh) * Urban_Pop_Pct,
    GDP_x_Urban = GDP_growth * Urban_Pop_Pct,
    Urban_x_Fert = Urban_Pop_Pct * Fertility_Rate
  )

```

```

t50 <- quantile(merged_final$CO2_per_capita, 0.50)
t75 <- quantile(merged_final$CO2_per_capita, 0.75)
t90 <- quantile(merged_final$CO2_per_capita, 0.90)

thresh_df <- data.frame(
  Threshold = c("50th pct", "75th pct (USED)", "90th pct"),
  Cutoff = round(c(t50, t75, t90), 3),
  Pct_High = round(c(mean(merged_final$CO2_per_capita > t50) * 100,
    mean(merged_final$CO2_per_capita > t75) * 100,
    mean(merged_final$CO2_per_capita > t90) * 100), 1)
)
print(thresh_df); save_table(thresh_df, "threshold_sensitivity")

merged_final <- merged_final %>%
  mutate(
    High_Emitter = factor(ifelse(CO2_per_capita > t75, "High", "Low"),
      levels=c("Low", "High")),
    log_CO2_target = log1p(CO2_per_capita)
  )

cat("\nClass distribution:\n"); print(table(merged_final$High_Emitter))

class_df <- as.data.frame(table(merged_final$High_Emitter)) %>%
  rename(Class=Var1, Count=Freq) %>%
  mutate(Pct=round(Count/sum(Count)*100, 1))

fig13 <- ggplot(class_df, aes(x=Class, y=Count, fill=Class)) +
  geom_bar(stat="identity", width=0.45, alpha=0.85) +
  geom_text(aes(label=paste0(Count, "\n(", Pct, "%)")), vjust=-0.4, size=5, fontface="bold") +
  scale_fill_manual(values=c("Low"="#2ECC71", "High"="#E74C3C")) +
  theme_minimal(base_size=13) +
  theme(legend.position="none", plot.title=element_text(face="bold", hjust=0.5, size=14)) +
  labs(title="Fig 13: Class Distribution — High vs Low CO2 Emitter (75th Percentile)",
    x="Emission Category", y="Count") +
  ylim(0, max(class_df$Count)*1.25)
save_fig(fig13, "Fig13_class_distribution", width=8, height=6)

selected_features <- c(
  "Energy_Per_Capita_kWh", "Urban_Pop_Pct", "Yearly_Change_Pct", "Median_Age",
  "GDP_growth", "Fertility_Rate", "Density", "log_Population", "log_Energy",
  "log_Density", "Energy_x_Urban", "GDP_x_Urban", "Urban_x_Fert"
)

model_data <- merged_final %>%
  select(all_of(selected_features), High_Emitter, log_CO2_target, CO2_per_capita)

X_all <- model.matrix(
  High_Emitter ~ Energy_Per_Capita_kWh + Urban_Pop_Pct + Yearly_Change_Pct +
    Median_Age + GDP_growth + Fertility_Rate + Density + log_Population +
    log_Energy + log_Density + Energy_x_Urban + GDP_x_Urban + Urban_x_Fert - 1,
  data = model_data
) %>% as.data.frame()
names(X_all) <- make.names(names(X_all), unique=TRUE)
y_class <- model_data$High_Emitter
y_reg <- model_data$log_CO2_target

```

```

set.seed(123)
train_idx <- createDataPartition(y_class, p=0.70, list=FALSE)
remaining <- setdiff(seq_len(nrow(X_all)), train_idx)
val_idx <- sample(remaining, floor(0.15 * nrow(X_all)))
test_idx <- setdiff(remaining, val_idx)

X_train_raw <- X_all[train_idx,]; X_val_raw <- X_all[val_idx,]; X_test_raw <- X_all[test_idx,]
y_train <- y_class[train_idx]; y_val <- y_class[val_idx]; y_test <- y_class[test_idx]
y_train_reg <- y_reg[train_idx]; y_val_reg <- y_reg[val_idx]; y_test_reg <- y_reg[test_idx]

cat("\nSplit — Train:",nrow(X_train_raw),"| Val:",nrow(X_val_raw),"| Test:",nrow(X_test_raw),"\\n")

# FIG 14: Split visualization
# SA FORMAT applied to row count labels

split_df <- data.frame(
  Split = c("Train (70%)", "Validation (15%)", "Test (15%)",
    Count = c(length(train_idx), length(val_idx), length(test_idx))
)
fig14 <- ggplot(split_df, aes(x=reorder(Split,-Count),y=Count,fill=Split)) +
  geom_bar(stat="identity",width=0.45,alpha=0.85) +
  # SA FORMAT: format_sa() used in label to show row counts cleanly
  geom_text(aes(label=paste0(format_sa(Count)," rows\\n(",
    round(Count/nrow(X_all)*100,1),"%"))),
    vjust=-0.4,size=4.5,fontface="bold") +
  scale_fill_manual(values=c("Train (70%)">#3498DB",
    "Validation (15%)">#F39C12","Test (15%)">#E74C3C")) +
  theme_minimal(base_size=13) +
  theme(legend.position="none",plot.title=element_text(face="bold",hjust=0.5,size=14)) +
  labs(title="Fig 14: Stratified Train / Validation / Test Split (70/15/15)",
    x="Dataset Split",y="Number of Observations") +
  ylim(0,max(split_df$Count)*1.25)
save_fig(fig14, "Fig14_split", width=9, height=6)

train_means <- colMeans(X_train_raw, na.rm=TRUE)
train_sds <- apply(X_train_raw, 2, sd, na.rm=TRUE)
train_sds[train_sds == 0] <- 1

scale_fn <- function(df, means, sds)
  as.data.frame(sweep(sweep(df, 2, means, "-"), 2, sds, "/"))

X_train <- scale_fn(X_train_raw, train_means, train_sds)
X_val <- scale_fn(X_val_raw, train_means, train_sds)
X_test <- scale_fn(X_test_raw, train_means, train_sds)

save_table(data.frame(Feature=names(train_means),
  Mean=round(train_means,4), SD=round(train_sds,4)),
  "scaling_parameters")

fig15 <- bind_rows(
  X_train_raw %>% select(1:6) %>%
    pivot_longer(everything(),names_to="Feature",values_to="Value") %>%
    mutate(Type="Before Scaling"),
  X_train %>% select(1:6) %>%
    pivot_longer(everything(),names_to="Feature",values_to="Value") %>%

```

```

mutate(Type="After Scaling")
) %>%
ggplot(aes(x=Value,fill=Type)) +
geom_density(alpha=0.5) +
facet_wrap(~Feature,scales="free",ncol=3) +
scale_fill_manual(values=c("Before Scaling"="#E74C3C","After Scaling"="#3498DB")) +
theme_minimal(base_size=10) +
theme(plot.title=element_text(face="bold",hjust=0.5,size=13),
strip.text=element_text(face="bold"), legend.position="top") +
labs(title="Fig 15: Feature Distributions Before vs After Z-Score Standardization",
x="Value",y="Density",fill="")
save_fig(fig15, "Fig15_scaling", width=13, height=8)

cat("\n SMOTE \n")
cat("Before SMOTE:\n"); print(table(y_train))
smote_input <- data.frame(X_train, High_Emitter=y_train)
smote_res <- SMOTE(X=smote_input[,!names(smote_input)%in%"High_Emitter"],
target=smote_input$High_Emitter, K=5, dup_size=0)
X_train_sm <- smote_res$data[,!names(smote_res$data)%in%"class"]
y_train_sm <- factor(smote_res$data$class, levels=c("Low","High"))
cat("After SMOTE:\n"); print(table(y_train_sm))

fig16 <- bind_rows(
data.frame(Class=y_train, Type="Before SMOTE"),
data.frame(Class=y_train_sm, Type="After SMOTE")
) %>%
ggplot(aes(x=Class,fill=Class)) +
geom_bar(width=0.45,alpha=0.85) +
geom_text(stat="count",aes(label=after_stat(count)),vjust=-0.4,size=5,fontface="bold") +
facet_wrap(~Type) +
scale_fill_manual(values=c("Low"="#2ECC71","High"="#E74C3C")) +
theme_minimal(base_size=13) +
theme(legend.position="none",plot.title=element_text(face="bold",hjust=0.5,size=14),
strip.text=element_text(face="bold",size=13)) +
labs(title="Fig 16: Class Distribution Before vs After SMOTE (Training Set Only)",
x="Emission Category",y="Count")
save_fig(fig16, "Fig16_smote", width=10, height=6)

train_cv_df <- data.frame(X_train_sm, High_Emitter=y_train_sm)
names(train_cv_df) <- make.names(names(train_cv_df), unique=TRUE)

cv_comparison <- map_df(c(3,5,10), function(k) {
ctrl <- trainControl(method="cv",number=k,classProbs=TRUE,
summaryFunction=twoClassSummary,savePredictions="final")
set.seed(123)
m <- train(High_Emitter~.,data=train_cv_df,method="glm",family="binomial",
trControl=ctrl,metric="ROC")
data.frame(CV_Method=paste0(k,"-Fold CV"),
AUC_ROC=round(max(m$results$ROC),4),
Sensitivity=round(max(m$results$Sens),4),
Specificity=round(max(m$results$Spec),4))
})
print(cv_comparison); save_table(cv_comparison, "cv_comparison_3_5_10_fold")

fig17 <- cv_comparison %>%
pivot_longer(-CV_Method,names_to="Metric",values_to="Value") %>%

```

```

ggplot(aes(x=CV_Method,y=Value,fill=CV_Method)) +
  geom_bar(stat="identity",width=0.5,alpha=0.85) +
  geom_text(aes(label=round(Value,3)),vjust=-0.4,size=4,fontface="bold") +
  facet_wrap(~Metric,scales="free_y") +
  scale_fill_manual(values=c("3-Fold CV"="#9B59B6","5-Fold CV"="#3498DB","10-Fold CV"="#E74C3C")) +
  theme_minimal(base_size=12) +
  theme(legend.position="none",plot.title=element_text(face="bold",hjust=0.5,size=13),
        strip.text=element_text(face="bold"),axis.text.x=element_text(angle=15,hjust=1)) +
  labs(title="Fig 17: Cross-Validation Comparison — 3-Fold vs 5-Fold vs 10-Fold",
        x="CV Method",y="Score") + ylim(0,1.15)
save_fig(fig17, "Fig17_cv_comparison", width=12, height=6)

ctrl_5fold <- trainControl(method="cv",number=5,classProbs=TRUE,
                           summaryFunction=twoClassSummary,savePredictions="final")
train_df <- data.frame(X_train_sm, High_Emitter=y_train_sm)
names(train_df) <- make.names(names(train_df), unique=TRUE)

# STEP 6: Classification Models

eval_caret <- function(model, X, y, model_name) {
  pred_class <- predict(model, X)
  pred_prob <- predict(model, X, type="prob")[, "High"]
  cm <- confusionMatrix(pred_class, y, positive="High")
  roc <- roc(as.numeric(y=="High"), pred_prob, quiet=TRUE)
  ci <- binom.test(sum(pred_class==y), length(y))$conf.int
  data.frame(Model=model_name,
             Accuracy=round(cm$overall["Accuracy"],4),
             Acc_CI_Low=round(ci[1],4), Acc_CI_High=round(ci[2],4),
             Precision=round(cm$byClass["Precision"],4),
             Recall=round(cm$byClass["Recall"],4),
             F1_Score=round(cm$byClass["F1"],4),
             AUC_ROC=round(auc(roc),4),
             Specificity=round(cm$byClass["Specificity"],4),
             Balanced_Acc=round(cm$byClass["Balanced Accuracy"],4))
}

eval_xgb <- function(cls, prob, y, name) {
  cm <- confusionMatrix(cls, y, positive="High")
  roc <- roc(as.numeric(y=="High"), prob, quiet=TRUE)
  ci <- binom.test(sum(cls==y), length(y))$conf.int
  data.frame(Model=name,
             Accuracy=round(cm$overall["Accuracy"],4),
             Acc_CI_Low=round(ci[1],4), Acc_CI_High=round(ci[2],4),
             Precision=round(cm$byClass["Precision"],4),
             Recall=round(cm$byClass["Recall"],4),
             F1_Score=round(cm$byClass["F1"],4),
             AUC_ROC=round(auc(roc),4),
             Specificity=round(cm$byClass["Specificity"],4),
             Balanced_Acc=round(cm$byClass["Balanced Accuracy"],4))
}

cat("\n Model 1: Logistic Regression \n")
set.seed(123)
model_lr <- train(High_Emitter~.,data=train_df,method="glm",family="binomial",
                  trControl=ctrl_5fold,metric="ROC")
cat("LR CV AUC:", round(max(model_lr$results$ROC),4),"n")

```

```

cat("\n Model 2: Decision Tree \n")
set.seed(123)
model_dt <- train(High_Emitter~.,data=train_df,method="rpart",
                  trControl=ctrl_5fold,metric="ROC",tuneLength=10)
cat("DT CV AUC:", round(max(model_dt$results$ROC),4),"\n")

png(file.path(output_folder,"figures","Fig18_decision_tree.png"),
    width=1400,height=900,res=120)
rpart.plot(model_dt$finalModel,type=4,extra=101,
            main="Fig 18: Decision Tree — CO2 High Emitter Classification",
            cex=0.75,fallen.leaves=TRUE)
dev.off()

cat("\n Model 3: Random Forest \n")
set.seed(123)
model_rf <- train(High_Emitter~.,data=train_df,method="rf",
                  trControl=ctrl_5fold,metric="ROC",
                  tuneGrid=expand.grid(mtry=c(2,3,4,5,6)),ntree=500)
cat("RF best mtry:",model_rf$bestTune$mtry,"| CV AUC:",round(max(model_rf$results$ROC),4),"\n")

cat("\n Model 4: SVM (RBF Kernel) \n")
set.seed(123)
model_svm <- train(High_Emitter~.,data=train_df,method="svmRadial",
                  trControl=ctrl_5fold,metric="ROC",
                  tuneGrid=expand.grid(C=c(0.1,0.5,1,2,5),sigma=c(0.01,0.05,0.1,0.5)))
cat("SVM best C:",model_svm$bestTune$C,"| sigma:",model_svm$bestTune$sigma,"\n")

cat("\n Model 5: XGBoost \n")
X_tr_mat <- as.matrix(X_train_sm); X_va_mat <- as.matrix(X_val); X_te_mat <- as.matrix(X_test)
y_tr_xgb <- as.numeric(y_train_sm=="High")
y_va_xgb <- as.numeric(y_val=="High")
y_te_xgb <- as.numeric(y_test=="High")

dtrain <- xgb.DMatrix(data=X_tr_mat,label=y_tr_xgb)
dval <- xgb.DMatrix(data=X_va_mat,label=y_va_xgb)
dtest <- xgb.DMatrix(data=X_te_mat,label=y_te_xgb)

xgb_params <- list(objective="binary:logistic",eval_metric="auc",max_depth=4,
                  eta=0.05,subsample=0.8,colsample_bytree=0.8,
                  min_child_weight=1,gamma=0.1)
set.seed(123)
model_xgb <- xgb.train(params=xgb_params,data=dtrain,nrounds=300,
                      evals=list(train=dtrain,val=dval),verbose=0,
                      early_stopping_rounds=20)
cat("XGBoost best iteration:",model_xgb$best_iteration,"\n")

xgb_val_prob <- predict(model_xgb,dval)
xgb_test_prob <- predict(model_xgb,dtest)
xgb_val_cls <- factor(ifelse(xgb_val_prob>0.5,"High","Low"),levels=c("Low","High"))
xgb_test_cls <- factor(ifelse(xgb_test_prob>0.5,"High","Low"),levels=c("Low","High"))

cat("\n Validation Results \n")
val_results <- bind_rows(
  eval_caret(model_lr, X_val,y_val,"Logistic Regression"),
  eval_caret(model_dt, X_val,y_val,"Decision Tree"),

```

```

eval_caret(model_rf, X_val, y_val, "Random Forest"),
eval_caret(model_svm, X_val, y_val, "SVM (RBF)",
eval_xgb(xgb_val_cls, xgb_val_prob, y_val, "XGBoost")
)
print(val_results); save_table(val_results, "validation_results_all_models")

cat("\n Test Results \n")
test_results <- bind_rows(
  eval_caret(model_lr, X_test, y_test, "Logistic Regression"),
  eval_caret(model_dt, X_test, y_test, "Decision Tree"),
  eval_caret(model_rf, X_test, y_test, "Random Forest"),
  eval_caret(model_svm, X_test, y_test, "SVM (RBF)",
  eval_xgb(xgb_test_cls, xgb_test_prob, y_test, "XGBoost")
)
print(test_results); save_table(test_results, "test_results_all_models")

plot_cm <- function(model_name, pred_cls, actual, colors) {
  cm_df <- as.data.frame(table(Predicted=pred_cls, Actual=actual))
  ggplot(cm_df, aes(x=Predicted, y=Actual, fill=Freq)) +
    geom_tile(color="white", linewidth=1.5) +
    geom_text(aes(label=Freq), fontface="bold", size=8, color="white") +
    scale_fill_gradient(low=colors[1], high=colors[2]) +
    theme_minimal(base_size=12) +
    theme(plot.title=element_text(face="bold", hjust=0.5), legend.position="none") +
    labs(title=model_name, x="Predicted", y="Actual")
}

cm1 <- plot_cm("Logistic Regression", predict(model_lr, X_val), y_val, c("#AED6F1", "#1A5276"))
cm2 <- plot_cm("Decision Tree", predict(model_dt, X_val), y_val, c("#D7BDE2", "#6C3483"))
cm3 <- plot_cm("Random Forest", predict(model_rf, X_val), y_val, c("#A9DFBF", "#1E8449"))
cm4 <- plot_cm("SVM (RBF)", predict(model_svm, X_val), y_val, c("#F5CBA7", "#A04000"))
cm5 <- plot_cm("XGBoost", xgb_val_cls, y_val, c("#F9E79F", "#B7950B"))

fig19 <- grid.arrange(cm1, cm2, cm3, cm4, cm5, ncol=3,
  top=grid::textGrob(
    "Fig 19: Confusion Matrices — All Models (Validation Set)",
    gp=grid::gpar(fontface="bold", fontsize=14)))
save_fig(NULL, "Fig19_confusion_matrices", width=14, height=9)

roc_lr <- roc(as.numeric(y_val=="High"), predict(model_lr, X_val, type="prob")[, "High"], quiet=TRUE)
roc_dt <- roc(as.numeric(y_val=="High"), predict(model_dt, X_val, type="prob")[, "High"], quiet=TRUE)
roc_rf <- roc(as.numeric(y_val=="High"), predict(model_rf, X_val, type="prob")[, "High"], quiet=TRUE)
roc_svm <- roc(as.numeric(y_val=="High"), predict(model_svm, X_val, type="prob")[, "High"], quiet=TRUE)
roc_xgb <- roc(as.numeric(y_val=="High"), xgb_val_prob, quiet=TRUE)

fig20 <- ggroc(list("Logistic Regression"=roc_lr, "Decision Tree"=roc_dt,
  "Random Forest"=roc_rf, "SVM (RBF)"=roc_svm, "XGBoost"=roc_xgb),
  linewidth=1.3) +
  geom_abline(slope=1, intercept=1, linetype="dashed", color="gray60") +
  scale_color_manual(values=c("Logistic Regression"="#3498DB", "Decision Tree"="#8E44AD",
    "Random Forest"="#2ECC71", "SVM (RBF)"="#F39C12", "XGBoost"="#E74C3C"),
    name="Model") +
  annotate("text", x=0.38, y=0.22, size=3.8, hjust=0, family="mono",
    label=paste0("LR AUC = ", round(auc(roc_lr), 3), "\n",
      "DT AUC = ", round(auc(roc_dt), 3), "\n",
      "RF AUC = ", round(auc(roc_rf), 3), "\n",

```



```

        "SVM AUC = ",round(auc(roc_svm),3),"\\n",
        "XGB AUC = ",round(auc(roc_xgb),3))) +
theme_minimal(base_size=13) +
theme(plot.title=element_text(face="bold",hjust=0.5,size=14)) +
labs(title="Fig 20: ROC Curves — All 5 Models (Validation Set)",
      x="Specificity",y="Sensitivity")
save_fig(fig20,"Fig20_roc_curves",width=10,height=7)

fig21 <- val_results %>%
  select(Model,Accuracy,Precision,Recall,F1_Score,AUC_ROC,Specificity,Balanced_Acc) %>%
  pivot_longer(-Model,names_to="Metric",values_to="Value") %>%
  ggplot(aes(x=Model,y=Value,fill=Model)) +
  geom_bar(stat="identity",width=0.6,alpha=0.85) +
  geom_text(aes(label=round(Value,3)),vjust=-0.4,size=3,fontface="bold") +
  facet_wrap(~Metric,ncol=4) +
  scale_fill_manual(values=c("Logistic Regression"="#3498DB","Decision Tree"="#8E44AD",
    "Random Forest"="#2ECC71","SVM (RBF)"="#F39C12","XGBoost"="#E74C3C")) +
  theme_minimal(base_size=9) +
  theme(legend.position="none",plot.title=element_text(face="bold",hjust=0.5,size=13),
        axis.text.x=element_text(angle=30,hjust=1),strip.text=element_text(face="bold")) +
  labs(title="Fig 21: Full Classification Metrics — All Models (Validation Set)",
        x="Model",y="Score") + ylim(0,1.2)
save_fig(fig21,"Fig21_model_comparison",width=16,height=10)

overfit_df <- bind_rows(
  val_results %>% select(Model,AUC_ROC) %>% mutate(Set="Validation"),
  test_results %>% select(Model,AUC_ROC) %>% mutate(Set="Test")
)
fig22 <- ggplot(overfit_df,aes(x=Model,y=AUC_ROC,fill=Set)) +
  geom_bar(stat="identity",position="dodge",width=0.6,alpha=0.85) +
  geom_text(aes(label=round(AUC_ROC,3)),position=position_dodge(0.6),
    vjust=-0.4,size=3.8,fontface="bold") +
  scale_fill_manual(values=c("Validation"="#3498DB","Test"="#E74C3C"),name="Dataset") +
  theme_minimal(base_size=12) +
  theme(plot.title=element_text(face="bold",hjust=0.5,size=13),
        axis.text.x=element_text(angle=20,hjust=1),legend.position="top") +
  labs(title="Fig 22: Validation vs Test AUC-ROC — Overfitting Check",
        x="Model",y="AUC-ROC") + ylim(0,1.15)
save_fig(fig22,"Fig22_overfit_check",width=12,height=7)

fig23 <- test_results %>%
  select(Model,Accuracy,Precision,Recall,F1_Score,AUC_ROC,Specificity) %>%
  pivot_longer(-Model,names_to="Metric",values_to="Value") %>%
  ggplot(aes(x=Metric,y=Model,fill=Value)) +
  geom_tile(color="white",linewidth=0.8) +
  geom_text(aes(label=round(Value,3)),fontface="bold",size=4) +
  scale_fill_gradient2(low="#E74C3C",mid="#F9E79F",high="#2ECC71",
    midpoint=0.80,name="Score") +
  theme_minimal(base_size=12) +
  theme(plot.title=element_text(face="bold",hjust=0.5,size=13),
        axis.text.x=element_text(angle=30,hjust=1)) +
  labs(title="Fig 23: Model Performance Heatmap — Test Set",
        x="Metric",y="Model")
save_fig(fig23,"Fig23_metric_heatmap_test",width=12,height=7)

# STEP 7: Regression Models + SHAP Explainability

```

```

eval_reg <- function(pred, actual, model_name) {
  rmse <- sqrt(mean((pred-actual)^2))
  mae <- mean(abs(pred-actual))
  r2 <- 1 - sum((pred-actual)^2)/sum((actual-mean(actual))^2)
  data.frame(Model=model_name, RMSE=round(rmse,4), MAE=round(mae,4), R2=round(r2,4))
}

ctrl_reg <- trainControl(method="cv",number=5)

cat("\n Reg Model 1: Linear Regression \n")
set.seed(123)
model_lm <- train(x=X_train,y=y_train_reg,method="lm",trControl=ctrl_reg)
cat("LM CV R^2:",round(max(model_lm$results$Rsquared),4),"\n")
pred_lm_val <- predict(model_lm,X_val)
pred_lm_test <- predict(model_lm,X_test)

cat("\n VIF Check \n")
vif_df <- data.frame(X_train, y=y_train_reg)
lm_vif <- lm(y~.,data=vif_df)
vif_vals <- tryCatch({
  v <- car::vif(lm_vif)
  data.frame(Feature=names(v),VIF=round(v,3)) %>% arrange(desc(VIF))
}, error=function(e) { cat("VIF error:",e$message,"\n"); NULL })
if (!is.null(vif_vals)) {
  print(vif_vals); save_table(vif_vals,"vif_results")
  high_vif <- vif_vals %>% filter(VIF>10)
  if (nrow(high_vif)>0) { cat("High VIF features:\n"); print(high_vif) }
}

lm_resid <- pred_lm_test - y_test_reg
sw_test <- shapiro.test(sample(lm_resid, min(5000,length(lm_resid))))
cat("Shapiro-Wilk: W=",round(sw_test$statistic,4),"| p=",round(sw_test$p.value,4),"\n")
save_table(data.frame(Test="Shapiro-Wilk",W=round(sw_test$statistic,4),
  p_value=round(sw_test$p.value,4),
  Normal=ifelse(sw_test$p.value>=0.05,"Yes","No")),
  "residual_normality_test")

cat("\n Reg Model 2: Ridge Regression \n")
set.seed(123)
model_ridge <- train(x=X_train,y=y_train_reg,method="glmnet",trControl=ctrl_reg,
  tuneGrid=expand.grid(alpha=0,lambda=10^seq(-4,2,length=50)))
cat("Ridge best lambda:",round(model_ridge$bestTune$lambda,5),"\n")
pred_ridge_val <- predict(model_ridge,X_val)
pred_ridge_test <- predict(model_ridge,X_test)

cat("\n Reg Model 3: Lasso Regression \n")
set.seed(123)
model_lasso <- train(x=X_train,y=y_train_reg,method="glmnet",trControl=ctrl_reg,
  tuneGrid=expand.grid(alpha=1,lambda=10^seq(-4,2,length=50)))
cat("Lasso best lambda:",round(model_lasso$bestTune$lambda,5),"\n")
lasso_coef <- coef(model_lasso$finalModel, s=model_lasso$bestTune$lambda)
lasso_selected <- data.frame(Feature=row.names(lasso_coef),
  Coefficient=round(as.numeric(lasso_coef),5)) %>%
  filter(abs(Coefficient)>0, Feature!="(Intercept)") %>%
  arrange(desc(abs(Coefficient)))

```

```

cat("Lasso selected",nrow(lasso_selected),"features:\n"); print(lasso_selected)
save_table(lasso_selected,"lasso_selected_features")
pred_lasso_val <- predict(model_lasso,X_val)
pred_lasso_test <- predict(model_lasso,X_test)

cat("\n Reg Model 4: Random Forest Regression \n")
set.seed(123)
model_rfr <- train(x=X_train,y=y_train_reg,method="rf",trControl=ctrl_reg,
                  tuneGrid=expand.grid(mtry=c(2,3,4,5,6)),ntree=500)
cat("RF Reg best mtry:",model_rfr$bestTune$mtry,"\n")
pred_rfr_val <- predict(model_rfr,X_val)
pred_rfr_test <- predict(model_rfr,X_test)

cat("\n Reg Model 5: XGBoost Regression \n")
dtrain_reg <- xgb.DMatrix(data=as.matrix(X_train),label=y_train_reg)
dval_reg <- xgb.DMatrix(data=as.matrix(X_val), label=y_val_reg)
dtest_reg <- xgb.DMatrix(data=as.matrix(X_test), label=y_test_reg)

xgb_reg_params <- list(objective="reg:squarederror",eval_metric="rmse",max_depth=4,
                      eta=0.05,subsample=0.8,colsample_bytree=0.8,
                      min_child_weight=1,gamma=0.1)
set.seed(123)
model_xgbr <- xgb.train(params=xgb_reg_params,data=dtrain_reg,nrounds=300,
                      evals=list(train=dtrain_reg,val=dval_reg),verbose=0,
                      early_stopping_rounds=20)
cat("XGBoost Reg best iteration:",model_xgbr$best_iteration,"\n")
pred_xgbr_val <- predict(model_xgbr,dval_reg)
pred_xgbr_test <- predict(model_xgbr,dtest_reg)

cat("\n Regression Results — Validation \n")
reg_val <- bind_rows(
  eval_reg(pred_lm_val, y_val_reg,"Linear Regression"),
  eval_reg(pred_ridge_val, y_val_reg,"Ridge Regression"),
  eval_reg(pred_lasso_val, y_val_reg,"Lasso Regression"),
  eval_reg(pred_rfr_val, y_val_reg,"RF Regression"),
  eval_reg(pred_xgbr_val, y_val_reg,"XGBoost Regression")
)
print(reg_val); save_table(reg_val,"regression_validation_results")

cat("\n Regression Results — Test \n")
reg_test <- bind_rows(
  eval_reg(pred_lm_test, y_test_reg,"Linear Regression"),
  eval_reg(pred_ridge_test, y_test_reg,"Ridge Regression"),
  eval_reg(pred_lasso_test, y_test_reg,"Lasso Regression"),
  eval_reg(pred_rfr_test, y_test_reg,"RF Regression"),
  eval_reg(pred_xgbr_test, y_test_reg,"XGBoost Regression")
)
print(reg_test); save_table(reg_test,"regression_test_results")

# FIG 24: Actual vs Predicted

ap_df <- data.frame(
  Actual = rep(y_test_reg,5),
  Predicted = c(pred_lm_test,pred_ridge_test,pred_lasso_test,pred_rfr_test,pred_xgbr_test),
  Model = rep(c("Linear Regression","Ridge Regression","Lasso Regression",
               "RF Regression","XGBoost Regression"), each=length(y_test_reg))

```

```

)
fig24 <- ggplot(ap_df,aes(x=Actual,y=Predicted,color=Model)) +
  geom_point(alpha=0.6,size=2) +
  geom_abline(slope=1,intercept=0,color="#E74C3C",linewidth=1.2,linetype="dashed") +
  facet_wrap(~Model,ncol=3) +
  scale_color_manual(values=c("Linear Regression"="#3498DB","Ridge Regression"="#9B59B6",
    "Lasso Regression"="#F39C12","RF Regression"="#2ECC71",
    "XGBoost Regression"="#E74C3C")) +
  theme_minimal(base_size=11) +
  theme(legend.position="none",plot.title=element_text(face="bold",hjust=0.5,size=13),
    strip.text=element_text(face="bold")) +
  labs(title="Fig 24: Actual vs Predicted — All Regression Models (Test Set)",
    x="Actual log(CO2 Per Capita)",y="Predicted log(CO2 Per Capita)")
save_fig(fig24,"Fig24_actual_vs_predicted",width=14,height=9)

```

FIG 25: Residual Plots

```

res_df <- data.frame(
  Predicted = c(pred_lm_test,pred_ridge_test,pred_lasso_test,pred_rfr_test,pred_xgbr_test),
  Residual = rep(y_test_reg,5) -
    c(pred_lm_test,pred_ridge_test,pred_lasso_test,pred_rfr_test,pred_xgbr_test),
  Model = rep(c("Linear Regression","Ridge Regression","Lasso Regression",
    "RF Regression","XGBoost Regression"), each=length(y_test_reg))
)
fig25 <- ggplot(res_df,aes(x=Predicted,y=Residual,color=Model)) +
  geom_point(alpha=0.6,size=2) +
  geom_hline(yintercept=0,color="#E74C3C",linewidth=1.1,linetype="dashed") +
  facet_wrap(~Model,ncol=3) +
  scale_color_manual(values=c("Linear Regression"="#3498DB","Ridge Regression"="#9B59B6",
    "Lasso Regression"="#F39C12","RF Regression"="#2ECC71",
    "XGBoost Regression"="#E74C3C")) +
  theme_minimal(base_size=11) +
  theme(legend.position="none",plot.title=element_text(face="bold",hjust=0.5,size=13),
    strip.text=element_text(face="bold")) +
  labs(title="Fig 25: Residual Plots — All Regression Models (Test Set)",
    x="Predicted log(CO2 Per Capita)",y="Residuals")
save_fig(fig25,"Fig25_residual_plots",width=14,height=9)

```

```

png(file.path(output_folder,"figures","Fig26_qqplot_lm_residuals.png"),
  width=800,height=700,res=120)
qqnorm(lm_resid, main="Fig 26: Q-Q Plot — Linear Regression Residuals (Test Set)",
  pch=16,col="#3498DB",cex=0.8)
qqline(lm_resid,col="#E74C3C",lwd=2)
dev.off()

```

```

fig27 <- reg_test %>%
  pivot_longer(-Model,names_to="Metric",values_to="Value") %>%
  ggplot(aes(x=reorder(Model,-Value),y=Value,fill=Model)) +
  geom_bar(stat="identity",width=0.6,alpha=0.85) +
  geom_text(aes(label=round(Value,4)),vjust=-0.4,size=3.8,fontface="bold") +
  facet_wrap(~Metric,scales="free_y") +
  scale_fill_manual(values=c("Linear Regression"="#3498DB","Ridge Regression"="#9B59B6",
    "Lasso Regression"="#F39C12","RF Regression"="#2ECC71",
    "XGBoost Regression"="#E74C3C")) +
  theme_minimal(base_size=11) +
  theme(legend.position="none",plot.title=element_text(face="bold",hjust=0.5,size=13),

```

```

axis.text.x=element_text(angle=25,hjust=1),strip.text=element_text(face="bold")) +
labs(title="Fig 27: Regression Model Comparison — RMSE, MAE, R2 (Test Set)",
x="Model",y="Value")
save_fig(fig27,"Fig27_regression_comparison",width=14,height=7)

# STEP 7B: SHAP Explainability

cat("\n SHAP Explainability \n")

rf_cls_wrapper <- function(object, newdata) predict(object,newdata=newdata,type="prob")[, "High"]
rf_reg_wrapper <- function(object, newdata) predict(object,newdata=newdata)

cat("Computing SHAP for RF Classification...\n")
set.seed(123)
shap_cls <- explain(object=model_rf, X=X_train_sm,
pred_wrapper=rf_cls_wrapper, nsim=50, adjust=TRUE)
shap_cls_df <- as.data.frame(shap_cls)
shap_cls_imp <- data.frame(Feature=names(shap_cls_df),
Mean_SHAP=colMeans(abs(shap_cls_df))) %>%
arrange(desc(Mean_SHAP))
cat("Top 5 SHAP (Classification):\n"); print(head(shap_cls_imp,5))
save_table(shap_cls_imp,"shap_classification_importance")

cat("Computing SHAP for RF Regression...\n")
set.seed(123)
shap_reg <- explain(object=model_rfr, X=X_train,
pred_wrapper=rf_reg_wrapper, nsim=50, adjust=TRUE)
shap_reg_df <- as.data.frame(shap_reg)
shap_reg_imp <- data.frame(Feature=names(shap_reg_df),
Mean_SHAP=colMeans(abs(shap_reg_df))) %>%
arrange(desc(Mean_SHAP))
cat("Top 5 SHAP (Regression):\n"); print(head(shap_reg_imp,5))
save_table(shap_reg_imp,"shap_regression_importance")

top10_cls <- head(shap_cls_imp$Feature,10)
shap_cls_long <- shap_cls_df %>%
select(all_of(top10_cls)) %>%
pivot_longer(everything(),names_to="Feature",values_to="SHAP_Value")

fig28 <- ggplot(shap_cls_long,
aes(x=SHAP_Value,y=reorder(Feature,abs(SHAP_Value)),color=SHAP_Value)) +
geom_jitter(height=0.25,alpha=0.4,size=1.8) +
geom_vline(xintercept=0,color="#2C3E50",linewidth=0.9) +
scale_color_gradient2(low="#3498DB",mid="white",high="#E74C3C",midpoint=0,name="SHAP Value") +
theme_minimal(base_size=12) +
theme(plot.title=element_text(face="bold",hjust=0.5,size=13),legend.position="right") +
labs(title="Fig 28: SHAP Summary — RF Classification (High CO2 Emitter)",
x="SHAP Value (impact on prediction)",y="Feature")
save_fig(fig28,"Fig28_shap_summary_classification",width=11,height=7)

fig29 <- ggplot(head(shap_cls_imp,12),
aes(x=reorder(Feature,Mean_SHAP),y=Mean_SHAP,fill=Mean_SHAP)) +
geom_bar(stat="identity",alpha=0.88) +
coord_flip() +
scale_fill_gradient(low="#AED6F1",high="#1A5276") +
geom_text(aes(label=round(Mean_SHAP,4)),hjust=-0.1,size=3.8,fontface="bold") +

```

```

theme_minimal(base_size=12) +
theme(legend.position="none",plot.title=element_text(face="bold",hjust=0.5,size=13)) +
ylim(0,max(shap_cls_imp$Mean_SHAP[1:12])*1.2) +
labs(title="Fig 29: SHAP Feature Importance — RF Classification",
      x="Feature",y="Mean |SHAP Value|")
save_fig(fig29,"Fig29_shap_bar_classification",width=10,height=7)

top10_reg <- head(shap_reg_imp$Feature,10)
shap_reg_long <- shap_reg_df %>%
  select(all_of(top10_reg)) %>%
  pivot_longer(everything(),names_to="Feature",values_to="SHAP_Value")

fig30 <- ggplot(shap_reg_long,
  aes(x=SHAP_Value,y=reorder(Feature,abs(SHAP_Value)),color=SHAP_Value)) +
  geom_jitter(height=0.25,alpha=0.4,size=1.8) +
  geom_vline(xintercept=0,color="#2C3E50",linewidth=0.9) +
  scale_color_gradient2(low="#9B59B6",mid="white",high="#F39C12",midpoint=0,name="SHAP Value") +
  theme_minimal(base_size=12) +
  theme(plot.title=element_text(face="bold",hjust=0.5,size=13),legend.position="right") +
  labs(title="Fig 30: SHAP Summary — RF Regression (log CO2 Per Capita)",
        x="SHAP Value (impact on prediction)",y="Feature")
save_fig(fig30,"Fig30_shap_summary_regression",width=11,height=7)

fig31 <- ggplot(head(shap_reg_imp,12),
  aes(x=reorder(Feature,Mean_SHAP),y=Mean_SHAP,fill=Mean_SHAP)) +
  geom_bar(stat="identity",alpha=0.88) +
  coord_flip() +
  scale_fill_gradient(low="#D5F5E3",high="#1E8449") +
  geom_text(aes(label=round(Mean_SHAP,4)),hjust=-0.1,size=3.8,fontface="bold") +
  theme_minimal(base_size=12) +
  theme(legend.position="none",plot.title=element_text(face="bold",hjust=0.5,size=13)) +
  ylim(0,max(shap_reg_imp$Mean_SHAP[1:12])*1.2) +
  labs(title="Fig 31: SHAP Feature Importance — RF Regression",
        x="Feature",y="Mean |SHAP Value|")
save_fig(fig31,"Fig31_shap_bar_regression",width=10,height=7)

rf_imp <- varImp(model_rf)$importance %>%
  as.data.frame() %>%
  rownames_to_column("Feature") %>%
  arrange(desc(Overall))
save_table(rf_imp,"rf_variable_importance")

fig32 <- ggplot(head(rf_imp,12),
  aes(x=reorder(Feature,Overall),y=Overall,fill=Overall)) +
  geom_bar(stat="identity",alpha=0.85) +
  coord_flip() +
  scale_fill_gradient(low="#A9DFBF",high="#1E8449") +
  geom_text(aes(label=round(Overall,2)),hjust=-0.1,size=3.8,fontface="bold") +
  theme_minimal(base_size=12) +
  theme(legend.position="none",plot.title=element_text(face="bold",hjust=0.5,size=13)) +
  ylim(0,max(rf_imp$Overall[1:12])*1.18) +
  labs(title="Fig 32: Random Forest Variable Importance (Gini Impurity)",
        x="Feature",y="Importance Score")
save_fig(fig32,"Fig32_rf_variable_importance",width=10,height=7)

cat("Regression results:\n"); print(reg_test)

```

```
cat("\nTop SHAP features (Classification):\n"); print(head(shap_cls_imp,5))
cat("\nTop SHAP features (Regression):\n"); print(head(shap_reg_imp,5))
cat("\nAll figures saved: Fig01 to Fig32\n")
cat("Output folder:", output_folder, "\n")
```